

Formal Verification of BayesianNetworks.jl

Simon Frost

2026-09-30

Contents

BayesianNetworksProofs	2
What is formalised	2
SPEC §61 propositions	3
BayesianNetworksProofs.Schema.Desc	4
ACSets.jl schema JSON	5
BayesianNetworksProofs.Schema.BayesNet	6
Theorems	7
BayesianNetworksProofs.Finite.BayesNet	7
BayesianNetworksProofs.Finite.Evaluation	9
Fibres and marginalisation	9
Partial joints along a list of variables	10
Proposition 1	11
Marginalising the downstream variables out (Proposition 3, closed-world shadow)	12
BayesianNetworksProofs.Finite.OrderedParents	13
BayesianNetworksProofs.Finite.VariableElimination	15
BayesianNetworksProofs.Finite.Intervention	18
BayesianNetworksProofs.Finite.Tensor	20
BayesianNetworksProofs.Finite.Open	22
Composition without quotients	22
Semantics (Proposition 3)	23
Semantics	27
BayesianNetworksProofs.Finite.OpenSemantics	29
BayesianNetworksProofs.Finite.BoundaryCases	34
BayesianNetworksProofs.Markov.Basic	36
BayesianNetworksProofs.Finite.Probability	37
Partial assignments and explicit conditioning	38
Finite posterior distributions and exact feasibility	40
Checked finite records, positions and repeated input slots	43
Finite reference tables and checked resolution	47
Scoped factor marginalization	54

Collect/distribute on a finite junction tree	58
Conditional independence from separator factorization	66
D-separation soundness on a finite DAG	68
Conditional numerical error contracts	71
Nonvacuity and boundary checks	75
Roadmap	78

BayesianNetworksProofs

```
import Mathlib.CategoryTheory.MarkovCategory.Basic
```

Lean 4 / Mathlib formalisation accompanying BayesianNetworks.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every library module is built by `lake build --wfail`; `Audit.lean` prints the axioms of the headline results (only `propext`, `Classical.choice`, `Quot.sound`). The Roadmap has no remaining proof holes.

What is formalised

BayesianNetworks.jl represents a Bayesian network as an ACSet on the schema `SchBayesNet` (SPEC §8) whose semantics is a finite stochastic state $p_G : I \rightarrow \otimes_v X_v$ computed by `joint_distribution` (SPEC §5, §10), with hard interventions `do_intervention` (SPEC §21–§22) and the tensor `otimes` of open networks (SPEC §13). Three layers are formalised:

- **Schema layer** (`Schema/`, Mathlib-free): the ACSet schemas as Lean terms, their well-formedness, and the JSON writer behind `lake exe emit_schema`. The Julia test compares `generate_json_acset_schema(SchBayesNet)` with the committed `schemas/*.schema.json`. The Lean terms and the Julia ACSet's `BasicSchema` values are two independent definitions that `emit_schema --check` and a Julia test check to agree; neither is generated from the other.
- **Finite model** (`Finite/`): `FinBayesNet` (finite variables and mechanisms, finite state spaces, target, parents), kernel families over an arbitrary commutative semiring R (instantiated at $\mathbb{R}_{\geq 0}$ in `Finite/Probability.lean`), Propositions 1, 2, 3 and 4 of SPEC §61, and the open-network closure theorem of SPEC §13 (`Finite/Open.lean`). Ordered-parent refinement and exact bucket variable elimination now connect further parts of the finite model, but neither is a theorem about Julia arrays or ACSet attribute operations.
- **Abstract layer** (`Markov/`): the generic consequences of Mathlib's `MarkovCategory` / `CopyDiscardCategory` axioms that the Julia tests check on the finite-stochastic instance.
- `Schema/Desc.lean`: `SchemaDesc`, decidable `WF`, `Sub`, `NoOutgoing`, `extend`, and `toACSetsJson` in `ACSets.jl`'s schema format.
- `Schema/BayesNet.lean`: the three schemas and their checked structural properties.
- `Finite/BayesNet.lean`: the finite shape, assignments, local/normalised kernels, closedness, topological order and joint.
- `Finite/Evaluation.lean`: Proposition 1, through fibres and marginalisation.
- `Finite/OrderedParents.lean`: contiguous positions, ordered CPT/local-kernel equivalence, coherent axis reindexing and joint normalisation.
- `Finite/VariableElimination.lean`: scoped factors, bucket elimination, exact joint marginalisation and elimination-order independence.
- `Finite/Intervention.lean` and `Finite/Tensor.lean`: Propositions 4 and 2.
- `Finite/Open.lean`: the typed-interface rule, open-network closure and split composition.

- `Finite/OpenSemantics.lean`: own-variable open semantics under partial gluing; the original `osem_compose`, a stronger theorem without locality, normalisation or total matching, and `osem_compose_passthrough` for arbitrary pass-through, with no hypotheses.
- `Finite/BoundaryCases.lean`: the pass-through counterexample to the glue formula, the pass-through formula checked on the same wire, and the mechanism-free copying limitation of subset feet.
- `Markov/Basic.lean`: consequences of Mathlib’s abstract Markov/copy-discard classes.
- `Finite/Probability.lean`: Propositions 1 and 4 at $R := \mathbb{R}_{\geq 0}$.
- `Roadmap.lean`: remaining categorical and representation questions; no unproved declarations.

New implementation-facing mathematics

- `Finite/RawRecords.lean` and `ReferenceTables.lean`: concrete finite records, references and checked positions, with repeated slots interpreted diagonally. Boolean checks imply the compiled local/normalized model; a Julia/JSON translation proof is not asserted.
- `Finite/Assignments.lean` and `Posterior.lean`: explicit conditioning equals indicator likelihood, genuine query distributions, positive normalization and exact zero feasibility.
- `Finite/FactorMarginal.lean` and `JunctionTree.lean`: actual collect/distribute messages, side-of-cut projections, calibrated beliefs and posterior agreement with VE. Structural running intersection and exact factor assignment are the hypotheses, not correct messages. Arbitrary branching and disconnected components have proved working encodings.
- `Finite/ConditionalIndependence.lean` and `DSeparation.lean`: soundness of graph separation in the moralized ancestral DAG. No path separation is defined in terms of numerical independence. The boundary to a concrete Bayes-ball implementation remains explicit.
- `Finite/NumericalContracts.lean`: finite product and posterior error bounds, with a positive evidence-mass floor, and rounded products under local arithmetic contracts.
- `Finite/RefinementExamples.lean`: nonvacuous repeated-slot, graph and disconnected zero-support fixtures, checked without native proof shortcuts.

SPEC §61 propositions

SPEC §61	Lean	Julia (BayesianNetworks.jl)
Prop 1 — BN evaluation, $\llbracket G \rrbracket(x) = \prod_v \kappa_v(x_v \mid x_{\text{pa}(v)})$	<code>sum_joint_eq_one</code> (the product is a distribution), <code>evalSeq_eq_joint</code> (the sequential evaluator computes it)	<code>joint_distribution(bn)</code> , <code>validate(bn; closed=true)</code> , <code>topological_order</code>
Prop 2 — tensor compositionality, $\llbracket A \otimes B \rrbracket = \llbracket A \rrbracket \otimes \llbracket B \rrbracket$	<code>joint_tensor</code> , <code>closed_tensor</code> , <code>normalised_tensor</code> , <code>local_tensor</code> , <code>TopoOrder.tensor</code>	<code>otimes(A, B)</code> of <code>OpenBayesNets</code>
Prop 3 — sequential compositionality, $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$	<code>marg_joint_compose_split</code> , <code>osem_compose</code> , <code>osem_compose_glue</code> (disjoint inputs/outputs), <code>osem_compose_passthrough</code> (arbitrary pass-through)	<code>compose(A, B)</code> , <code>glue(A, B; along)</code> in the categorical package
SPEC §13 revision note — pushout composition preserves “at most one mechanism per variable”	<code>composeNet_target_injective</code> , <code>compose_input_exogenous</code> , <code>compose_exogenous_input</code> , <code>composeTopo</code> , <code>compose</code>	<code>Open(bn; inputs, outputs)</code> , <code>validate(::OpenBayesNet)</code> , <code>compose</code> , <code>glue</code>
Prop 4 — hard intervention, truncated factorisation of $\text{do}(X = x)$	<code>joint_intervene</code> , <code>normalised_intervene</code> , <code>local_cut</code> , <code>sum_joint_intervene_eq_one</code>	<code>do_intervention(bn, :X $\Rightarrow$ x)</code>
Props 5–7 — influence diagrams	<code>InfluenceDiagrams.jl/proofs</code> (depends on this project by path)	<code>InfluenceDiagrams.jl</code>

The Markov-category laws of the kernel calculus itself (`compose`, `otimes`, `mcopy`, `delete`) and the concrete Mathlib `FinStoch` instances are proved in `FiniteKernels.jl/proofs`.

Over a general semiring, the sum-to-one results establish normalisation, not positivity. The probability interpretation additionally requires nonnegative entries (automatic over $\mathbb{R}_{\geq 0}$). Inference results here are unnormalised marginal identities and do not authorise conditioning on zero-probability evidence.

```
namespace BayesianNetworksProofs

/-- Smoke lemma so that an empty project builds and the axiom audit has something to print. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl

end BayesianNetworksProofs
```

BayesianNetworksProofs.Schema.Desc

```
import Lean.Data.Json
```

Schema descriptions — a small, Mathlib-free data type describing the *presentation* of an ACSet schema (objects, homs, attribute types, attributes), together with the decidable well-formedness / inclusion / “no outgoing arrow” predicates that the Julia layer relies on and the JSON writer that reproduces ACSets.jl’s `generate_json_acset_schema` format.

The Lean terms here and the `@present` schemas in `BayesianNetworks.jl` are two independent hand-written definitions, checked to agree rather than generated from one another: `lake exe emit_schema` writes `proofs/schemas/*.schema.json` from the terms in `BayesianNetworksProofs.Schema.BayesNet`, and a Julia test asserts that the `@present` schema matches (ADR 0005).

This module deliberately imports only `Lean.Data.Json` so that the `emit_schema` executable links against Lean core alone (no Mathlib object code is required).

```
namespace BayesianNetworksProofs

open Lean (Json ToJson FromJson)

/-- A generator of arrow shape: a hom 'name : dom → cod' between objects, or an attribute
'name : dom → cod' from an object to an attribute type. -/
structure Arrow where
  name : String
  dom : String
  cod : String
  deriving ToJson, FromJson, DecidableEq, Repr

/-- A presentation of an ACSet schema: object generators, hom generators, attribute-type
generators and attribute generators. Equations are not modelled (the schemas of
'BayesianNetworks.jl' have none). -/
structure SchemaDesc where
  obs : List String
  homs : List Arrow
  attrtypes : List String
  attrs : List Arrow
  deriving ToJson, FromJson, DecidableEq, Repr

namespace SchemaDesc

/-- All generator names of a schema, in the order 'obs, homs, attrtypes, attrs'. -/
def names (S : SchemaDesc) : List String :=
  S.obs ++ S.homs.map Arrow.name ++ S.attrtypes ++ S.attrs.map Arrow.name

/-- Well-formedness: every generator name is distinct; every hom has an object as domain and
codomain; every attribute has an object as domain and an attribute type as codomain. -/
def WF (S : SchemaDesc) : Prop :=
  S.names.Nodup ∧
  (∀ h ∈ S.homs, h.dom ∈ S.obs ∧ h.cod ∈ S.obs) ∧
  (∀ a ∈ S.attrs, a.dom ∈ S.obs ∧ a.cod ∈ S.attrtypes)

instance (S : SchemaDesc) : Decidable S.WF := by unfold WF; infer_instance
```

```

/-- 'Sub S0 S': every generator of 'S0' is a generator of 'S' (generator-wise inclusion, the
shape of a Catlab '@present Sch <: Sch0'). -/
def Sub (S0 S : SchemaDesc) : Prop :=
  (∀ x ∈ S0.obs, x ∈ S.obs) ∧ (∀ h ∈ S0.homs, h ∈ S.homs) ∧
  (∀ x ∈ S0.attrtypes, x ∈ S.attrtypes) ∧ (∀ a ∈ S0.attrs, a ∈ S.attrs)

instance (S0 S : SchemaDesc) : Decidable (Sub S0 S) := by unfold Sub; infer_instance

/-- 'NoOutgoing S0 S': no hom or attribute of 'S' whose domain is an object of 'S0' has a
codomain outside 'S0'. This is the precondition of Catlab's multi-object 'OpenACSetTypes':
the sub-schema 'S0' can then serve as the interface (foot) type of open 'S'-ACSets. -/
def NoOutgoing (S0 S : SchemaDesc) : Prop :=
  (∀ h ∈ S.homs, h.dom ∈ S0.obs → h.cod ∈ S0.obs) ∧
  (∀ a ∈ S.attrs, a.dom ∈ S0.obs → a.cod ∈ S0.attrtypes)

instance (S0 S : SchemaDesc) : Decidable (NoOutgoing S0 S) := by unfold NoOutgoing; infer_instance

/-- Extend a schema by appending generators (the Lean counterpart of '@present Sch <: Sch0'). -/
def extend (S : SchemaDesc) (obs : List String := []) (homs : List Arrow := [])
  (attrtypes : List String := []) (attrs : List Arrow := []) : SchemaDesc where
  obs := S.obs ++ obs
  homs := S.homs ++ homs
  attrtypes := S.attrtypes ++ attrtypes
  attrs := S.attrs ++ attrs

/-- An extension always contains the schema it extends. -/
theorem sub_extend (S : SchemaDesc) (obs homs attrtypes attrs) :
  Sub S (S.extend obs homs attrtypes attrs) := by
  refine ⟨?, ?, ?, ?⟩ <|> intro x hx <|> simp [extend, hx]

```

ACSets.jl schema JSON

generate_json_acset_schema in ACSets/src/serialization/JSONACSets.jl emits

```

{ "version": { "ACSetSchema": "0.0.1", "ACSets": <pkg version> },
  "Ob": [ { "name": .. } ], "Hom": [ { "name", "dom", "codom" } ],
  "AttrType": [ { "name" } ], "Attr": [ { "name", "dom", "codom" } ],
  "equations": [ ] }

```

(codomains are spelled codom; Ob/AttrType entries are objects with a single name). JSON objects are unordered; Lean's `Json.obj` stores keys in an RB-map, so `Json.pretty` prints keys in a fixed (reverse-alphabetical) order. Consumers must compare structurally.

```

/-- The 'ACSetSchema' format version written by ACSets.jl. -/
def acsetSchemaVersion : String := "0.0.1"

/-- The ACSets.jl package version recorded in emitted files (informational; Julia compares
modulo 'version'). -/
def acsetsPkgVersion : String := "0.2.29"

private def arrowJson (a : Arrow) : Json :=
  Json.mkObj [ ("name", a.name), ("dom", a.dom), ("codom", a.cod) ]

private def namedJson (x : String) : Json := Json.mkObj [ ("name", x) ]

/-- Render a schema description in the ACSets.jl 'generate_json_acset_schema' format. -/
def toACSetsJson (S : SchemaDesc) : Json :=
  Json.mkObj
    [ ("version", Json.mkObj [ ("ACSetSchema", acsetSchemaVersion), ("ACSets", acsetsPkgVersion) ]),
      ("Ob", Json.arr (S.obs.map namedJson).toArray),
      ("Hom", Json.arr (S.homs.map arrowJson).toArray),
      ("AttrType", Json.arr (S.attrtypes.map namedJson).toArray),
      ("Attr", Json.arr (S.attrs.map arrowJson).toArray),
      ("equations", Json.arr #[]) ]

```

```

/-- Pretty-printed, deterministic text of 'toACSetsJson' (trailing newline included). -/
def toACSetsJsonString (S : SchemaDesc) : String :=
  (S.toACSetsJson.pretty 100) ++ "\n"

end SchemaDesc

end BayesianNetworksProofs

```

BayesianNetworksProofs.Schema.BayesNet

```
import BayesianNetworksProofs.Schema.Desc
```

The three ACSet schemas of the ecosystem, as SchemaDesc terms:

- schVariableSpace — the interface sub-schema (variables and their states);
- schBayesNet — adds mechanisms and their ordered inputs (SPEC §8);
- schInfluenceDiagram — adds decisions, information arcs, utilities and decision precedence (SPEC §24).

The theorems (all by decide) are what the Julia layer assumes: each schema is well formed, each extension contains its base, and schVariableSpace has *no outgoing arrows* in schBayesNet / schInfluenceDiagram, which is the precondition of Catlab's multi-object OpenACSetTypes(BayesNet, VariableSpace).

```

namespace BayesianNetworksProofs

open SchemaDesc

/-- 'SchVariableSpace': objects 'Variable', 'State'; hom 'state_variable'; attribute types
'Label', 'Position', 'Ref'; attributes naming variables and states, positioning states and
referencing the semantic state space. -/
def schVariableSpace : SchemaDesc where
  obs := ["Variable", "State"]
  homs := [{"state_variable", "State", "Variable"}]
  attrtypes := ["Label", "Position", "Ref"]
  attrs :=
    [ <"variable_name", "Variable", "Label">
      , <"space_ref", "Variable", "Ref">
      , <"state_name", "State", "Label">
      , <"state_position", "State", "Position"> ]

/-- 'SchBayesNet <: SchVariableSpace': mechanisms with a 'target' variable and ordered inputs. -/
def schBayesNet : SchemaDesc :=
  schVariableSpace.extend
    (obs := ["Mechanism", "Input"])
    (homs :=
      [ <"target", "Mechanism", "Variable">
        , <"input_mechanism", "Input", "Mechanism">
        , <"input_variable", "Input", "Variable"> ])
    (attrs :=
      [ <"mechanism_name", "Mechanism", "Label">
        , <"kernel_ref", "Mechanism", "Ref">
        , <"input_position", "Input", "Position"> ])

/-- 'SchInfluenceDiagram <: SchBayesNet': decisions, information inputs, utilities with their
inputs, and decision precedence. -/
def schInfluenceDiagram : SchemaDesc :=
  schBayesNet.extend
    (obs := ["Decision", "InformationInput", "Utility", "UtilityInput", "DecisionPrecedence"])
    (homs :=
      [ <"decision_variable", "Decision", "Variable">
        , <"information_decision", "InformationInput", "Decision">
        , <"information_variable", "InformationInput", "Variable">
        , <"utility_node", "UtilityInput", "Utility">
        , <"utility_variable", "UtilityInput", "Variable">
        , <"earlier", "DecisionPrecedence", "Decision"> ])

```

```

    , <"later", "DecisionPrecedence", "Decision"> ]])
(attrs :=
  [ <"decision_name", "Decision", "Label">
    , <"utility_name", "Utility", "Label">
    , <"utility_ref", "Utility", "Ref">
    , <"information_position", "InformationInput", "Position">
    , <"utility_position", "UtilityInput", "Position"> ]])

```

Theorems

```

theorem schVariableSpace_wf : schVariableSpace.WF := by decide

theorem schBayesNet_wf : schBayesNet.WF := by decide

theorem schInfluenceDiagram_wf : schInfluenceDiagram.WF := by decide

theorem sub_variableSpace_bayesNet : Sub schVariableSpace schBayesNet := by decide

/-- No hom or attribute of 'SchBayesNet' leaves '{Variable, State}': 'VariableSpace' is a valid
interface type for 'OpenACSetTypes(BayesNet, VariableSpace)'. -/
theorem noOutgoing_variableSpace_bayesNet : NoOutgoing schVariableSpace schBayesNet := by decide

theorem sub_bayesNet_influenceDiagram : Sub schBayesNet schInfluenceDiagram := by decide

theorem noOutgoing_variableSpace_influenceDiagram :
  NoOutgoing schVariableSpace schInfluenceDiagram := by decide

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.BayesNet

```

import Mathlib.Data.Fintype.Pi
import Mathlib.Algebra.BigOperators.Group.Finset.Basic

```

Concrete finite Bayesian networks — the semantic model behind `BayesianNetworks.jl`'s finite stochastic evaluator (SPEC §5, §8, §10, §14).

A `FinBayesNet` is a finite type V of variables, a finite type M of mechanisms, a finite non-empty state space $states$ v for each variable, a target variable for each mechanism and a finite set of parents for each mechanism. This is the shape of the `SchBayesNet ACSet (Mechanism --target--> Variable, Input rows collected into parents)` with the attribute data erased; kernels live outside the structure, exactly as `kernel_ref` points outside the `ACSet` in Julia.

Kernels are valued in an arbitrary commutative semiring R (instantiate with $\mathbb{R}_{\geq 0}$ or $\mathbb{R}$): $\kappa_m \times y$ is the weight the mechanism m assigns to the value y of its target given the assignment x . Two properties matter:

- Local $\kappa_m \text{---} \kappa_m \times$ depends on x only through the parents of m (the mechanism is a kernel $\otimes \text{parents} \rightarrow \text{target}$);
- Normalised $\kappa_m \text{---} \sum y, \kappa_m \times y = 1$ (the kernel is a stochastic map).

Closed `bn` (exactly one mechanism per variable, target bijective) is `validate(bn; closed=true)` and `TopoOrder bn` is a topological order of the derived variable graph (SPEC §11 item 7).

The joint $\kappa x = \prod m, \kappa_m \times (x \text{ (target } m))$ is Proposition 1's right-hand side.

```

namespace BayesianNetworksProofs

/-- A finite Bayesian-network *shape*: variables, mechanisms, state spaces, targets, parents. -/
structure FinBayesNet where
  /-- Variables. -/
  V : Type
  /-- Mechanisms (one per 'Mechanism' row of the ACSet). -/
  M : Type

```

```

[fintypeV : Fintype V]
[decV : DecidableEq V]
[fintypeM : Fintype M]
[decM : DecidableEq M]
/-- The finite state space of each variable. -/
states : V → Type
[fintypeS : ∀ v, Fintype (states v)]
[decS : ∀ v, DecidableEq (states v)]
[nonemptyS : ∀ v, Nonempty (states v)]
/-- The variable generated by each mechanism ('target : Mechanism → Variable'). -/
target : M → V
/-- The input variables of each mechanism (the 'Input' rows of the ACSet). -/
parents : M → Finset V

attribute [instance] FinBayesNet.fintypeV FinBayesNet.decV FinBayesNet.fintypeM FinBayesNet.decM
FinBayesNet.fintypeS FinBayesNet.decS FinBayesNet.nonemptyS

namespace FinBayesNet

variable (bn : FinBayesNet)

/-- A joint assignment of a state to every variable. -/
abbrev Assignment := ∀ v, bn.states v

/-- A family of conditional kernels, one per mechanism: 'κ m x y' is the weight of the target
value 'y' given the assignment 'x'. -/
abbrev Kernel (R : Type) := (m : bn.M) → bn.Assignment → bn.states (bn.target m) → R

variable {bn} {R : Type}

/-- 'k' depends on the assignment only through the variables in 'P'. -/
def LocalOn (P : Finset bn.V) {v : bn.V} (k : bn.Assignment → bn.states v → R) : Prop :=
  ∀ x x' : bn.Assignment, (∀ p ∈ P, x p = x' p) → k x = k x'

/-- The kernel of 'm' reads only the parents of 'm'. -/
def Local (κ : bn.Kernel R) (m : bn.M) : Prop := LocalOn (bn.parents m) (κ m)

/-- The kernel of 'm' is a stochastic map: its weights sum to one for every assignment. -/
def Normalised [AddCommMonoid R] [One R] (κ : bn.Kernel R) (m : bn.M) : Prop :=
  ∀ x, ∑ y, κ m x y = 1

/-- Closed network: every variable has exactly one generating mechanism
('validate(bn; closed=true)'). -/
def Closed (bn : FinBayesNet) : Prop := Function.Bijective bn.target

/-- A topological order of the variables: a duplicate-free, complete list in which every parent
of a mechanism precedes the mechanism's target, and no mechanism reads its own target. -/
structure TopoOrder (bn : FinBayesNet) where
  order : List bn.V
  nodup : order.Nodup
  complete : ∀ v, v ∈ order
  -- For 'a' before 'b' in the order, 'b' is not a parent of the mechanism generating 'a'. -/
  parents_before : order.Pairwise (fun a b ⇒ ∀ m, bn.target m = a → b ∉ bn.parents m)
  no_self : ∀ m, bn.target m ∉ bn.parents m

/-- The joint weight of an assignment: the product of all mechanism kernels evaluated at it
(Proposition 1, right-hand side). -/
def joint [CommMonoid R] (κ : bn.Kernel R) (x : bn.Assignment) : R :=
  ∏ m, κ m x (bn.target m)

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Evaluation

```
import BayesianNetworksProofs.Finite.BayesNet
import Mathlib.Algebra.BigOperators.Ring.Finset
```

Proposition 1 (BN evaluation) for the concrete finite model.

- `sum_joint_eq_one` (Prop 1a): for a closed network with local, normalised kernels the joint $\prod m, \kappa m \times (x \text{ (target } m))$ sums to one over all assignments, i.e. it is a probability distribution — the state $p_G : I \rightarrow \otimes X_v$ of SPEC §5.
- `evalSeq_eq_joint` (Prop 1b): the sequential evaluator that walks a topological order and multiplies in the kernels of the mechanisms generating each variable computes the same product.

The proof of 1a integrates the variables out one at a time along the topological order. To stay free of dependent tuples, “integrating out S ” is expressed with fibre $S \times_{x_0}$, the finite set of assignments agreeing with a base assignment x_0 outside S ; `marg S F x0` sums F over that fibre. Adding a variable to S splits the fibre by the value of that variable (`marg_insert`), and locality lets the kernel of the newest variable be pulled out of the sum.

```
namespace BayesianNetworksProofs
```

```
namespace FinBayesNet
```

```
variable {bn : FinBayesNet} {R : Type}
```

Fibres and marginalisation

```
/-- Assignments agreeing with 'x0' outside 'S'. -/
def fibre (S : Finset bn.V) (x0 : bn.Assignment) : Finset bn.Assignment :=
  Finset.univ.filter (fun y => ∀ v, v ∉ S → y v = x0 v)

theorem mem_fibre {S : Finset bn.V} {x0 y : bn.Assignment} :
  y ∈ fibre S x0 ↔ ∀ v, v ∉ S → y v = x0 v := by
  simp [fibre]

theorem fibre_empty (x0 : bn.Assignment) : fibre (∅ : Finset bn.V) x0 = {x0} := by
  ext y
  simp [mem_fibre, funext_iff]

theorem fibre_univ (x0 : bn.Assignment) : fibre (Finset.univ : Finset bn.V) x0 = Finset.univ := by
  ext y
  simp [mem_fibre]

/-- Splitting the fibre of 'insert v0 S' by the value at 'v0'. -/
theorem fibre_insert_filter {S : Finset bn.V} {v0 : bn.V} (hv : v0 ∉ S) (x0 : bn.Assignment)
  (z : bn.states v0) :
  (fibre (insert v0 S) x0).filter (fun y => y v0 = z) = fibre S (Function.update x0 v0 z) := by
  ext y
  simp only [Finset.mem_filter, mem_fibre, Finset.mem_insert, not_or]
  constructor
  · rintro ⟨h, hz⟩ v hvS
    by_cases hvv : v = v0
    · subst hvv
      simp [hz]
    · rw [Function.update_of_ne hvv]
      exact h v ⟨hvv, hvS⟩
  · intro h
    refine ⟨fun v ⟨hvv, hvS⟩ => ?, ?_⟩
    · have := h v hvS
      rwa [Function.update_of_ne hvv] at this
    · have := h v0 hv
      simp using this

variable [CommSemiring R]
```

```

/-- Sum of 'F' over the assignments agreeing with 'x₀' outside 'S'. -/
def marg (S : Finset bn.V) (F : bn.Assignment → R) (x₀ : bn.Assignment) : R :=
  ∑ y ∈ fibre S x₀, F y

theorem marg_empty (F : bn.Assignment → R) (x₀ : bn.Assignment) : marg ∅ F x₀ = F x₀ := by
  simp [marg, fibre_empty]

theorem marg_univ (F : bn.Assignment → R) (x₀ : bn.Assignment) :
  marg Finset.univ F x₀ = ∑ y, F y := by
  simp [marg, fibre_univ]

theorem marg_insert {S : Finset bn.V} {v₀ : bn.V} (hv : v₀ ∉ S) (F : bn.Assignment → R)
  (x₀ : bn.Assignment) :
  marg (insert v₀ S) F x₀ = ∑ z, marg S F (Function.update x₀ v₀ z) := by
  unfold marg
  rw [← Finset.sum_fiberwise (fibre (insert v₀ S) x₀) (fun y ⇒ y v₀) F]
  refine Finset.sum_congr rfl fun z _ => ?_
  rw [fibre_insert_filter hv]

/-- Integrating out 'S ∪ T' (disjoint) is integrating out 'S' and then, inside, 'T': the finite
Fubini theorem for fibres. -/
theorem marg_union_disjoint {S T : Finset bn.V} (hd : Disjoint S T) (F : bn.Assignment → R)
  (x₀ : bn.Assignment) : marg (S ∪ T) F x₀ = marg S (fun y ⇒ marg T F y) x₀ := by
  induction S using Finset.induction_on generalizing x₀ with
  | empty => simp [marg_empty]
  | insert v₀ S hv ih =>
    have hvT : v₀ ∉ T := fun h => (Finset.disjoint_left.1 hd (Finset.mem_insert_self v₀ S)) h
    have hd' : Disjoint S T := hd.mono_left (Finset.subset_insert _ _)
    rw [Finset.insert_union, marg_insert (by simp [hv, hvT]), marg_insert hv]
    exact Finset.sum_congr rfl fun z _ => ih hd' _

/-- A factor that is constant on the fibre comes out of the sum. -/
theorem marg_mul_left {T : Finset bn.V} {F G : bn.Assignment → R} {x₀ : bn.Assignment}
  (hF : ∀ y ∈ fibre T x₀, F y = F x₀) :
  marg T (fun y ⇒ F y * G y) x₀ = F x₀ * marg T G x₀ := by
  unfold marg
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun y hy => by show F y * G y = F x₀ * G y; rw [hF y hy]

```

Partial joints along a list of variables

```

/-- The product of the kernels of the mechanisms whose target lies in 'l'. -/
def partialJoint (κ : bn.Kernel R) (l : List bn.V) (x : bn.Assignment) : R :=
  ∏ m ∈ Finset.univ.filter (fun m => bn.target m ∈ l), κ m x (x (bn.target m))

theorem filter_target_cons (hinj : Function.Injective bn.target) {l : List bn.V} (m₀ : bn.M) :
  Finset.univ.filter (fun m => bn.target m ∈ bn.target m₀ :: l) =
  insert m₀ (Finset.univ.filter (fun m => bn.target m ∈ l)) := by
  ext m
  simp only [Finset.mem_filter, Finset.mem_univ, true_and, List.mem_cons, Finset.mem_insert]
  constructor
  · rintro (h | h)
    · exact Or.inl (hinj h)
    · exact Or.inr h
  · rintro (rfl | h)
    · exact Or.inl rfl
    · exact Or.inr h

/-- The core induction: integrating out the variables of a suffix 'l' of a topological order
(every variable in 'l' has a mechanism, every parent of a mechanism targeting 'l' that lies in
'l' comes earlier) gives total weight one, whatever the values of the other variables. -/
theorem marg_partialJoint_eq_one (κ : bn.Kernel R) (hinj : Function.Injective bn.target)
  (hloc : ∀ m, Local κ m)
  (hself : ∀ m, bn.target m ∉ bn.parents m) (l : List bn.V) (hnd : l.Nodup)

```

```

(hnorm : ∀ m, bn.target m ∈ l → Normalised κ m)
(hpw : l.Pairwise (fun a b ⇒ ∀ m, bn.target m = a → b ∉ bn.parents m))
(hsurj : ∀ v ∈ l, ∃ m, bn.target m = v) (x₀ : bn.Assignment) :
marg l.toFinset (partialJoint κ l) x₀ = 1 := by
induction l generalizing x₀ with
| nil ⇒ simp [marg_empty, partialJoint]
| cons v₀ l ih ⇒
  obtain ⟨m₀, hm₀⟩ := hsurj v₀ (List.mem_cons_self ..)
  subst hm₀
  rw [List.nodup_cons] at hnd
  rw [List.pairwise_cons] at hpw
  have hnotl : bn.target m₀ ∉ l.toFinset := by simp using hnd.1
  rw [List.toFinset_cons, marg_insert hnotl]
  have key : ∀ z, marg l.toFinset (partialJoint κ (bn.target m₀ :: l))
    (Function.update x₀ (bn.target m₀) z) = κ m₀ x₀ z := by
    intro z
    have hfac : ∀ y ∈ fibre l.toFinset (Function.update x₀ (bn.target m₀) z),
      partialJoint κ (bn.target m₀ :: l) y = κ m₀ x₀ z * partialJoint κ l y := by
      intro y hy
      rw [mem_fibre] at hy
      unfold partialJoint
      rw [filter_target_cons hnj, Finset.prod_insert (by simp using hnd.1)]
      congr 1
      have hy0 : y (bn.target m₀) = z := by
        rw [hy _ hnotl, Function.update_self]
      have hk : κ m₀ y = κ m₀ x₀ := by
        refine hloc m₀ y x₀ fun p hp ⇒ ?_
        have hpl : p ∉ l := fun hpl ⇒ hpw.1 p hpl m₀ rfl hp
        have hne : p ≠ bn.target m₀ := fun h ⇒ hself m₀ (h ▶ hp)
        rw [hy p (by simp using hpl), Function.update_of_ne hne]
      rw [hk, hy0]
    unfold marg
    rw [Finset.sum_congr rfl hfac, ← Finset.mul_sum]
    have := ih hnd.2 (fun m hm ⇒ hnorm m (List.mem_cons_of_mem _ hm)) hpw.2
      (fun v hv ⇒ hsurj v (List.mem_cons_of_mem _ hv))
      (Function.update x₀ (bn.target m₀) z)
    unfold marg at this
    rw [this, mul_one]
  rw [Finset.sum_congr rfl fun z _ ⇒ key z]
  exact hnorm m₀ (List.mem_cons_self ..) x₀

```

Proposition 1

```

/-- Proposition 1a. In a closed network with a topological order, local and normalised
kernels make the joint sum to one. A probability interpretation additionally requires
nonnegative entries; this is automatic over  $\mathbb{R}_{\geq 0}$ , but not over a general semiring. -/
theorem sum_joint_eq_one (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
(hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) :
  ∑ x, joint κ x = 1 := by
  obtain ⟨x₀⟩ : Nonempty bn.Assignment := inferInstance
  have h := marg_partialJoint_eq_one κ hclosed.1 hloc ord.no_self ord.order ord.nodup
    (fun m _ ⇒ hnorm m) ord.parents_before (fun v _ ⇒ hclosed.2 v) x₀
  have hu : ord.order.toFinset = Finset.univ :=
    Finset.eq_univ_of_forall fun v ⇒ by simp using ord.complete v
  rw [hu, marg_univ] at h
  rw [← h]
  refine Finset.sum_congr rfl fun x _ ⇒ ?_
  unfold joint partialJoint
  rw [Finset.filter_true_of_mem fun m _ ⇒ ord.complete _]

/-- The sequential evaluator: walk the variables in the given order and multiply in the kernels
of the mechanisms generating each of them. This is the loop 'for v in order' of
'joint_distribution' in Julia. -/
def evalSeq (κ : bn.Kernel R) (l : List bn.V) (x : bn.Assignment) : R :=

```

```

(l.map fun v =>  $\prod$  m ∈ Finset.univ.filter (fun m => bn.target m = v),  $\kappa$  m x (x (bn.target m))).prod

/-- **Proposition 1b.** The sequential evaluator along any complete duplicate-free order of the
variables computes the joint product (no acyclicity or closedness is needed for this identity). -/
theorem evalSeq_eq_joint ( $\kappa$  : bn.Kernel R) (l : List bn.V) (hnd : l.Nodup) (hcomp :  $\forall$  v, v ∈ l)
  (x : bn.Assignment) : evalSeq  $\kappa$  l x = joint  $\kappa$  x := by
  unfold evalSeq joint
  have hu : l.toFinset = Finset.univ := Finset.eq_univ_of_forall fun v => by simp using hcomp v
  rw [← List.prod_toFinset _ hnd, hu]
  exact Finset.prod_fiberwise Finset.univ bn.target _

/-- Proposition 1b specialised to a topological order. -/
theorem evalSeq_topo_eq_joint ( $\kappa$  : bn.Kernel R) (ord : bn.TopoOrder) (x : bn.Assignment) :
  evalSeq  $\kappa$  ord.order x = joint  $\kappa$  x :=
  evalSeq_eq_joint  $\kappa$  ord.order ord.nodup ord.complete x

```

Marginalising the downstream variables out (Proposition 3, closed-world shadow)

```

/-- A set of "upstream" variables 'U' is *closed under parents*: every mechanism generating a
variable of 'U' reads only variables of 'U'. -/
def UpstreamClosed (U : Finset bn.V) : Prop :=
   $\forall$  m, bn.target m ∈ U → bn.parents m  $\subseteq$  U

/-- **Integrating the downstream variables out.** If 'U' is closed under parents, every variable
outside 'U' is generated by some mechanism, and the kernels of those mechanisms are normalised,
then summing the joint over all assignments that agree with 'x0' on 'U' leaves exactly the
product of the kernels of the mechanisms targeting 'U'. Only the *downstream* kernels have to be
normalised; the upstream ones merely have to be local.

```

This is the closed-world shadow of **Proposition 3** (SPEC §61): the downstream half of a sequential composite integrates out to one, so the marginal of '[B ◦ A]' on 'A's variables is '[A]'. See 'BayesianNetworksProofs.Finite.Open' for the open-network form

```

('OpenFinBayesNet.Composable.marg_joint_compose'). -/
theorem marg_joint_downstream ( $\kappa$  : bn.Kernel R) (hij : Function.Injective bn.target)
  (ord : bn.TopoOrder) (hloc :  $\forall$  m, Local  $\kappa$  m) (U : Finset bn.V)
  (hnorm :  $\forall$  m, bn.target m  $\notin$  U → Normalised  $\kappa$  m)
  (hU : UpstreamClosed U) (hmech :  $\forall$  v  $\notin$  U,  $\exists$  m, bn.target m = v) (x0 : bn.Assignment) :
  marg Uc (joint  $\kappa$ ) x0 =
     $\prod$  m ∈ Finset.univ.filter (fun m => bn.target m ∈ U),  $\kappa$  m x0 (x0 (bn.target m)) := by
  set l := ord.order.filter (fun v => decide (v  $\notin$  U)) with hl
  have hmemL :  $\forall$  v, v ∈ l ↔ v  $\notin$  U := by
    intro v
    rw [hl, List.mem_filter]
    simp [ord.complete v]
  have hlt : l.toFinset = Uc := by
    ext v
    simp [hmemL v]
  have key := marg_partialJoint_eq_one  $\kappa$  hij hloc ord.no_self l (ord.nodup.filter _)
    (fun m hm => hnorm m ((hmemL _).1 hm)) (ord.parents_before.filter _)
    (fun v hv => hmech v ((hmemL v).1 hv)) x0
  rw [← hlt]
  unfold marg at key
  have hfac :  $\forall$  y ∈ fibre l.toFinset x0, joint  $\kappa$  y =
    ( $\prod$  m ∈ Finset.univ.filter (fun m => bn.target m ∈ U),  $\kappa$  m x0 (x0 (bn.target m)))
    * partialJoint  $\kappa$  l y := by
    intro y hy
    rw [mem_fibre] at hy
    have hagree :  $\forall$  v ∈ U, y v = x0 v := fun v hv =>
      hy v (by simp [List.mem_toFinset, hmemL v, hv])
    unfold joint partialJoint
    rw [← Finset.prod_filter_mul_prod_filter_not Finset.univ (fun m => bn.target m ∈ U)]
    congr 1
    • refine Finset.prod_congr rfl fun m hm => ?_
      have hmU : bn.target m ∈ U := (Finset.mem_filter.1 hm).2

```

```

    rw [hloc m y x₀ fun p hp ⇒ hagree p (hU m hmU hp), hagree _ hmU]
    · exact Finset.prod_congr (by ext m; simp [hmemL]) fun m _ ⇒ rfl
  rw [Finset.sum_congr rfl hfac, ← Finset.mul_sum, key, mul_one]

/-- Proposition 3 (closed-world shadow). In a closed network with a topological order and local, normalised kernels, integrating the downstream variables out of the full joint leaves the joint of the upstream mechanisms. -/
theorem marg_joint_upstream (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (U : Finset bn.V)
  (hU : UpstreamClosed U) (x₀ : bn.Assignment) :
  marg Uᶜ (joint κ) x₀ =
  ∏ m ∈ Finset.univ.filter (fun m ⇒ bn.target m ∈ U), κ m x₀ (x₀ (bn.target m)) :=
  marg_joint_downstream κ hclosed.1 ord hloc U (fun m _ ⇒ hnorm m) hU
  (fun v _ ⇒ hclosed.2 v) x₀

end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.OrderedParents

```

import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Logic.Equiv.Basic

```

An exact ordered-parent refinement of the finite kernel model (SPEC §8.2, §10.2, §11 and the §61 revision-note qualification).

ParentOrder equips every mechanism with a bijection from contiguous positions $\text{Fin } n$ to its parent variables. Thus each parent occurs exactly once and every position is used. An ordered CPT takes a dependent tuple of states in this order, followed by its child's state. Gathering an assignment into that tuple gives a local kernel. Conversely, every local kernel has an ordered CPT, and the two constructions are inverse.

Changing parent order must be accompanied by a corresponding change in the CPT axes: `toKernel_reindex` proves the precise transport law, including heterogeneous parent state types. `sum_joint_ordered` transfers the existing normalisation theorem to ordered CPTs.

This closes the unordered-versus-ordered *finite function* gap, not the entire ACSet gap. The bijection is supplied as data; this file does not parse Input rows, verify Julia's sorting routine, model `state_position`, resolve `KernelRefs`, or relate real arithmetic to floating-point arrays. Those correspondence claims remain unproved.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet

variable (bn : FinBayesNet)

/-- Contiguous, total and duplicate-free positions for each mechanism's parents.
Julia uses one-based positions; 'Fin n' is their zero-based mathematical index. -/
structure ParentOrder where
  count : bn.M → ℕ
  atPosition : (m : bn.M) → Fin (count m) ≈ {v : bn.V // v ∈ bn.parents m}

namespace ParentOrder

variable {bn} (o : ParentOrder bn) {R : Type}

/-- A tuple of parent states in explicit positional order. -/
abbrev Values (m : bn.M) :=
  (i : Fin (o.count m)) → bn.states (o.atPosition m i).val

/-- The equivalent parent-indexed view of a positional tuple. -/
def valuesEquiv (m : bn.M) :
  o.Values m ≈ ((v : {v : bn.V // v ∈ bn.parents m}) → bn.states v.val) :=
  Equiv.piCongrLeft (fun v : {v : bn.V // v ∈ bn.parents m} ⇒ bn.states v.val) (o.atPosition m)

```

```

/-- Gather a joint assignment according to the declared input positions. -/
def gather (m : bn.M) (x : bn.Assignment) : o.Values m :=
  fun i => x (o.atPosition m i).val

theorem valuesEquiv_gather (m : bn.M) (x : bn.Assignment) :
  o.valuesEquiv m (o.gather m x) = fun v : {v : bn.V // v ∈ bn.parents m} => x v.val := by
  apply (o.valuesEquiv m).symm.injective
  rw [Equiv.symm_apply_apply]
  rfl

/-- Extend parent values to all variables, using the model's nonempty state spaces off-scope. -/
noncomputable def extend (m : bn.M)
  (p : (v : {v : bn.V // v ∈ bn.parents m}) → bn.states v.val) : bn.Assignment :=
  fun v => if hv : v ∈ bn.parents m then p ⟨v, hv⟩ else Classical.choice (bn.nonemptyS v)

theorem gather_extend (m : bn.M) (z : o.Values m) :
  o.gather m (extend m (o.valuesEquiv m z)) = z := by
  have hg : o.gather m (extend m (o.valuesEquiv m z)) =
    (o.valuesEquiv m).symm (o.valuesEquiv m z) := by
    funext i
    simp [gather, extend, (o.atPosition m i).property, valuesEquiv]
  rw [hg, Equiv.symm_apply_apply]

/-- User-facing CPT convention: ordered parent tuple first, child's state last. -/
abbrev CPT (R : Type) :=
  (m : bn.M) → o.Values m → bn.states (bn.target m) → R

/-- Interpret an ordered CPT in the abstract assignment-based model. -/
def toKernel (t : o.CPT R) : bn.Kernel R :=
  fun m x => t m (o.gather m x)

theorem toKernel_local (t : o.CPT R) : ∀ m, Local (o.toKernel t) m := by
  intro m x y h
  apply congrArg (t m)
  funext i
  exact h (o.atPosition m i).val (o.atPosition m i).property

/-- Read a kernel on all possible positional tuples, completing the unused variables. -/
noncomputable def ofKernel (κ : bn.Kernel R) : o.CPT R :=
  fun m z => κ m (extend m (o.valuesEquiv m z))

/-- Locality makes the arbitrary off-parent completion unobservable. -/
theorem toKernel_ofKernel (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) :
  o.toKernel (o.ofKernel κ) = κ := by
  funext m x
  apply hloc m
  intro v hv
  simp [extend, valuesEquiv_gather, hv]

/-- Ordered CPTs are recovered exactly, not only on the assignments used by an evaluator. -/
theorem ofKernel_toKernel (t : o.CPT R) : o.ofKernel (o.toKernel t) = t := by
  funext m z
  change t m (o.gather m (extend m (o.valuesEquiv m z))) = t m z
  rw [gather_extend]

/-- The unordered locality condition is precisely representability by an ordered CPT. -/
theorem local_iff_ordered (κ : bn.Kernel R) :
  (∀ m, Local κ m) ↔ ∃ t : o.CPT R, o.toKernel t = κ := by
  constructor
  · intro h
    exact ⟨o.ofKernel κ, o.toKernel_ofKernel κ h⟩
  · rintro ⟨t, rfl⟩
    exact o.toKernel_local t

/-- Transport new-order tuples into the old order through their common parent-indexed view. -/

```

```

def reindexValues (o' : ParentOrder bn) (m : bn.M) : o'.Values m ≈ o.Values m :=
  (o'.valuesEquiv m).trans (o.valuesEquiv m).symm

theorem reindexValues_gather (o' : ParentOrder bn) (m : bn.M) (x : bn.Assignment) :
  o.reindexValues o' m (o'.gather m x) = o.gather m x := by
  change (o.valuesEquiv m).symm (o'.valuesEquiv m (o'.gather m x)) = _
  rw [valuesEquiv_gather]
  rfl

/-- Permute the CPT's parent axes along with their declared order, leaving the child axis last. -/
def reindexCPT (o' : ParentOrder bn) (t : o.CPT R) : o'.CPT R :=
  fun m z => t m (o.reindexValues o' m z)

/-- **Ordered-parent fidelity:** coherent reindexing of parent positions and CPT axes preserves
every kernel entry, and hence every joint, marginal and intervention computed from the kernel. -/
theorem toKernel_reindex (o' : ParentOrder bn) (t : o.CPT R) :
  o'.toKernel (o.reindexCPT o' t) = o.toKernel t := by
  funext m x
  change t m (o.reindexValues o' m (o'.gather m x)) = t m (o.gather m x)
  rw [reindexValues_gather]

theorem toKernel_normalised [CommSemiring R] (t : o.CPT R)
  (ht : ∀ m z, ∑ a, t m z a = 1) : ∀ m, Normalised (o.toKernel t) m :=
  fun m x => ht m (o.gather m x)

/-- **Proposition 1 for genuinely ordered CPTs.** Normalising the child-last axis produces a
normalised joint on a closed acyclic network, independently of the chosen parent ordering. -/
theorem sum_joint_ordered [CommSemiring R] (t : o.CPT R) (hclosed : bn.Closed)
  (ord : bn.TopoOrder) (ht : ∀ m z, ∑ a, t m z a = 1) :
  ∑ x, joint (o.toKernel t) x = 1 :=
  sum_joint_eq_one _ hclosed ord (o.toKernel_local t) (o.toKernel_normalised t ht)

end ParentOrder

end BayesianNetworksProofs.FinBayesNet

```

BayesianNetworksProofs.Finite.VariableElimination

```

import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.List.Pairwise

```

Bucket variable elimination preserves the exact marginal (SPEC §15–§17, §55.3).

A factor has a finite scope and a value depending only on that scope. Eliminating a variable multiplies only the factors whose scopes contain it, sums that bucket over the variable, and retains all other factors. We prove this actual bucket operation equals the full-enumeration one-variable marginal. Iteration along a duplicate-free list equals the marginal over the whole list, so elimination order does not change the result.

The arithmetic is over any commutative semiring. No positivity or division is used. The result is an **unnormalised** marginal: normalising evidence of zero total weight remains undefined in probability theory and is not justified by this theorem.

This is a scoped finite-function algorithm and an oracle theorem, not a verified Julia array implementation, not a junction-tree theorem, and not decision-variable elimination.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet

variable {bn : FinBayesNet} {R : Type} [CommSemiring R]

/-- A scalar factor together with a sufficient (not necessarily minimal) dependency scope. -/
structure Factor (bn : FinBayesNet) (R : Type) where
  scope : Finset bn.V

```

```

value : bn.Assignment → R
dependsOn : ∀ x y, (∀ v ∈ scope, x v = y v) → value x = value y

namespace Factor

/-- Product of a list of factors, keeping repeated factors and their multiplicities. -/
def product (fs : List (Factor bn R)) (x : bn.Assignment) : R :=
  (fs.map fun f => f.value x).prod

theorem product_nil (x : bn.Assignment) : product ([] : List (Factor bn R)) x = 1 := rfl

theorem product_cons (f : Factor bn R) (fs : List (Factor bn R)) (x : bn.Assignment) :
  product (f :: fs) x = f.value x * product fs x := rfl

/-- The union of all factor scopes. -/
def jointScope (fs : List (Factor bn R)) : Finset bn.V :=
  fs.foldr (fun f S => f.scope ∪ S) ∅

theorem product_local (fs : List (Factor bn R)) (x y : bn.Assignment)
  (h : ∀ v ∈ jointScope fs, x v = y v) : product fs x = product fs y := by
  induction fs with
  | nil => rfl
  | cons f fs ih =>
    rw [product_cons, product_cons, f.dependsOn x y (fun v hv => h v (Finset.mem_union_left _ hv)),
      ih (fun v hv => h v (Finset.mem_union_right _ hv))]

/-- Multiply an entire bucket and sum its distinguished variable out. -/
def sumBucket (v : bn.V) (fs : List (Factor bn R)) : Factor bn R where
  scope := (jointScope fs).erase v
  value x := ∑ a : bn.states v, product fs (Function.update x v a)
  dependsOn x y h := by
    refine Finset.sum_congr rfl fun a _ => ?_
    apply product_local
    intro w hw
    by_cases he : w = v
    • subst w
      simp
    • simp only [Function.update_of_ne he]
      exact h w (Finset.mem_erase.2 ⟨he, hw⟩)

/-- The bucket is selected by membership in a factor's scope, not by factor position. -/
def bucket (v : bn.V) (fs : List (Factor bn R)) : List (Factor bn R) :=
  fs.filter (fun f => decide (v ∈ f.scope))

/-- Factors not containing the eliminated variable. -/
def outside (v : bn.V) (fs : List (Factor bn R)) : List (Factor bn R) :=
  fs.filter (fun f => decide (v ∉ f.scope))

theorem product_partition (v : bn.V) (fs : List (Factor bn R)) (x : bn.Assignment) :
  product fs x = product (outside v fs) x * product (bucket v fs) x := by
  induction fs with
  | nil => simp [outside, bucket, product]
  | cons f fs ih =>
    by_cases hv : v ∈ f.scope <=>
    simp [outside, bucket, product, hv] at * <=> rw [ih] <=> ac_rfl

theorem outside_update (v : bn.V) (fs : List (Factor bn R))
  (x : bn.Assignment) (a : bn.states v) :
  product (outside v fs) (Function.update x v a) = product (outside v fs) x := by
  unfold product
  apply congrArg List.prod
  apply List.map_congr_left
  intro f hf
  have hv : v ∉ f.scope := by simp [outside] using (List.mem_filter.1 hf).2
  apply f.dependsOn
  intro w hw

```

```

have hne : w ≠ v := fun he ⇒ hv (he ▸ hw)
rw [Function.update_of_ne hne]

/-- Replace one bucket by its marginal, preserving every factor outside it. -/
def eliminate (v : bn.V) (fs : List (Factor bn R)) : List (Factor bn R) :=
  sumBucket v (bucket v fs) :: outside v fs

/-- **One elimination step equals brute-force marginalisation**, even with overlapping scopes,
repeated factors, and variables absent from all scopes. -/
theorem eliminate_correct (v : bn.V) (fs : List (Factor bn R)) (x : bn.Assignment) :
  product (eliminate v fs) x = marg {v} (product fs) x := by
  rw [show ({v} : Finset bn.V) = insert v ∅ from rfl, marg_insert (by simp)]
  simp only [marg_empty]
  change (∑ a : bn.states v, product (bucket v fs) (Function.update x v a)) *
    product (outside v fs) x = _
  rw [Finset.sum_mul]
  refine Finset.sum_congr rfl fun a _ => ?_
  rw [product_partition v fs, outside_update, mul_comm]

/-- Successive bucket eliminations. -/
def eliminateAll (fs : List (Factor bn R)) : List bn.V → List (Factor bn R)
| [] => fs
| v :: vs => eliminateAll (eliminate v fs) vs

/-- **Variable-elimination oracle theorem.** A duplicate-free elimination order computes exactly
the full joint product summed over those variables. -/
theorem eliminateAll_correct (fs : List (Factor bn R)) (vs : List bn.V) (hvs : vs.Nodup)
(x : bn.Assignment) :
  product (eliminateAll fs vs) x = marg vs.toFinset (product fs) x := by
  induction vs generalizing fs with
  | nil => simp [eliminateAll, marg_empty]
  | cons v vs ih =>
    have hv : v ∉ vs.toFinset := by simp using (List.nodup_cons.1 hvs).1
    rw [eliminateAll, ih _ (List.nodup_cons.1 hvs).2]
    have he : product (eliminate v fs) = marg {v} (product fs) :=
      funext (eliminate_correct v fs)
    rw [he, ← marg_union_disjoint (Finset.disjoint_singleton_right.2 hv)]
    simp

/-- Any two duplicate-free orders eliminating the same variables have the same marginal. -/
theorem eliminateAll_order_independent (fs : List (Factor bn R)) (vs ws : List bn.V)
(hvs : vs.Nodup) (hws : ws.Nodup) (hset : vs.toFinset = ws.toFinset) :
  product (eliminateAll fs vs) = product (eliminateAll fs ws) := by
  funext x
  rw [eliminateAll_correct fs vs hvs, eliminateAll_correct fs ws hws, hset]

/-- Compile one local mechanism into a factor on its parents and child. -/
def mechanism (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) (m : bn.M) : Factor bn R where
  scope := insert (bn.target m) (bn.parents m)
  value x := κ m x (x (bn.target m))
  dependsOn x y h := by
    rw [hloc m x y (fun v hv => h v (Finset.mem_insert_of_mem hv)),
      h (bn.target m) (Finset.mem_insert_self _ _)]

/-- Compile all mechanisms, retaining each mechanism exactly once. -/
noncomputable def ofKernel (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) : List (Factor bn R) :=
  Finset.univ.toList.map (mechanism κ hloc)

theorem product_ofKernel (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) :
  product (ofKernel κ hloc) = joint κ := by
  funext x
  simp [product, ofKernel, mechanism, joint, List.map_map]

/-- The bucket algorithm on compiled Bayesian-network factors computes the exact joint
marginal. Closedness and stochasticity are needed for a probability interpretation, not for
this semiring identity. -/

```

```

theorem eliminateAll_joint ( $\kappa$  : bn.Kernel R) (hloc :  $\forall$  m, Local  $\kappa$  m)
  (vs : List bn.V) (hvs : vs.Nodup) (x : bn.Assignment) :
  product (eliminateAll (ofKernel  $\kappa$  hloc) vs) x = marg vs.toFinset (joint  $\kappa$ ) x := by
  rw [eliminateAll_correct _ vs hvs, product_ofKernel]

end Factor

end BayesianNetworksProofs.FinBayesNet

```

BayesianNetworksProofs.Finite.Intervention

```
import BayesianNetworksProofs.Finite.Evaluation
```

Proposition 4 (hard intervention) for the concrete finite model (SPEC §21.2, §22).

do($X = a$) replaces the mechanism m_θ generating X by the point mass δ_a , ignoring its former parents. Syntactically this is cut bn m_θ (the Input rows of m_θ are deleted, i.e. parents $m_\theta := \emptyset$); semantically it is intervene κ m_θ a , which overwrites the kernel of m_θ with the indicator of a .

- joint_intervene — the truncated factorisation: the intervened joint is the indicator [$x \ X = a$] times the product of the *other* mechanisms' kernels, which are untouched.
- normalised_intervene, local_cut — the intervened family is again normalised and local for the rewritten network, so Proposition 1 applies to it (sum_joint_intervene_eq_one).

```

namespace BayesianNetworksProofs

namespace FinBayesNet

variable {bn : FinBayesNet} {R : Type}

/-- The syntactic rewrite 'do': the mechanism 'mθ' keeps its target but loses its inputs. -/
def cut (bn : FinBayesNet) (mθ : bn.M) : FinBayesNet :=
  { bn with parents := Function.update bn.parents mθ  $\emptyset$  }

@[simp] theorem cut_parents (mθ m : bn.M) :
  (bn.cut mθ).parents m = Function.update bn.parents mθ  $\emptyset$  m := rfl

/-- A topological order of 'bn' is one of 'bn.cut mθ' (parents only shrink). -/
def TopoOrder.cut (ord : bn.TopoOrder) (mθ : bn.M) : (bn.cut mθ).TopoOrder where
  order := ord.order
  nodup := ord.nodup
  complete := ord.complete
  parents_before := ord.parents_before.imp fun {a b} h (m : bn.M) hm  $\Rightarrow$  by
    show b  $\notin$  Function.update bn.parents mθ  $\emptyset$  m
    by_cases hmm : m = mθ
    • subst hmm
      simp
    • rw [Function.update_of_ne hmm]
      exact h m hm
  no_self := fun (m : bn.M)  $\Rightarrow$  by
    show bn.target m  $\notin$  Function.update bn.parents mθ  $\emptyset$  m
    by_cases hmm : m = mθ
    • subst hmm
      simp
    • rw [Function.update_of_ne hmm]
      exact ord.no_self m

theorem closed_cut (hclosed : bn.Closed) (mθ : bn.M) : (bn.cut mθ).Closed := hclosed

variable [CommSemiring R]

/-- The semantic rewrite: replace the kernel of 'mθ' by the point mass at 'a'. -/
def intervene ( $\kappa$  : bn.Kernel R) (mθ : bn.M) (a : bn.states (bn.target mθ)) : bn.Kernel R :=
  Function.update  $\kappa$  mθ (fun _ y  $\Rightarrow$  if y = a then 1 else 0)

```

```

@[simp] theorem intervene_self (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) (y : bn.states (bn.target m₀)) :
  intervene κ m₀ a m₀ x y = if y = a then 1 else 0 := by
  simp [intervene]

theorem intervene_of_ne (κ : bn.Kernel R) {m₀ m : bn.M} (h : m ≠ m₀)
  (a : bn.states (bn.target m₀)) : intervene κ m₀ a m = κ m := by
  simp [intervene, Function.update_of_ne h]

/-- Proposition 4 (truncated factorisation). -/
theorem joint_intervene (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) :
  joint (intervene κ m₀ a) x =
  (if x (bn.target m₀) = a then 1 else 0) *
  ∏ m ∈ Finset.univ.erase m₀, κ m x (x (bn.target m)) := by
  unfold joint
  rw [← Finset.mul_prod_erase Finset.univ _ (Finset.mem_univ m₀)]
  congr 1
  · simp
  · exact Finset.prod_congr rfl fun m hm => by
    rw [intervene_of_ne κ (Finset.ne_of_mem_erase hm)]

/-- The point-mass kernel is normalised. -/
theorem normalised_intervene_self (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀)) :
  Normalised (intervene κ m₀ a) m₀ := by
  intro x
  simp

/-- Intervening preserves normalisation of every mechanism. -/
theorem normalised_intervene (κ : bn.Kernel R) (hnorm : ∀ m, Normalised κ m) (m₀ : bn.M)
  (a : bn.states (bn.target m₀)) : ∀ m, Normalised (intervene κ m₀ a) m := by
  intro m
  by_cases h : m = m₀
  · subst h
    exact normalised_intervene_self κ m a
  · intro x
    rw [intervene_of_ne κ h]
    exact hnorm m x

/-- The point-mass kernel reads no parents at all. -/
theorem localOn_empty_intervene (κ : bn.Kernel R) (m₀ : bn.M) (a : bn.states (bn.target m₀)) :
  LocalOn ∅ (intervene κ m₀ a m₀) := by
  intro x x' _
  funext y
  simp

/-- The intervened family is local for the rewritten network 'bn.cut m₀'. -/
theorem local_cut (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) (m₀ : bn.M)
  (a : bn.states (bn.target m₀)) :
  ∀ m, Local (bn := bn.cut m₀) (intervene κ m₀ a) m := by
  refine fun (m : bn.M) (x x' : bn.Assignment) hx => ?_
  by_cases h : m = m₀
  · subst h
    exact localOn_empty_intervene κ m a x x' (fun p hp => absurd hp (Finset.notMem_empty p))
  · show intervene κ m₀ a m x = intervene κ m₀ a m x'
    rw [intervene_of_ne κ h]
    refine hloc m x x' fun p hp => hx p ?_
    show p ∈ Function.update bn.parents m₀ ∅ m
    rw [Function.update_of_ne h]
    exact hp

/-- The intervened joint is again a probability distribution (Proposition 1 applied to the
rewritten network). -/
theorem sum_joint_intervene_eq_one (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (m₀ : bn.M)

```

```

(a : bn.states (bn.target m₀)) :
  ∑ x, joint (intervene κ m₀ a) x = 1 :=
sum_joint_eq_one (bn := bn.cut m₀) (intervene κ m₀ a) (closed_cut hclosed m₀) (ord.cut m₀)
(local_cut κ hloc m₀ a) (normalised_intervene κ hnorm m₀ a)

```

```
end FinBayesNet
```

```
end BayesianNetworksProofs
```

BayesianNetworksProofs.Finite.Tensor

```

import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Fintype.Sum
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Finset.Sum

```

Proposition 2 (tensor compositionality) for the concrete finite model (SPEC §13.1).

The tensor $bn_1.tensor\ bn_2$ of two networks has variables $V_1 \oplus V_2$ and mechanisms $M_1 \oplus M_2$ (the disjoint union of the two ACSets, $\oplus$ of OpenBayesNets with no shared feet). Its kernels are the two families side by side, and `joint_tensor` shows that the joint of the tensor is the product of the two joints — $\llbracket A \oplus B \rrbracket = \llbracket A \rrbracket \otimes \llbracket B \rrbracket$ in the finite stochastic semantics. The tensor of closed / local / normalised / topologically ordered networks is again so.

```
namespace BayesianNetworksProofs
```

```
namespace FinBayesNet
```

```
-- Disjoint union of two networks. Reducible so that 'simp'/'rw' see 'V = V₁ ⊕ V₂' etc. -/
```

```
@[reducible] def tensor (bn₁ bn₂ : FinBayesNet) : FinBayesNet where
```

```
  V := bn₁.V ⊕ bn₂.V
```

```
  M := bn₁.M ⊕ bn₂.M
```

```
  states := Sum.elim bn₁.states bn₂.states
```

```
  fintypeS := fun
```

```
    | .inl v => bn₁.fintypeS v
```

```
    | .inr v => bn₂.fintypeS v
```

```
  decS := fun
```

```
    | .inl v => bn₁.decS v
```

```
    | .inr v => bn₂.decS v
```

```
  nonemptyS := fun
```

```
    | .inl v => bn₁.nonemptyS v
```

```
    | .inr v => bn₂.nonemptyS v
```

```
  target := Sum.map bn₁.target bn₂.target
```

```
  parents := fun
```

```
    | .inl m => (bn₁.parents m).map Function.Embedding.inl
```

```
    | .inr m => (bn₂.parents m).map Function.Embedding.inr
```

```
variable {bn₁ bn₂ : FinBayesNet} {R : Type}
```

```
@[simp] theorem tensor_target_inl (m : bn₁.M) :
```

```
(bn₁.tensor bn₂).target (.inl m) = .inl (bn₁.target m) := rfl
```

```
@[simp] theorem tensor_target_inr (m : bn₂.M) :
```

```
(bn₁.tensor bn₂).target (.inr m) = .inr (bn₂.target m) := rfl
```

```
@[simp] theorem tensor_parents_inl (m : bn₁.M) :
```

```
(bn₁.tensor bn₂).parents (.inl m) = (bn₁.parents m).map Function.Embedding.inl := rfl
```

```
@[simp] theorem tensor_parents_inr (m : bn₂.M) :
```

```
(bn₁.tensor bn₂).parents (.inr m) = (bn₂.parents m).map Function.Embedding.inr := rfl
```

```
@[simp] theorem inl_mem_map_inl {α β : Type} {s : Finset α} {a : α} :
```

```
Sum.inl a ∈ s.map (Function.Embedding.inl (β := β)) ↔ a ∈ s := by
```

```
simp [Finset.mem_map]
```

```

@[simp] theorem inr_mem_map_inr {α β : Type} {s : Finset β} {b : β} :
  Sum.inr b ∈ s.map (Function.Embedding.inr (α := α)) ↔ b ∈ s := by
  simp [Finset.mem_map]

@[simp] theorem inr_notMem_map_inl {α β : Type} {s : Finset α} {b : β} :
  Sum.inr b ∉ s.map (Function.Embedding.inl (β := β)) := by
  simp [Finset.mem_map]

@[simp] theorem inl_notMem_map_inr {α β : Type} {s : Finset β} {a : α} :
  Sum.inl a ∉ s.map (Function.Embedding.inr (α := α)) := by
  simp [Finset.mem_map]

/-- Restrict an assignment of the tensor to the left factor. -/
def Assignment.left (x : (bn₁.tensor bn₂).Assignment) : bn₁.Assignment := fun v ⇒ x (.inl v)

/-- Restrict an assignment of the tensor to the right factor. -/
def Assignment.right (x : (bn₁.tensor bn₂).Assignment) : bn₂.Assignment := fun v ⇒ x (.inr v)

/-- Two kernel families side by side. -/
def tensorKernel (κ₁ : bn₁.Kernel R) (κ₂ : bn₂.Kernel R) : (bn₁.tensor bn₂).Kernel R := fun
| .inl m ⇒ fun x y ⇒ κ₁ m (Assignment.left x) y
| .inr m ⇒ fun x y ⇒ κ₂ m (Assignment.right x) y

/-- Proposition 2. The joint of the tensor is the product of the joints. -/
theorem joint_tensor [CommMonoid R] (κ₁ : bn₁.Kernel R) (κ₂ : bn₂.Kernel R)
  (x : (bn₁.tensor bn₂).Assignment) :
  joint (tensorKernel κ₁ κ₂) x = joint κ₁ (Assignment.left x) * joint κ₂ (Assignment.right x) := by
  unfold joint
  exact Fintype.prod_sum_type _

theorem closed_tensor (h₁ : bn₁.Closed) (h₂ : bn₂.Closed) : (bn₁.tensor bn₂).Closed :=
  h₁.sumMap h₂

theorem normalised_tensor [AddCommMonoid R] [One R] (κ₁ : bn₁.Kernel R) (κ₂ : bn₂.Kernel R)
  (h₁ : ∀ m, Normalised κ₁ m) (h₂ : ∀ m, Normalised κ₂ m) :
  ∀ m, Normalised (tensorKernel κ₁ κ₂) m
| .inl m ⇒ fun x ⇒ h₁ m (Assignment.left x)
| .inr m ⇒ fun x ⇒ h₂ m (Assignment.right x)

theorem local_tensor (κ₁ : bn₁.Kernel R) (κ₂ : bn₂.Kernel R)
  (h₁ : ∀ m, Local κ₁ m) (h₂ : ∀ m, Local κ₂ m) :
  ∀ m, Local (tensorKernel κ₁ κ₂) m
| .inl m ⇒ fun x x' hx ⇒
  h₁ m (Assignment.left x) (Assignment.left x') fun p hp ⇒
  hx (.inl p) (Finset.mem_map_of_mem _ hp)
| .inr m ⇒ fun x x' hx ⇒
  h₂ m (Assignment.right x) (Assignment.right x') fun p hp ⇒
  hx (.inr p) (Finset.mem_map_of_mem _ hp)

/-- Concatenating topological orders gives a topological order of the tensor. -/
def TopoOrder.tensor (o₁ : bn₁.TopoOrder) (o₂ : bn₂.TopoOrder) : (bn₁.tensor bn₂).TopoOrder where
  order := o₁.order.map Sum.inl ++ o₂.order.map Sum.inr
  nodup := by
    refine List.Nodup.append (o₁.nodup.map Sum.inl_injective) (o₂.nodup.map Sum.inr_injective) ?_
    intro a h₁ h₂
    obtain ⟨v, _, rfl⟩ := List.mem_map.1 h₁
    obtain ⟨w, _, h⟩ := List.mem_map.1 h₂
    exact Sum.inr_ne_inl h
  complete := fun
  | .inl v ⇒ List.mem_append_left _ (List.mem_map_of_mem (o₁.complete v))
  | .inr v ⇒ List.mem_append_right _ (List.mem_map_of_mem (o₂.complete v))
  parents_before := by
    refine List.pairwise_append.2 (List.pairwise_map.2 (o₁.parents_before.imp ?_),
      List.pairwise_map.2 (o₂.parents_before.imp ?_), ?_)
    · rintro a b h (m | m) hm
    · simp using h m (Sum.inl_injective hm)

```

```

    • simp at hm
  • rintro a b h (m | m) hm
    • simp at hm
    • simp using h m (Sum.inr_injective hm)
  • rintro a ha b hb (m | m) hm
    • obtain ⟨w, _, rfl⟩ := List.mem_map.1 hb
      simp
    • obtain ⟨v, _, rfl⟩ := List.mem_map.1 ha
      simp at hm
no_self := fun
| .inl m ⇒ by simp using o₁.no_self m
| .inr m ⇒ by simp using o₂.no_self m
end FinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.Open

```
import BayesianNetworksProofs.Finite.Tensor
```

Open networks and the closure theorem (SPEC §13 and its revision note, §55.5, §61 Propositions 2 and 3), for the concrete finite model — no category theory, no quotient types.

An `OpenFinBayesNet` is a `FinBayesNet` with two distinguished sets of variables, inputs and outputs, satisfying the finite-model part of the *typed-interface rule* of `CategoricalBayesianNetworks.jl`'s `Open` (`src/open.jl`, `validation_errors(::OpenBayesNet)`):

Julia rule	Lean field
at most one mechanism per variable (<code>validate(bn; closed = false)</code>)	<code>target_inj</code>
rule 1: input-foot variables have no mechanism	<code>input_exogenous</code>
rule 3: every mechanism-free apex variable is an input	<code>exogenous_input</code>
rule 4: the derived graph is acyclic	<code>topo</code> (a <code>TopoOrder</code>)

Rule 2 (the input leg is injective) is encoded by taking feet to be *subsets* of apex variables. Rule 5's name-, reference- and position-preserving attribute conditions are **not** modelled: those attributes are erased. The interface match of `compose` is the injection $\imath$ below; `states_equiv` transports abstract state types, but does not assert preservation of a `state_position` attribute. Noninjective output legs are also absent from this representation.

Composition without quotients

`Composable A B` is an injection $\imath$ from `B`'s inputs into `A`'s outputs together with the matching of their state spaces. The pushout of the two structured cospans identifies each input of `B` with its partner output of `A`; rather than quotient $A.V \oplus B.V$, the composite takes

- $V := A.V \oplus \text{Priv } B$ where $\text{Priv } B = \{v : B.V \mid v \notin B.\text{inputs}\}$ — one representative per class, the glued variables being represented on the `A` side;
- $M := A.M \oplus B.M$ — mechanisms are never identified;
- $\text{tr} : B.V \rightarrow A.V \oplus \text{Priv } B$, injective, sending an input of `B` to `Sum.inl` ($\imath \dots$) and any other variable to `Sum.inr` — the right-hand pushout leg.

`composeNet` is the resulting shape and `compose` bundles it back into an `OpenFinBayesNet`, which is the closure theorem: producing the four fields *is* the proof that the composite is again valid. The individual statements are

- `composeNet_target_injective` — **at most one mechanism per variable is preserved**. The key step is that a mechanism of `B` never targets a glued variable, because `B`'s inputs are exogenous (`target_notMem_inputs`); so the two mechanism families cannot collide;
- `compose_input_exogenous`, `compose_exogenous_input` — the composite's inputs are exactly `A.inputs`, and every variable contributed by `B` keeps its mechanism;

- `composeTopo` — **acyclicity is preserved**: A's order followed by B's order restricted to `Priv B` is a topological order of the composite. It is *derived*, not assumed.

Outputs are $(A.outputs \setminus \text{glued}) \cup \text{tr } (B.outputs)$, exactly as in Julia's glue.

Semantics (Proposition 3)

`composeKernel` puts two kernel families side by side; `restrictA` and `restrictB` read an assignment of the composite as an assignment of A and of B (a glued variable is read on the A side and transported by `states_equiv`).

- `joint_compose` — $\llbracket B \circ A \rrbracket = \llbracket A \rrbracket \cdot \llbracket B \rrbracket$ pointwise: the joint of the composite factors as the product of the two joints. This is the sequential analogue of `Finite/Tensor.lean's joint_tensor`, and the algebraic heart of Proposition 3;
- `local_compose`, `normalised_compose`, `closed_compose`, `sum_joint_compose_eq_one` — the composite of local / normalised kernels is local / normalised, and if A has no inputs the composite is closed, so its joint is a probability distribution;
- `marg_joint_compose` — summing the composite joint over all values of B's private variables returns A's joint: $\llbracket B \circ A \rrbracket$ marginalised back onto A is $\llbracket A \rrbracket$. Only B's kernels need to be normalised;
- `marg_joint_compose_split` — **Proposition 3**. Integrating out $S \cup T$, with S on the A side and T on the B side, is $\sum_S \llbracket A \rrbracket \cdot (\sum_T \llbracket B \rrbracket)$: the composite semantics is the sum over the interface of the product of the two open semantics. Taking S to be A's hidden variables together with the glued interface and T to be B's hidden variables gives $\llbracket B \circ A \rrbracket = \llbracket B \rrbracket \circ \llbracket A \rrbracket$ when neither network has pass-through variables. The transfer to the open semantics of A and B on their own variable types is proved in `Finite/OpenSemantics.lean`, with partial matching and, in `osem_compose_passthrough`, arbitrary pass-through on either side.

```
namespace BayesianNetworksProofs

structure OpenFinBayesNet extends FinBayesNet where
  inputs : Finset V
  outputs : Finset V
  target_inj : Function.Injective target
  input_exogenous : ∀ v ∈ inputs, ∀ m, target m ≠ v
  exogenous_input : ∀ v, (∀ m, target m ≠ v) → v ∈ inputs
  topo : FinBayesNet.TopoOrder toFinBayesNet

namespace OpenFinBayesNet

open FinBayesNet

variable {A B : OpenFinBayesNet}

/-- Rule 1 restated: a mechanism never targets an input. -/
theorem target_notMem_inputs (O : OpenFinBayesNet) (m : O.M) : O.target m ∉ O.inputs :=
  fun h => O.input_exogenous _ h m rfl

/-- Rule 3 restated: a non-input variable is generated by some mechanism. -/
theorem exists_mechanism (O : OpenFinBayesNet) {v : O.V} (h : v ∉ O.inputs) :
  ∃ m, O.target m = v := by
  by_contra hc
  exact h (O.exogenous_input v (fun m hm => hc ⟨m, hm⟩))

/-- The hidden (non-interface) variables of an open network. -/
def hidden (O : OpenFinBayesNet) : Finset O.V := (O.inputs ∪ O.outputs)ᶜ

/-- The open semantics '[0]' of an open network: the joint with the hidden variables summed out, a function of the interface (input and output) variables alone. -/
def osem {R : Type} [CommSemiring R] (O : OpenFinBayesNet) (κ : O.Kernel R)
  (x : O.Assignment) : R := marg O.hidden (joint κ) x

/-- The variables of 'B' that are not inputs – one representative per pushout class on the 'B' side. -/
abbrev Priv (B : OpenFinBayesNet) := {v : B.V // v ∉ B.inputs}

/-- 'some' on the non-input variables of 'B', 'none' on the inputs. -/
def privOf (v : B.V) : Option (Priv B) := if h : v ∈ B.inputs then none else some ⟨v, h⟩
```

```

/-- 'privOf' picks out exactly the non-input variables. -/
theorem privOf_eq_some {v : B.V} {w : Priv B} : privOf v = some w ↔ v = w.val := by
  unfold privOf
  by_cases h : v ∈ B.inputs
  · simp only [dif_pos h, reduceCtorEq, false_iff]
    rintro rfl
    exact w.property h
  · simp [dif_neg h, Subtype.ext_iff]

/-- An interface match: an injection of 'B''s inputs into 'A''s outputs (Catlab's gluing map,
validated by 'interface_matches' in Julia) together with the identification of the state spaces
it pairs up. -/
structure Composable (A B : OpenFinBayesNet) where
  ι : {v : B.V // v ∈ B.inputs} → {v : A.V // v ∈ A.outputs}
  ι_inj : Function.Injective ι
  states_equiv : ∀ w : {v : B.V // v ∈ B.inputs}, A.states (ι w).val ≈ B.states w.val

namespace Composable

/-- The right-hand pushout leg: 'B''s variables inside the composite. -/
def tr (c : Composable A B) (v : B.V) : A.V ⊕ Priv B :=
  if h : v ∈ B.inputs then Sum.inl (c.ι ⟨v, h⟩).val else Sum.inr ⟨v, h⟩

variable {c : Composable A B}

theorem tr_of_mem {v : B.V} (h : v ∈ B.inputs) : c.tr v = Sum.inl (c.ι ⟨v, h⟩).val := dif_pos h

theorem tr_of_notMem {v : B.V} (h : v ∉ B.inputs) : c.tr v = Sum.inr ⟨v, h⟩ := dif_neg h

/-- The right leg is injective: two 'B' variables are identified in the composite only if they
were equal. -/
theorem tr_injective : Function.Injective c.tr := by
  intro v w h
  by_cases hv : v ∈ B.inputs <=> by_cases hw : w ∈ B.inputs
  · rw [tr_of_mem hv, tr_of_mem hw, Sum.inl.injEq] at h
    exact congrArg Subtype.val (c.ι_inj (Subtype.ext h))
  · rw [tr_of_mem hv, tr_of_notMem hw] at h
    exact absurd h (by simp)
  · rw [tr_of_notMem hv, tr_of_mem hw] at h
    exact absurd h (by simp)
  · rw [tr_of_notMem hv, tr_of_notMem hw, Sum.inr.injEq] at h
    exact congrArg Subtype.val h

variable (c) in
/-- 'tr' as an embedding, for 'Finset.map'. -/
def trEmb : B.V ↪ A.V ⊕ Priv B := ⟨c.tr, tr_injective⟩

@[simp] theorem trEmb_apply (v : B.V) : c.trEmb v = c.tr v := rfl

@[simp] theorem inr_mem_map_trEmb {s : Finset B.V} {w : Priv B} :
  Sum.inr w ∈ s.map c.trEmb ↔ w.val ∈ s := by
  simp only [Finset.mem_map, trEmb_apply]
  constructor
  · rintro ⟨p, hp, h⟩
    by_cases hpi : p ∈ B.inputs
    · rw [tr_of_mem hpi] at h
      exact absurd h (by simp)
    · rw [tr_of_notMem hpi, Sum.inr.injEq] at h
      rw [← h]
      exact hp
  · exact fun h => ⟨w.val, h, tr_of_notMem w.property⟩

variable (c) in
/-- The 'A'-outputs that are glued to inputs of 'B'. -/
def glued : Finset A.V := Finset.univ.image (fun w : {v : B.V // v ∈ B.inputs} => (c.ι w).val)

```

```

variable (c) in
/-- The underlying shape of the composite. -/
@[reducible] def composeNet : FinBayesNet where
  V := A.V ⊕ Priv B
  M := A.M ⊕ B.M
  states := Sum.elim A.states (fun w ⇒ B.states w.val)
  fintypeS := fun
    | .inl v ⇒ A.fintypeS v
    | .inr w ⇒ B.fintypeS w.val
  decS := fun
    | .inl v ⇒ A.decS v
    | .inr w ⇒ B.decS w.val
  nonemptyS := fun
    | .inl v ⇒ A.nonemptyS v
    | .inr w ⇒ B.nonemptyS w.val
  target := Sum.elim (fun m ⇒ Sum.inl (A.target m))
    (fun m ⇒ Sum.inr ⟨B.target m, B.target_notMem_inputs m⟩)
  parents := Sum.elim (fun m ⇒ (A.parents m).map Function.Embedding.inl)
    (fun m ⇒ (B.parents m).map c.trEmb)

-- 'composeNet' is reducible, so instance search sees through '(composeNet c).states v' to the
-- 'Sum.elim' and no longer matches 'FinBayesNet.fintypeS'; re-expose the instances by hand.
instance instFintypeComposeStates (v : (composeNet c).V) : Fintype ((composeNet c).states v) :=
  (composeNet c).fintypeS v

instance instDecidableEqComposeStates (v : (composeNet c).V) :
  DecidableEq ((composeNet c).states v) := (composeNet c).decS v

@[simp] theorem composeNet_target_inl (m : A.M) :
  (composeNet c).target (.inl m) = .inl (A.target m) := rfl

@[simp] theorem composeNet_target_inr (m : B.M) :
  (composeNet c).target (.inr m) = .inr ⟨B.target m, B.target_notMem_inputs m⟩ := rfl

@[simp] theorem composeNet_parents_inl (m : A.M) :
  (composeNet c).parents (.inl m) = (A.parents m).map Function.Embedding.inl := rfl

@[simp] theorem composeNet_parents_inr (m : B.M) :
  (composeNet c).parents (.inr m) = (B.parents m).map c.trEmb := rfl

variable (c) in
/-- Appending 'B's topological order (on its non-input variables) after 'A's. -/
def composeTopo : (composeNet c).TopoOrder where
  order := A.topo.order.map Sum.inl ++ (B.topo.order.filterMap privOf).map Sum.inr
  nodup := by
    refine List.Nodup.append (A.topo.nodup.map Sum.inl_injective)
      (List.Nodup.map Sum.inr_injective ?_) ?_
    • refine List.Nodup.filterMap ?_ B.topo.nodup
      intro a a' b hb hb'
      rw [Option.mem_def, privOf_eq_some] at hb hb'
      exact hb.trans hb'.symm
    • rintro x hx hx'
      obtain ⟨a, _, rfl⟩ := List.mem_map.1 hx
      obtain ⟨w, _, h⟩ := List.mem_map.1 hx'
      exact Sum.inr_ne_inl h
  complete := fun
    | .inl a ⇒ List.mem_append_left _ (List.mem_map_of_mem (A.topo.complete a))
    | .inr w ⇒ List.mem_append_right _
      (List.mem_map_of_mem (List.mem_filterMap.2 ⟨w.val, B.topo.complete w.val,
        privOf_eq_some.2 rfl⟩))
  parents_before := by
    refine List.pairwise_append.2 {?, ?, ?}
    • refine List.pairwise_map.2 (A.topo.parents_before.imp ?_)
      rintro a b hab (m | m) hm
      • rw [composeNet_target_inl, Sum.inl.injEq] at hm

```

```

    simp using hab m hm
  · exact absurd hm (by simp)
· have hB : ((B.topo.order.filterMap privOf).Pairwise
  fun w w' : Priv B ⇒ ∀ m, B.target m = w.val → w'.val ∉ B.parents m) := by
  refine List.Pairwise.filterMap privOf ?_ B.topo.parents_before
  intro a a' hR b hb b' hb'
  rw [privOf_eq_some] at hb hb'
  subst hb; subst hb'
  exact hR
refine List.pairwise_map.2 (hB.imp ?_)
· rintro w w' hww' (m | m) hm
  · exact absurd hm (by simp)
  · rw [composeNet_target_inr, Sum.inr.injEq] at hm
    rw [composeNet_parents_inr, inr_mem_map_trEmb]
    exact hww' m (congrArg Subtype.val hm)
· rintro x hx y hy (m | m) hm
  · obtain ⟨w, _, rfl⟩ := List.mem_map.1 hy
    simp
  · obtain ⟨a, _, rfl⟩ := List.mem_map.1 hx
    exact absurd hm (by simp)
no_self := fun
| .inl m ⇒ by simp using A.topo.no_self m
| .inr m ⇒ by
  rw [composeNet_target_inr, composeNet_parents_inr, inr_mem_map_trEmb]
  exact B.topo.no_self m

variable (c) in
/-- **The closure theorem, rule 1 ("at most one mechanism per variable").** The target map of
the composite is injective whenever those of 'A' and 'B' are. -/
theorem composeNet_target_injective : Function.Injective (composeNet c).target := by
  rintro (m | m) (m' | m') h
  · exact congrArg Sum.inl (A.target_inj (Sum.inl.inj h))
  · exact absurd h (by simp)
  · exact absurd h (by simp)
  · exact congrArg Sum.inr (B.target_inj (congrArg Subtype.val (Sum.inr.inj h)))

variable (c) in
/-- **The closure theorem, rule 1' (inputs are exogenous).** No composite mechanism targets a
variable of 'A.inputs'. -/
theorem compose_input_exogenous :
  ∀ v ∈ A.inputs.map Function.Embedding.inl, ∀ m, (composeNet c).target m ≠ v := by
  intro v hv m
  obtain ⟨a, ha, rfl⟩ := Finset.mem_map.1 hv
  cases m with
  | inl m ⇒ exact fun h ⇒ A.input_exogenous a ha m (Sum.inl.inj h)
  | inr m ⇒ exact fun h ⇒ by simp at h

variable (c) in
/-- **The closure theorem, rule 3 (every mechanism-free variable is an input).** A composite
variable with no generating mechanism comes from 'A' and is an input of 'A'; the variables
contributed by 'B' all keep their mechanism. -/
theorem compose_exogenous_input :
  ∀ v, (∀ m, (composeNet c).target m ≠ v) → v ∈ A.inputs.map Function.Embedding.inl := by
  rintro (a | w) h
  · exact Finset.mem_map_of_mem _
    (A.exogenous_input a fun m hm ⇒ h (.inl m) (congrArg Sum.inl hm))
  · exact absurd (B.exogenous_input w.val fun m hm ⇒
    h (.inr m) (congrArg Sum.inr (Subtype.ext hm))) w.property

variable (c) in
/-- **The open-network closure theorem (SPEC §13, revision note).** The pushout composite of two
valid open networks along an injective interface match is again a valid open network: its inputs
are 'A''s inputs, its outputs are 'A''s unglued outputs together with 'B''s outputs, and the four
typed-interface rules hold. -/
@[reducible] def compose : OpenFinBayesNet where
  toFinBayesNet := composeNet c

```

```

inputs := A.inputs.map Function.Embedding.inl
outputs := (A.outputs \ c.glued).map Function.Embedding.inl ∪ B.outputs.map c.trEmb
target_inj := composeNet_target_injective c
input_exogenous := compose_input_exogenous c
exogenous_input := compose_exogenous_input c
topo := composeTopo c

```

Semantics

```

variable (c) in
/-- The 'A'-part of a composite assignment. -/
def restrictA (x : (composeNet c).Assignment) : A.Assignment := fun a => x (Sum.inl a)

variable (c) in
/-- The 'B'-part of a composite assignment: glued variables are read off the 'A' side
through the interface equivalence. -/
def restrictB (x : (composeNet c).Assignment) : B.Assignment := fun v =>
  if h : v ∈ B.inputs then c.states_equiv ⟨v, h⟩ (x (Sum.inl (c.ι ⟨v, h⟩).val))
  else x (Sum.inr ⟨v, h⟩)

variable (c) in
/-- On a variable that is not an input of 'B', 'restrictB' just reads the 'Sum.inr' component. -/
theorem restrictB_of_notMem (x : (composeNet c).Assignment) {v : B.V} (h : v ∉ B.inputs) :
  restrictB c x v = x (Sum.inr ⟨v, h⟩) := dif_neg h

variable (c) in
/-- Two kernel families side by side on the composite. -/
def composeKernel {R : Type} (κA : A.Kernel R) (κB : B.Kernel R) : (composeNet c).Kernel R
| .inl m => fun x y => κA m (restrictA c x) y
| .inr m => fun x y => κB m (restrictB c x) y

section Semantics

variable {R : Type}

/-- **Proposition 2 for sequential composition.** The joint of the composite is the product of
the two joints, each read off the corresponding part of the composite assignment. -/
theorem joint_compose [CommMonoid R] (κA : A.Kernel R) (κB : B.Kernel R)
(x : (composeNet c).Assignment) :
  joint (composeKernel c κA κB) x = joint κA (restrictA c x) * joint κB (restrictB c x) := by
have h2 : ∀ m : B.M, composeKernel c κA κB (.inr m) x (x ((composeNet c).target (.inr m)))
  = κB m (restrictB c x) (restrictB c x (B.target m)) := fun m =>
  congrArg _ (restrictB_of_notMem c x (B.target_notMem_inputs m)).symm
unfold joint
rw [Fintype.prod_sum_type, Finset.prod_congr rfl fun m _ => h2 m]
rfl

theorem normalised_compose [AddCommMonoid R] [One R] (κA : A.Kernel R) (κB : B.Kernel R)
(hA : ∀ m, Normalised κA m) (hB : ∀ m, Normalised κB m) :
  ∀ m, Normalised (composeKernel c κA κB) m
| .inl m => fun x => hA m (restrictA c x)
| .inr m => fun x => hB m (restrictB c x)

theorem local_compose (κA : A.Kernel R) (κB : B.Kernel R)
(hA : ∀ m, Local κA m) (hB : ∀ m, Local κB m) :
  ∀ m, Local (composeKernel c κA κB) m
| .inl m => fun x x' hx => hA m (restrictA c x) (restrictA c x') fun p hp =>
  hx (Sum.inl p) (Finset.mem_map_of_mem _ hp)
| .inr m => fun x x' hx => hB m (restrictB c x) (restrictB c x') fun p hp => by
  have hxx := hx (c.tr p) (Finset.mem_map_of_mem c.trEmb hp)
  by_cases h : p ∈ B.inputs
  · rw [tr_of_mem h] at hxx
  simp only [restrictB, dif_pos h]
  exact congrArg _ hxx

```

```

    • rw [tr_of_notMem h] at hxx
    rw [restrictB_of_notMem c x h, restrictB_of_notMem c x' h]
    exact hxx

variable (c) in
/-- The composite variables contributed by 'A' (its own variables and the glued ones). -/
def varsA : Finset (composeNet c).V := Finset.univ.map Function.Embedding.inl

variable (c) in
/-- The composite variables contributed by 'B' alone: its non-input ("private") variables. -/
def varsB : Finset (composeNet c).V := Finset.univ.map Function.Embedding.inr

@[simp] theorem inl_mem_varsA (a : A.V) : (Sum.inl a : (composeNet c).V) ∈ varsA c :=
  Finset.mem_map_of_mem _ (Finset.mem_univ a)

@[simp] theorem inr_notMem_varsA (w : Priv B) : (Sum.inr w : (composeNet c).V) ∉ varsA c := by
  simp [varsA]

@[simp] theorem inr_mem_varsB (w : Priv B) : (Sum.inr w : (composeNet c).V) ∈ varsB c :=
  Finset.mem_map_of_mem _ (Finset.mem_univ w)

@[simp] theorem inl_notMem_varsB (a : A.V) : (Sum.inl a : (composeNet c).V) ∉ varsB c := by
  simp [varsB]

/-- The two halves are disjoint. -/
theorem disjoint_varsA_varsB : Disjoint (varsA c) (varsB c) := by
  rw [Finset.disjoint_left]
  rintro (a | w) hA hB
  • exact inl_notMem_varsB a hB
  • exact inr_notMem_varsA w hA

/-- The two halves of the composite's variables partition it. -/
theorem compl_varsA : (varsA c)c = varsB c := by
  ext v; cases v <|> simp [varsA, varsB]

/-- The composite mechanisms targeting the 'A' side are exactly 'A's. -/
theorem filter_target_mem_varsA :
  (Finset.univ.filter fun m => (composeNet c).target m ∈ varsA c)
  = Finset.univ.map Function.Embedding.inl := by
  ext m; cases m <|> simp [varsA]

/-- **Proposition 3, downstream half, for open networks.** Summing the joint of the composite
over all values of 'B's private variables leaves the joint of 'A' – the semantics of 'B ∘ A'
marginalised back onto 'A's variables is the semantics of 'A'. Only 'B's kernels have to be
normalised. -/
theorem marg_joint_compose [CommSemiring R] (κA : A.Kernel R) (κB : B.Kernel R)
  (hlocA : ∀ m, Local κA m) (hlocB : ∀ m, Local κB m) (hnormB : ∀ m, Normalised κB m)
  (x₀ : (composeNet c).Assignment) :
  marg (varsB c) (joint (composeKernel c κA κB)) x₀ = joint κA (restrictA c x₀) := by
  have h := marg_joint_downstream (bn := composeNet c) (composeKernel c κA κB)
  (composeNet_target_injective c) (composeTopo c) (local_compose κA κB hlocA hlocB) (varsA c)
  (by
    rintro (m | m) hm
    • exact absurd (inl_mem_varsA (A.target m)) hm
    • exact fun x => hnormB m (restrictB c x))
  (by
    rintro (m | m) hm p hp
    • rw [composeNet_parents_inl, Finset.mem_map] at hp
    obtain ⟨a, _, rfl⟩ := hp
    exact inl_mem_varsA a
    • exact absurd hm (by simp [varsA]))
  (by
    rintro (a | w) hv
    • exact absurd (inl_mem_varsA a) hv
    • obtain ⟨m, hm⟩ := B.exists_mechanism w.property
    exact ⟨.inr m, congrArg Sum.inr (Subtype.ext hm)⟩)

```

```

    x₀
  rw [compl_varsA] at h
  rw [h, filter_target_mem_varsA, Finset.prod_map]
  rfl

/-- Proposition 3 (sequential compositionality, SPEC §13.2, §61). For any set 'S' of composite variables coming from 'A' and any disjoint set 'T' coming from 'B's private part, integrating 'S ∪ T' out of the joint of the composite is the same as integrating 'T' out of 'B's joint first, multiplying by 'A's joint, and then integrating 'S' out. Taking 'S' to be 'A's hidden variables together with the glued interface and 'T' to be 'B's hidden variables, this reads '[B ∘ A] = [B] ∘ [A]': the composite semantics is the sum over the interface of the product of the two open semantics. -/
theorem marg_joint_compose_split [CommSemiring R] (κA : A.Kernel R) (κB : B.Kernel R)
  {S T : Finset (composeNet c).V} (hS : S ⊆ varsA c) (hT : T ⊆ varsB c)
  (x : (composeNet c).Assignment) :
  marg (S ∪ T) (joint (composeKernel c κA κB)) x
  = marg S (fun y ⇒ joint κA (restrictA c y))
    * marg T (fun z ⇒ joint κB (restrictB c z)) y x := by
have hjc : joint (composeKernel c κA κB)
  = fun z ⇒ joint κA (restrictA c z) * joint κB (restrictB c z) :=
  funext fun z ⇒ joint_compose κA κB z
rw [marg_union_disjoint ((disjoint_varsA_varsB (c := c)).mono hS hT), hjc]
unfold marg
refine Finset.sum_congr rfl fun y _ => ?_
refine marg_mul_left fun z hz => ?_
have : restrictA c z = restrictA c y := by
  funext a
  exact mem_fibre.1 hz (Sum.inl a) fun hmem => inl_notMem_varsB a (hT hmem)
rw [this]

variable (c) in
/-- Composing a *closed* network 'A' (no inputs) with an open 'B' gives a closed network: every composite variable still has exactly one mechanism. -/
theorem closed_compose (h : A.inputs = ∅) : (composeNet c).Closed := by
  refine ⟨composeNet_target_injective c, ?_⟩
  rintro (a | w)
  • obtain ⟨m, hm⟩ := A.exists_mechanism (show a ∉ A.inputs by simp [h])
    exact ⟨.inl m, congrArg Sum.inl hm⟩
  • obtain ⟨m, hm⟩ := B.exists_mechanism w.property
    exact ⟨.inr m, congrArg Sum.inr (Subtype.ext hm)⟩

/-- Proposition 1a for a composite with a closed left factor: '[B ∘ A]' is a probability distribution on the composite variables. -/
theorem sum_joint_compose_eq_one [CommSemiring R] (h : A.inputs = ∅) (κA : A.Kernel R)
  (κB : B.Kernel R) (hlocA : ∀ m, Local κA m) (hlocB : ∀ m, Local κB m)
  (hnormA : ∀ m, Normalised κA m) (hnormB : ∀ m, Normalised κB m) :
  ∑ x, joint (composeKernel c κA κB) x = 1 :=
  sum_joint_eq_one _ (closed_compose c h) (composeTopo c) (local_compose κA κB hlocA hlocB)
  (normalised_compose κA κB hnormA hnormB)

end Semantics

end Composable

end OpenFinBayesNet

end BayesianNetworksProofs

```

BayesianNetworksProofs.Finite.OpenSemantics

```
import BayesianNetworksProofs.Finite.Open
```

Proposition 3 in each component's own open semantics (SPEC §13.2, §61).

Finite marginal sums transfer across the two pushout legs. The proof uses one-variable updates and finite Fubini, so state-space equivalences on the glued interface need not be identities. The resulting composition theorem permits a partial interface match (glue), not only a total match (compose), and needs neither kernel locality nor normalisation: the equality is an algebraic identity of finite sums and products.

osem_compose_passthrough is the formula for arbitrary interfaces. A glued variable that stays on the composite's interface – an input of A, or passed by B straight to its outputs – keeps its visible value and is never summed; only the innerGlued variables are. It needs no hypotheses at all. With disjoint input and output sets on both networks every glued variable is inner (innerGlued_eq_glued), which gives osem_compose_glue.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.OpenFinBayesNet.Composable

open FinBayesNet

variable {A B : OpenFinBayesNet} (c : Composable A B) {R : Type} [CommSemiring R]

theorem restrictA_update (x : (composeNet c).Assignment) (v : A.V) (z : A.states v) :
  restrictA c (Function.update x (.inl v) z) =
    Function.update (restrictA c x) v z := by
  funext w
  by_cases h : w = v
  · subst w
    simp [restrictA]
  · simp [restrictA, h]

theorem restrictB_update (x : (composeNet c).Assignment) (v : B.V)
  (hv : v ∉ B.inputs) (z : B.states v) :
  restrictB c (Function.update x (.inr ⟨v, hv⟩) z) =
    Function.update (restrictB c x) v z := by
  funext w
  by_cases h : w = v
  · subst w
    simp [restrictB, hv]
  · by_cases hw : w ∈ B.inputs
    · simp [restrictB, hw, Function.update_of_ne h]
    · have hsub : (⟨w, hw⟩ : Priv B) ≠ ⟨v, hv⟩ := fun he ⇒ h (congrArg Subtype.val he)
      simp [restrictB, hw, Function.update_of_ne h, hsub]

/-- Transfer a finite marginal across the left pushout leg. -/
theorem marg_restrictA (S : Finset A.V) (F : A.Assignment → R)
  (x : (composeNet c).Assignment) :
  marg (S.map Function.Embedding.inl) (fun y ⇒ F (restrictA c y)) x =
    marg S F (restrictA c x) := by
  induction S using Finset.induction_on generalizing x with
  | empty ⇒
    simp only [Finset.map_empty]
    rw [marg_empty (bn := composeNet c), marg_empty]
  | @insert v S hv ih ⇒
    rw [Finset.map_insert, marg_insert (by simpa using hv), marg_insert hv]
    exact Finset.sum_congr rfl fun z _ => by
      rw [ih]
      simp only [Function.Embedding.inl_apply]
      rw [restrictA_update]

/-- Transfer a finite marginal across the right leg when no summed variable is glued. -/
theorem marg_restrictB (S : Finset B.V) (hS : Disjoint S B.inputs)
  (F : B.Assignment → R) (x : (composeNet c).Assignment) :
  marg (S.map c.trEmb) (fun y ⇒ F (restrictB c y)) x =
    marg S F (restrictB c x) := by
  induction S using Finset.induction_on generalizing x with
  | empty ⇒
    simp only [Finset.map_empty]
    rw [marg_empty (bn := composeNet c), marg_empty]
  | @insert v S hv ih ⇒

```

```

have hvi : v ∉ B.inputs :=
  fun h ⇒ Finset.disjoint_left.1 hS (Finset.mem_insert_self _ _) h
have hSi : Disjoint S B.inputs := hS.mono_left (Finset.subset_insert _ _)
have hnot : c.trEmb v ∉ S.map c.trEmb := by
  intro hm
  obtain ⟨w, hw, he⟩ := Finset.mem_map.1 hm
  exact hv (c.trEmb.injective he ▸ hw)
rw [trEmb_apply, tr_of_notMem hvi] at hnot
rw [Finset.map_insert, trEmb_apply, tr_of_notMem hvi,
  marg_insert hnot, marg_insert hv]
exact Finset.sum_congr rfl fun z _ ⇒ by rw [ih hSi, restrictB_update]

theorem glued_subset_outputs : c.glued ⊆ A.outputs := by
  intro v hv
  obtain ⟨w, _, rfl⟩ := Finset.mem_image.1 hv
  exact (c.1 w).property

theorem hidden_disjoint_inputs (O : OpenFinBayesNet) : Disjoint O.hidden O.inputs := by
  rw [Finset.disjoint_left]
  intro v hv hi
  exact (Finset.mem_compl.1 hv) (Finset.mem_union_left _ hi)

/-- An 'A'-hidden variable cannot affect the assignment seen by 'B', even at a glued input. -/
theorem restrictB_eq_of_mem_fibre_hidden (x y : (composeNet c).Assignment)
  (hy : y ∈ fibre (A.hidden.map Function.Embedding.inl) x) :
  restrictB c y = restrictB c x := by
  funext v
  by_cases hv : v ∈ B.inputs
  · simp only [restrictB, dif_pos hv]
    congr 1
    apply mem_fibre.1 hy
    simp [hidden, (c.1 ⟨v, hv⟩).property]
  · rw [restrictB_of_notMem c y hv, restrictB_of_notMem c x hv]
    exact mem_fibre.1 hy _ (by simp)

/-- The glued variables that the composite hides. A glued variable stays on the composite's
interface, and must not be summed, when it is an input of 'A' (a pass-through on 'A's side,
which remains a composite input) or when 'B' passes it straight to its outputs (a
pass-through on 'B's side, which becomes a composite output). Only the rest is internal. -/
def innerGlued : Finset A.V :=
  c.glued.filter fun a ⇒ a ∉ A.inputs ∧ Sum.inl a ∉ B.outputs.map c.trEmb

theorem innerGlued_subset_glued : c.innerGlued ⊆ c.glued := Finset.filter_subset _ _

/-- An 'A' variable that 'B's outputs reach through the right leg is glued. -/
theorem mem_glued_of_inl_mem_map (a : A.V) (h : Sum.inl a ∈ B.outputs.map c.trEmb) :
  a ∈ c.glued := by
  obtain ⟨w, _, he⟩ := Finset.mem_map.1 h
  by_cases hw : w ∈ B.inputs
  · rw [trEmb_apply, tr_of_mem hw, Sum.inl.injEq] at he
    exact Finset.mem_image.2 ⟨⟨w, hw⟩, Finset.mem_univ _, he⟩
  · rw [trEmb_apply, tr_of_notMem hw] at he
    exact absurd he (by simp)

theorem inl_notMem_map_of_disjoint (a : A.V) (S : Finset B.V) (hs : Disjoint S B.inputs) :
  Sum.inl a ∉ S.map c.trEmb := by
  intro hm
  obtain ⟨v, hv, he⟩ := Finset.mem_map.1 hm
  have hvi : v ∉ B.inputs := fun hi ⇒ Finset.disjoint_left.1 hs hv hi
  rw [trEmb_apply, tr_of_notMem hvi] at he
  exact Sum.inr_ne_inl he

/-- The hidden variables of a composite, for arbitrary interfaces: pass-through variables
on either side are allowed, and exactly the inner glued variables join the two hidden sets. -/
theorem hidden_compose_general :
  (compose c).hidden =

```

```

(c.innerGlued.map Function.Embedding.inl U A.hidden.map Function.Embedding.inl) U
  B.hidden.map c.trEmb := by
ext v
cases v with
| inl a =>
  have hbh : Sum.inl a ∉ B.hidden.map c.trEmb :=
    inl_notMem_map_of_disjoint c a B.hidden (hidden_disjoint_inputs B)
  have hin : (Sum.inl a : A.V ⊕ Priv B) ∈ A.inputs.map Function.Embedding.inl ↔ a ∈ A.inputs :=
    Finset.mem_map' _
  have hout : (Sum.inl a : A.V ⊕ Priv B) ∈ (A.outputs \ c.glued).map Function.Embedding.inl ↔
    a ∈ A.outputs ∧ a ∉ c.glued :=
    (Finset.mem_map' _).trans Finset.mem_sdiff
  have hinner : (Sum.inl a : A.V ⊕ Priv B) ∈ c.innerGlued.map Function.Embedding.inl ↔
    a ∈ c.glued ∧ a ∉ A.inputs ∧ Sum.inl a ∉ B.outputs.map c.trEmb :=
    (Finset.mem_map' _).trans Finset.mem_filter
  have hhid : (Sum.inl a : A.V ⊕ Priv B) ∈ A.hidden.map Function.Embedding.inl ↔
    ¬(a ∈ A.inputs ∨ a ∈ A.outputs) :=
    (Finset.mem_map' _).trans (by rw [hidden, Finset.mem_compl, Finset.mem_union])
  rw [hidden, Finset.mem_compl]
  change (Sum.inl a ∉ A.inputs.map Function.Embedding.inl U
    ((A.outputs \ c.glued).map Function.Embedding.inl U B.outputs.map c.trEmb)) ↔
    Sum.inl a ∈ (c.innerGlued.map Function.Embedding.inl U
      A.hidden.map Function.Embedding.inl) U B.hidden.map c.trEmb
  rw [Finset.mem_union, Finset.mem_union, Finset.mem_union, Finset.mem_union, hin, hout,
    hinner, hhid]
  simp only [hbh, or_false]
  -- Propositional reasoning, spelled out: 'tauto' would unfold the memberships of the
  -- composite, whose definition is large.
  by_cases hg : a ∈ c.glued
  · have ho := glued_subset_outputs c hg
    constructor
    · intro h
      exact Or.inl ⟨hg, fun hi => h (Or.inl hi), fun hbo => h (Or.inr (Or.inr hbo))⟩
    · rintro (⟨-, hi, hbo⟩ | hn) (h | ⟨-, hng⟩ | h)
      · exact hi h
      · exact hng hg
      · exact hbo h
      · exact hn (Or.inl h)
      · exact hng hg
      · exact hn (Or.inr ho)
  · have hb : Sum.inl a ∉ B.outputs.map c.trEmb :=
      fun h => hg (mem_glued_of_inl_mem_map c a h)
    constructor
    · intro h
      exact Or.inr fun h' => h'.elim (fun hi => h (Or.inl hi))
        (fun ho => h (Or.inr (Or.inl ⟨ho, hg⟩)))
    · rintro (⟨hg', -, -⟩ | hn) (h | ⟨ho, -⟩ | h)
      · exact hg hg'
      · exact hg hg'
      · exact hg hg'
      · exact hn (Or.inl h)
      · exact hn (Or.inr ho)
      · exact hb h
| inr b =>
  rw [hidden, Finset.mem_compl]
  change (Sum.inr b ∉ A.inputs.map Function.Embedding.inl U
    ((A.outputs \ c.glued).map Function.Embedding.inl U B.outputs.map c.trEmb)) ↔
    Sum.inr b ∈ (c.innerGlued.map Function.Embedding.inl U
      A.hidden.map Function.Embedding.inl) U B.hidden.map c.trEmb
  simp only [Finset.mem_union, inr_mem_map_trEmb (c := c)]
  simp [hidden, b.property]

/-- The inner glued variables are exactly the glued variables that the composite hides. -/
theorem innerGlued_eq_filter_hidden :
  c.innerGlued =
    c.glued.filter fun a => (Sum.inl a : A.V ⊕ Priv B) ∈ (compose c).hidden := by

```

```

ext a
rw [Finset.mem_filter, hidden_compose_general c]
constructor
• intro h
  exact ⟨innerGlued_subset_glued c h,
    Finset.mem_union_left _ (Finset.mem_union_left _ (Finset.mem_map_of_mem _ h))⟩
• rintro ⟨hg, h⟩
  rcases Finset.mem_union.1 h with h | h
  • rcases Finset.mem_union.1 h with h | h
    • exact (Finset.mem_map' _).1 h
    • exfalso
      have hah : a ∈ A.hidden := (Finset.mem_map' _).1 h
      exact Finset.mem_compl.1 hah (Finset.mem_union_right _ (glued_subset_outputs c hg))
  • exact absurd h (inl_notMem_map_of_disjoint c a B.hidden (hidden_disjoint_inputs B))

/-- With disjoint input and output sets on both networks there is no pass-through, and every
glued variable is inner. -/
theorem innerGlued_eq_glued (hA : A.inputs ∩ A.outputs = ∅)
  (hB : B.inputs ∩ B.outputs = ∅) : c.innerGlued = c.glued := by
  refine Finset.filter_true_of_mem fun a ha ⇒ ⟨fun hi ⇒ ?_, fun hm ⇒ ?_⟩
  • have := Finset.mem_inter.2 ⟨hi, glued_subset_outputs c ha⟩
    rw [hA] at this
    exact Finset.notMem_empty _ this
  • obtain ⟨w, hwo, he⟩ := Finset.mem_map.1 hm
    by_cases hw : w ∈ B.inputs
    • have := Finset.mem_inter.2 ⟨hw, hwo⟩
      rw [hB] at this
      exact Finset.notMem_empty _ this
    • rw [trEmb_apply, tr_of_notMem hw] at he
      exact absurd he (by simp)

/-- The hidden variables of a composite, with unglued outputs retained. Total matching is
not required. -/
theorem hidden_compose (hA : A.inputs ∩ A.outputs = ∅)
  (hB : B.inputs ∩ B.outputs = ∅) :
  (compose c).hidden =
    (c.glued.map Function.Embedding.inl ∪ A.hidden.map Function.Embedding.inl) ∪
    B.hidden.map c.trEmb := by
  rw [hidden_compose_general, innerGlued_eq_glued c hA hB]

/-- **Proposition 3 with pass-through.** For arbitrary interfaces -- copied or passed-through
variables on either side, partial gluing, no locality, normalisation or matching assumptions --
the composite's open semantics is the product of the two own-variable open semantics summed
over the inner glued variables only. A glued variable that stays on the composite's interface
(an input of 'A', or passed by 'B' to its outputs) keeps its visible value: it is never summed. -/
theorem osem_compose_passthrough (κA : A.Kernel R) (κB : B.Kernel R)
  (x : (compose c).Assignment) :
  osem (compose c) (composeKernel c κA κB) x =
    marg (c.innerGlued.map Function.Embedding.inl)
    (fun y ⇒ osem A κA (restrictA c y) * osem B κB (restrictB c y)) x := by
  have hg : Disjoint (c.innerGlued.map Function.Embedding.inl : Finset (composeNet c).V)
    (A.hidden.map Function.Embedding.inl) := by
    rw [Finset.disjoint_left]
  intro v hv hh
  obtain ⟨a, ha, rfl⟩ := Finset.mem_map.1 hv
  have hah : a ∈ A.hidden := by simp using hh
  exact Finset.mem_compl.1 hah (Finset.mem_union_right _
    (glued_subset_outputs c (innerGlued_subset_glued c ha)))
  have hS : c.innerGlued.map Function.Embedding.inl ∪ A.hidden.map Function.Embedding.inl
    ⊆ varsA c := by
    intro v hv
    rcases Finset.mem_union.1 hv with hv | hv <|>
    obtain ⟨a, _, rfl⟩ := Finset.mem_map.1 hv <|> exact inl_mem_varsA a
  have hT : B.hidden.map c.trEmb ⊆ varsB c := by
    intro v hv
    obtain ⟨b, hb, rfl⟩ := Finset.mem_map.1 hv

```

```

have hbi : b ∉ B.inputs := fun hi =>
  Finset.disjoint_left.1 (hidden_disjoint_inputs B) hb hi
rw [trEmb_apply, tr_of_notMem hbi]
exact inr_mem_varsB _
unfold osem
rw [hidden_compose_general c, marg_joint_compose_split κA κB hS hT,
  marg_union_disjoint hg]
have heq : ∀ z, marg (B.hidden.map c.trEmb)
  (fun w => joint κB (restrictB c w)) z = marg B.hidden (joint κB) (restrictB c z) :=
  marg_restrictB c B.hidden (hidden_disjoint_inputs B) (joint κB)
simp_rw [heq]
apply congrArg (fun F => marg (c.innerGlued.map Function.Embedding.inl) F x)
funext y
calc
- = marg (A.hidden.map Function.Embedding.inl)
  (fun z => joint κA (restrictA c z)) y *
  marg B.hidden (joint κB) (restrictB c y) := by
  unfold marg
  rw [Finset.sum_mul]
  refine Finset.sum_congr rfl fun z hz => ?_
  dsimp only
  rw [restrictB_eq_of_mem_fibre_hidden c y z hz]
- = _ := by rw [marg_restrictA]

/-- Proposition 3, including partial gluing. Sum the product of the two own-variable
open semantics over the glued interface. No stochastic or locality assumptions are needed.
Without pass-through every glued variable is inner, so this is 'osem_compose_passthrough'. -/
theorem osem_compose_glue (κA : A.Kernel R) (κB : B.Kernel R)
  (hA : A.inputs ∩ A.outputs = ∅) (hB : B.inputs ∩ B.outputs = ∅)
  (x : (compose c).Assignment) :
  osem (compose c) (composeKernel c κA κB) x =
    marg (c.glued.map Function.Embedding.inl)
      (fun y => osem A κA (restrictA c y) * osem B κB (restrictB c y)) x := by
  rw [← innerGlued_eq_glued c hA hB]
  exact osem_compose_passthrough c κA κB x

/-- The original Roadmap statement, with its original (now known to be stronger than
necessary) premises retained. 'osem_compose_glue' proves it without locality, normalisation,
or surjectivity of the interface map. -/
theorem osem_compose (κA : A.Kernel R) (κB : B.Kernel R)
  (_hlocA : ∀ m, Local κA m) (_hlocB : ∀ m, Local κB m) (_hnormB : ∀ m, Normalised κB m)
  (hA : A.inputs ∩ A.outputs = ∅) (hB : B.inputs ∩ B.outputs = ∅)
  (_hι : Function.Surjective c.ι) (x : (compose c).Assignment) :
  osem (compose c) (composeKernel c κA κB) x =
    marg (c.glued.map Function.Embedding.inl)
      (fun y => osem A κA (restrictA c y) * osem B κB (restrictB c y)) x := by
  exact osem_compose_glue c κA κB hA hB x

end BayesianNetworksProofs.OpenFinBayesNet.Composable

```

BayesianNetworksProofs.Finite.BoundaryCases

```

import BayesianNetworksProofs.Finite.OpenSemantics
import Mathlib.Data.Finset.Card

```

Boundary counterexamples and representation limits (SPEC §13, §61 and §62.1).

The own-semantics formula `osem_compose_glue` sums over every glued variable, so it needs disjoint inputs/outputs on both networks. On the binary identity wire, the open semantics is one, while blindly summing its visible glued state gives two: `passthrough_formula_fails` refutes that formula with its side conditions removed. `osem_compose_passthrough` is the formula for arbitrary interfaces: it sums only the inner glued variables, and on the same wire it gives one (`passthrough_formula_holds_on_wire`).

There is a separate structural limitation: with no mechanisms, the typed-interface rule forces every variable to be an

input. Since outputs are a subset of the apex, their number cannot exceed the input count. Thus this representation cannot express a mechanism-free one-input/two-output copying *foot* by listing an apex variable twice. General boundary maps, including noninjective output legs, are not modelled. This is not a proof that no possible category or semantic copy construction exists.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.OpenFinBayesNet

open FinBayesNet

/-- In a mechanism-free network every apex variable is necessarily an input. -/
theorem inputs_eq_univ_of_no_mechanisms (O : OpenFinBayesNet) [IsEmpty O.M] :
  O.inputs = Finset.univ := by
  apply Finset.eq_univ_of_forall
  intro v
  exact O.exogenous_input v fun m => isEmptyElim m

/-- Subset feet cannot represent mechanism-free copying by duplicated output ports. -/
theorem outputs_card_le_inputs_card_of_no_mechanisms (O : OpenFinBayesNet) [IsEmpty O.M] :
  O.outputs.card ≤ O.inputs.card := by
  rw [inputs_eq_univ_of_no_mechanisms]
  exact Finset.card_le_card (Finset.subset_univ _)

/-- The binary pass-through wire, with no mechanism. -/
def binaryWire : OpenFinBayesNet where
  V := Unit
  M := Empty
  states := fun _ => Bool
  target := Empty.elim
  parents := Empty.elim
  inputs := Finset.univ
  outputs := Finset.univ
  target_inj := by intro m; exact isEmptyElim m
  input_exogenous := by intro _ _ m; exact isEmptyElim m
  exogenous_input := by intro v _; exact Finset.mem_univ v
  topo := {
    order := [()]
    nodup := by simp
    complete := by intro v; cases v; simp
    parents_before := by simp
    no_self := by intro m; exact isEmptyElim m
  }

/-- Total matching of the binary wire with itself. -/
def binaryWireMatch : Composable binaryWire binaryWire where
  ι := id
  ι_inj := Function.injective_id
  states_equiv := fun _ => Equiv.refl _

/-- The unique empty mechanism family. -/
def binaryWireKernel : binaryWire.Kernel ℕ := fun m => nomatch m

/-- The retained external input of the composite is fixed to 'false'. -/
def binaryWireAssignment : (Composable.compose binaryWireMatch).Assignment
| .inl _ => false
| .inr w => False.elim (w.property (Finset.mem_univ w.val))

/-- A checked counterexample to silently removing the no-pass-through side conditions. -/
theorem passthrough_formula_fails :
  osem (Composable.compose binaryWireMatch)
    (Composable.composeKernel binaryWireMatch binaryWireKernel binaryWireKernel)
    binaryWireAssignment = 1 ∧
  marg (binaryWireMatch.glued.map Function.Embedding.inl)
    (fun y => osem binaryWire binaryWireKernel (Composable.restrictA binaryWireMatch y) *
      osem binaryWire binaryWireKernel (Composable.restrictB binaryWireMatch y))

```

```

        binaryWireAssignment = 2 := by
    decide

/-- The pass-through formula 'osem_compose_passthrough' on the same wire. The glued variable is
an input of 'A' and passed straight to 'B''s outputs, so it stays on the composite's interface:
it is not inner, nothing is summed, and the formula gives the true value one. -/
theorem passthrough_formula_holds_on_wire :
  binaryWireMatch.innerGlued = 0 ∧
  marg (binaryWireMatch.innerGlued.map Function.Embedding.inl)
    (fun y => osem binaryWire binaryWireKernel (Composable.restrictA binaryWireMatch y) *
      osem binaryWire binaryWireKernel (Composable.restrictB binaryWireMatch y))
  binaryWireAssignment = 1 := by
    decide

end BayesianNetworksProofs.OpenFinBayesNet

```

BayesianNetworksProofs.Markov.Basic

```

import Mathlib.CategoryTheory.MarkovCategory.Basic
import Mathlib.CategoryTheory.CopyDiscardCategory.Deterministic

```

Generic lemmas about Mathlib's abstract CopyDiscardCategory / MarkovCategory that the MarkovCategories.jl / BayesianNetworks.jl tests mirror for the concrete finite-stochastic instance (@instance ThMarkovCategory{FiniteSpace,FiniteKernel}):

- `discard_natural` — $f \gg \varepsilon[Y] = \varepsilon[X]$: composing a kernel with delete is delete. In Julia this is the test “discard naturality holds iff the kernel is normalised”; it is the axiom that ThMarkovCategory adds on top of ThMonoidalCategoryWithDiagonals.
- `deterministic_comp` — the composite of two deterministic (copy-preserving) kernels is deterministic; in Julia, point-mass kernels compose to point-mass kernels.
- `state_discard` — a state $p : I \rightarrow X$ composed with discard is the identity of the unit (a normalised distribution has total mass one).

The concrete finite-stochastic instance is in the sibling FiniteKernels.jl proof project, module Theory/FinStoch.lean; this module only uses Mathlib's abstract classes. Mathlib's Deterministic is IsComonHom, whose composition instance already exists; the theorem below simply names it.

```

namespace BayesianNetworksProofs.Markov

open CategoryTheory MonoidalCategory CopyDiscardCategory ComonObj

universe v u

variable {C : Type u} [Category.{v} C] [MonoidalCategory.{v} C]

section Markov

variable [MarkovCategory C]

/-- Discard is natural: every morphism of a Markov category is "normalised". -/
theorem discard_natural {X Y : C} (f : X → Y) : f >> ε[Y] = ε[X] :=
  MarkovCategory.discard_natural f

/-- A state has total mass one: composing 'p : I → X' with discard is the identity on 'I'. -/
theorem state_discard {X : C} (p : 1_C → X) : p >> ε[X] = 1 (1_C) := by
  rw [MarkovCategory.discard_natural, discard_unit]

end Markov

section CopyDiscard

variable [CopyDiscardCategory C]

```

```

/-- The composite of two deterministic morphisms is deterministic (Mathlib's 'IsComonHom'
composition instance, named here for the Julia test 'compose(δ_a, δ_b)' is a point mass). -/
theorem deterministic_comp {X Y Z : C} (f : X → Y) (g : Y → Z) [Deterministic f]
  [Deterministic g] : Deterministic (f > g) :=
  inferInstance

/-- Identities are deterministic. -/
theorem deterministic_id (X : C) : Deterministic (1 X) :=
  inferInstance

/-- Deterministic morphisms commute with copy: 'f > Δ = Δ > (f ⊗ f)'. -/
theorem deterministic_copy {X Y : C} (f : X → Y) [Deterministic f] :
  f > Δ[Y] = Δ[X] > (f ⊗m f) :=
  Deterministic.copy_natural f

/-- The comonoid laws every object satisfies (the Julia comonoid-law tests). -/
example (X : C) : Δ[X] > (ε[X] ▷ X) = (λ_ X).inv := counit_comul X
example (X : C) : Δ[X] > (X < ε[X]) = (ρ_ X).inv := comul_counit X
example (X : C) : Δ[X] > (X < Δ[X]) = Δ[X] > (Δ[X] ▷ X) > (α_ X X X).hom := comul_assoc X

end CopyDiscard

end BayesianNetworksProofs.Markov

```

BayesianNetworksProofs.Finite.Probability

```

import BayesianNetworksProofs.Finite.Intervention
import BayesianNetworksProofs.Finite.Tensor
import Mathlib.Data.NNReal.Defs

```

The finite model instantiated at $\mathbb{R}_{\geq 0}$, the value semiring of `FiniteKernel{Float64}` tables in `MarkovCategories.jl` (non-negative weights). Everything in `Finite/` is proved for an arbitrary commutative semiring; these corollaries just fix $R := \mathbb{R}_{\geq 0}$ so that the statements read as statements about probability distributions.

```

namespace BayesianNetworksProofs

namespace FinBayesNet

open scoped NNReal

/-- A family of non-negative real conditional kernels. -/
abbrev ProbKernel (bn : FinBayesNet) := bn.Kernel ℝ≥0

variable {bn : FinBayesNet}

/-- Proposition 1a over 'ℝ≥0': the joint of a closed, acyclic network with normalised local
kernels is a probability distribution on assignments. -/
theorem sum_joint_eq_one_nnreal (κ : bn.ProbKernel) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) : ∑ x, joint κ x = 1 :=
  sum_joint_eq_one κ hclosed ord hloc hnorm

/-- Proposition 4 over 'ℝ≥0': 'do(target m₀ = a)' truncates the factorisation. -/
theorem joint_intervene_nnreal (κ : bn.ProbKernel) (m₀ : bn.M) (a : bn.states (bn.target m₀))
  (x : bn.Assignment) :
  joint (intervene κ m₀ a) x =
    (if x (bn.target m₀) = a then 1 else 0) *
    ∏ m ∈ Finset.univ.erase m₀, κ m x (x (bn.target m)) :=
  joint_intervene κ m₀ a x

end FinBayesNet

end BayesianNetworksProofs

```

Partial assignments and explicit conditioning

```
import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Real.Basic
```

Restriction and patching connect concrete tuples of retained/free variables with the existing full-assignment fibre semantics. Sums over `PartialAssignment S` enumerate only the coordinates in `S`. Explicitly fixing evidence and multiplying by its indicator give the same finite sum.

```
set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet

variable {bn : FinBayesNet}

abbrev PartialAssignment (bn : FinBayesNet) (S : Finset bn.V) :=
  (v : {v : bn.V // v ∈ S}) → bn.states v.val

def restrictTo (S : Finset bn.V) (x : bn.Assignment) : PartialAssignment bn S :=
  fun v => x v.val

def patch (S : Finset bn.V) (base : bn.Assignment) (x : PartialAssignment bn S) : bn.Assignment :=
  fun v => if hv : v ∈ S then x ⟨v, hv⟩ else base v

theorem restrict_patch (S : Finset bn.V) (base : bn.Assignment) (x : PartialAssignment bn S) :
  restrictTo S (patch S base x) = x := by
  funext v
  simp [restrictTo, patch, v.property]

theorem patch_mem_fibre (S : Finset bn.V) (base : bn.Assignment) (x : PartialAssignment bn S) :
  patch S base x ∈ fibre S base := by
  apply mem_fibre.2
  intro v hv
  simp [patch, hv]

theorem patch_restrict_of_mem {S : Finset bn.V} {base x : bn.Assignment}
  (h : x ∈ fibre S base) : patch S base (restrictTo S x) = x := by
  funext v
  by_cases hv : v ∈ S
  · simp [patch, restrictTo, hv]
  · simp [patch, hv, mem_fibre.1 h v hv]

/-- A finite sum over only the free coordinates equals the existing full-assignment marginal. -/
theorem sum_partial_eq_marg {R : Type} [CommSemiring R] (S : Finset bn.V)
  (F : bn.Assignment → R) (base : bn.Assignment) :
  (∑ x : PartialAssignment bn S, F (patch S base x)) = marg S F base := by
  unfold marg
  refine Finset.sum_bij (fun x _ => patch S base x) ?_ ?_ ?_ ?_
  · intro x _
    exact patch_mem_fibre S base x
  · intro x _ y _ h
    have he := congrArg (restrictTo S) h
    simpa only [restrict_patch] using he
  · intro x hx
    exact ⟨restrictTo S x, Finset.mem_univ _, patch_restrict_of_mem hx⟩
  · intro x _
    rfl

def clamp (E : Finset bn.V) (observed x : bn.Assignment) : bn.Assignment :=
  fun v => if v ∈ E then observed v else x v

abbrev agrees (E : Finset bn.V) (observed x : bn.Assignment) : Prop :=
  ∀ v ∈ E, x v = observed v

def indicator (E : Finset bn.V) (observed x : bn.Assignment) : ℝ :=
  if agrees E observed x then 1 else 0
```

```

theorem clamp_agrees (E : Finset bn.V) (observed x : bn.Assignment) :
  agrees E observed (clamp E observed x) := by
  intro v hv
  simp [clamp, hv]

theorem clamp_patch {E S : Finset bn.V} (h : Disjoint E S)
  (observed base : bn.Assignment) (x : PartialAssignment bn S) :
  clamp E observed (patch S base x) = patch S (clamp E observed base) x := by
  funext v
  by_cases hv : v ∈ S
  · have he : v ∉ E := fun he => Finset.disjoint_left.1 h he hv
    simp [clamp, patch, hv, he]
  · simp [clamp, patch, hv]

theorem marg_clamp {E S : Finset bn.V} (h : Disjoint E S)
  (F : bn.Assignment → ℝ) (observed base : bn.Assignment) :
  marg S (fun x => F (clamp E observed x)) base = marg S F (clamp E observed base) := by
  rw [← sum_partial_eq_marg, ← sum_partial_eq_marg]
  exact Finset.sum_congr rfl fun x _ => congrArg F (clamp_patch h observed base x)

theorem fibre_condition {E S : Finset bn.V} (h : Disjoint E S)
  (observed base : bn.Assignment) :
  (fibre (S ∪ E) base).filter (agrees E observed) = fibre S (clamp E observed base) := by
  ext x
  simp only [Finset.mem_filter, mem_fibre]
  constructor
  · rintro ⟨hf, he⟩ v hv
    by_cases hvE : v ∈ E
    · simpa [clamp, hvE] using he v hvE
    · simpa [clamp, hvE] using hf v (by simp [hv, hvE])
  · intro hf
    constructor
    · intro v hv
      have hs : v ∉ S := fun hs => hv (Finset.mem_union_left _ hs)
      have he : v ∉ E := fun he => hv (Finset.mem_union_right _ he)
      simpa [clamp, he] using hf v hs
    · intro v hv
      have hs : v ∉ S := fun hs => Finset.disjoint_left.1 h hv hs
      simpa [clamp, hv] using hf v hs

/-- Explicit conditioning versus an indicator factor. The summed coordinate sets are disjoint;
no normalization or positive-mass assumption is needed for this unnormalized identity. -/
theorem marg_indicator_eq_clamp {E S : Finset bn.V} (h : Disjoint E S)
  (F : bn.Assignment → ℝ) (observed base : bn.Assignment) :
  marg (S ∪ E) (fun x => F x * indicator E observed x) base =
  marg S (fun x => F (clamp E observed x)) base := by
  rw [marg_clamp h]
  unfold marg
  rw [← fibre_condition h observed base, Finset.sum_filter]
  apply Finset.sum_congr rfl
  intro x _
  simp [indicator, mul_ite]

/-- Full evidence-conditioned enumeration needs only the unobserved coordinates. -/
theorem sum_conditioned_eq_indicator (E : Finset bn.V) (F : bn.Assignment → ℝ)
  (observed : bn.Assignment) :
  (∑ x : PartialAssignment bn Eᶜ, F (patch Eᶜ observed x)) =
  ∑ x, F x * indicator E observed x := by
  rw [sum_partial_eq_marg]
  have h := marg_indicator_eq_clamp (S := Eᶜ) (E := E)
  (Finset.disjoint_left.2 fun _ he hc => Finset.mem_compl.1 hc he) F observed observed
  have he : clamp E observed observed = observed := by funext v; simp [clamp]
  have hs : Eᶜ ∪ E = Finset.univ := by
    ext v
    by_cases hv : v ∈ E <;> simp [hv]

```

```

rw [hs, marg_univ, marg_clamp
  (Finset.disjoint_left.2 fun _ he hc => Finset.mem_compl.1 hc he), he] at h
exact h.symm

end BayesianNetworksProofs.FinBayesNet

```

Finite posterior distributions and exact feasibility

```

import BayesianNetworksProofs.Finite.Assignments
import BayesianNetworksProofs.Finite.VariableElimination
import Mathlib.Algebra.Order.BigOperators.GroupWithZero.Finset
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Data.Fintype.BigOperators

```

Normalization returns a probability distribution only at positive mass. With nonnegative weights, zero mass is exactly the absence of a positive-weight assignment. Query distributions live on partial assignments, so eliminated coordinates are not counted repeatedly.

The VE theorem uses the existing bucket algorithm and joint semantics. Explicit conditioning is modeled by clamping evidence coordinates and dropping them from factor scopes; its finite elimination result equals indicator-likelihood elimination on the original model.

```

set_option autoImplicit false

namespace BayesianNetworksProofs

noncomputable section

namespace FiniteDistribution

variable {A B : Type} [Fintype A] [Fintype B] [DecidableEq B]

def mass (w : A → ℝ) : ℝ := ∑ a, w a

structure Distribution (A : Type) [Fintype A] where
  pmf : A → ℝ
  nonneg : ∀ a, 0 ≤ pmf a
  normalized : ∑ a, pmf a = 1

def normalize (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) : Option (Distribution A) :=
  if h : 0 < mass w then some {
    pmf a := w a / mass w
    nonneg a := div_nonneg (hw a) h.le
    normalized := by
      simp only [div_eq_mul_inv]
      rw [← Finset.sum_mul]
      exact mul_inv_cancel₀ (ne_of_gt h)
  } else none

theorem mass_nonneg (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) : 0 ≤ mass w :=
  Finset.sum_nonneg fun a _ => hw a

theorem mass_zero_iff (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) :
  mass w = 0 ↔ ∀ a, w a = 0 := by
  simp [mass] using Finset.sum_eq_zero_iff_of_nonneg (s := Finset.univ) (fun a _ => hw a)

theorem mass_pos_iff (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) :
  0 < mass w ↔ ∃ a, 0 < w a := by
  simp [mass] using Finset.sum_pos_iff_of_nonneg (s := Finset.univ) (fun a _ => hw a)

theorem normalize_none_iff (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) :
  normalize w hw = none ↔ mass w = 0 := by
  unfold normalize
  by_cases hp : 0 < mass w

```

```

• simp [hp, ne_of_gt hp]
• have hz := le_antisymm (le_of_not_gt hp) (mass_nonneg w hw)
  simp [hz]

theorem normalize_value (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) (d : Distribution A)
  (h : normalize w hw = some d) : ∀ a, d.pmf a = w a / mass w := by
  unfold normalize at h
  split_ifs at h with hp
  • cases Option.some.inj h
    exact fun _ => rfl

def pushWeight (f : A → B) (w : A → ℝ) (b : B) : ℝ :=
  ∑ a, if f a = b then w a else 0

omit [Fintype B] in
theorem pushWeight_nonneg (f : A → B) (w : A → ℝ) (hw : ∀ a, 0 ≤ w a) :
  ∀ b, 0 ≤ pushWeight f w b := by
  intro b
  apply Finset.sum_nonneg
  intro a _
  split_ifs
  • exact hw a
  • exact le_refl 0

theorem mass_pushWeight (f : A → B) (w : A → ℝ) :
  mass (pushWeight f w) = mass w := by
  unfold mass pushWeight
  rw [Finset.sum_comm]
  apply Finset.sum_congr rfl
  intro a _
  simp

end FiniteDistribution

namespace FinBayesNet

open FiniteDistribution

variable {bn : FinBayesNet}

def likelihoodWeight (p : bn.Assignment → ℝ) (E : Finset bn.V) (observed x : bn.Assignment) : ℝ :=
  p x * indicator E observed x

theorem likelihoodWeight_nonneg (p : bn.Assignment → ℝ) (hp : ∀ x, 0 ≤ p x)
  (E : Finset bn.V) (observed : bn.Assignment) : ∀ x, 0 ≤ likelihoodWeight p E observed x := by
  intro x
  unfold likelihoodWeight indicator
  split_ifs <=> simp_all

def posterior (Q E : Finset bn.V) (observed : bn.Assignment)
  (p : bn.Assignment → ℝ) (hp : ∀ x, 0 ≤ p x) :
  Option (Distribution (PartialAssignment bn Q)) :=
  normalize (pushWeight (restrictTo Q) (likelihoodWeight p E observed))
  (pushWeight_nonneg _ _ (likelihoodWeight_nonneg p hp E observed))

/-- Impossible evidence is rejected even for an empty query or a query consisting entirely
of observed variables; feasibility concerns the whole joint, not just the requested component. -/
theorem posterior_none_iff (Q E : Finset bn.V) (observed : bn.Assignment)
  (p : bn.Assignment → ℝ) (hp : ∀ x, 0 ≤ p x) :
  posterior Q E observed p hp = none ↔ ¬∃ x, agrees E observed x ∧ 0 < p x := by
  rw [posterior, normalize_none_iff, mass_pushWeight]
  have hmass := mass_nonneg _ (likelihoodWeight_nonneg p hp E observed)
  have hz : mass (likelihoodWeight p E observed) = 0 ↔
    ¬0 < mass (likelihoodWeight p E observed) :=
    ⟨fun he => by rw [he]; simp, fun he => le_antisymm (le_of_not_gt he) hmass⟩
  rw [hz, mass_pos_iff _ (likelihoodWeight_nonneg p hp E observed)]

```

```

apply not_congr
constructor
· rintro ⟨x, hx⟩
  by_cases he : agrees E observed x
  · have hi : indicator E observed x = 1 := if_pos he
    exact ⟨x, he, by simp only [likelihoodWeight, hi, mul_one] using hx⟩
  · have hi : indicator E observed x = 0 := if_neg he
    simp only [likelihoodWeight, hi, mul_zero, lt_self_iff_false] at hx
· rintro ⟨x, he, hp⟩
  have hi : indicator E observed x = 1 := if_pos he
  exact ⟨x, by simp only [likelihoodWeight, hi, mul_one] using hp⟩

theorem fibre_query (Q : Finset bn.V) (x : bn.Assignment) :
  fibre Qc x = Finset.univ.filter (fun y ⇒ restrictTo Q y = restrictTo Q x) := by
ext y
simp only [mem_fibre, Finset.mem_filter, Finset.mem_univ, true_and]
constructor
· intro h
  funext v
  exact h v.val (by simp)
· intro h v hv
  exact congrFun h ⟨v, by simp using hv⟩

theorem marginal_eq_query_weight (Q : Finset bn.V) (p : bn.Assignment → ℝ) (x : bn.Assignment) :
  marg Qc p x = pushWeight (restrictTo Q) p (restrictTo Q x) := by
rw [marg, fibre_query, Finset.sum_filter]
rfl

def evidenceFactor (E : Finset bn.V) (observed : bn.Assignment) : Factor bn ℝ where
scope := E
value := indicator E observed
dependsOn x y h := by
  have he : agrees E observed x ↔ agrees E observed y := by
    constructor
    · intro hx v hv
      rw [← h v hv]
      exact hx v hv
    · intro hy v hv
      rw [h v hv]
      exact hy v hv
  simp only [indicator, he]

theorem joint_nonneg (κ : bn.Kernel ℝ) (hκ : ∀ m x a, 0 ≤ κ m x a) (x : bn.Assignment) :
  0 ≤ joint κ x := Finset.prod_nonneg fun m _ ⇒ hκ m x _

/-- The already-defined VE driver, with a real evidence indicator factor, computes the
unnormalized numerator of the genuine query posterior. -/
theorem ve_posterior_numerator (κ : bn.Kernel ℝ) (hloc : ∀ m, Local κ m)
(Q E : Finset bn.V) (observed : bn.Assignment) (vs : List bn.V)
(hvs : vs.Nodup) (hset : vs.toFinset = Qc) (x : bn.Assignment) :
Factor.product (Factor.eliminateAll (evidenceFactor E observed :: Factor.ofKernel κ hloc) vs) x =
  pushWeight (restrictTo Q) (likelihoodWeight (joint κ) E observed) (restrictTo Q x) := by
rw [Factor.eliminateAll_correct _ vs hvs, hset]
have he : Factor.product (evidenceFactor E observed :: Factor.ofKernel κ hloc) =
  likelihoodWeight (joint κ) E observed := by
  funext y
  rw [Factor.product_cons, Factor.product_ofKernel]
  exact mul_comm _ _
rw [he, marginal_eq_query_weight]

/-- Dividing the actual VE numerator by positive evidence mass gives the normalized posterior
entry. A returned Distribution supplies nonnegativity and total mass one. -/
theorem ve_normalized_posterior (κ : bn.Kernel ℝ) (hloc : ∀ m, Local κ m)
(hκ : ∀ m x a, 0 ≤ κ m x a) (Q E : Finset bn.V) (observed : bn.Assignment)
(vs : List bn.V) (hvs : vs.Nodup) (hset : vs.toFinset = Qc)
(d : Distribution (PartialAssignment bn Q))

```

```

(hd : posterior Q E observed (joint κ) (joint_nonneg κ hκ) = some d) (x : bn.Assignment) :
Factor.product (Factor.eliminateAll (evidenceFactor E observed :: Factor.ofKernel κ hloc) vs) x /
mass (likelihoodWeight (joint κ) E observed) = d.pmf (restrictTo Q x) := by
rw [ve_posterior_numerator κ hloc Q E observed vs hvs hset]
have h := normalize_value _ _ d hd (restrictTo Q x)
rw [mass_pushWeight] at h
exact h.symm

namespace Factor

def condition (E : Finset bn.V) (observed : bn.Assignment) (f : Factor bn R) : Factor bn R where
scope := f.scope \ E
value x := f.value (clamp E observed x)
dependsOn x y h := by
  apply f.dependsOn
  intro v hv
  by_cases he : v ∈ E
  · simp [clamp, he]
  · simp [clamp, he] using h v (Finset.mem_sdiff.2 ⟨hv, he⟩)

theorem product_condition (fs : List (Factor bn R)) (E : Finset bn.V) (observed : bn.Assignment) :
product (fs.map (condition E observed)) = fun x ⇒ product fs (clamp E observed x) := by
funext x
simp [product, condition, List.map_map, Function.comp_def]

theorem explicit_conditioning_elimination (fs : List (Factor bn R)) (E : Finset bn.V)
(observed : bn.Assignment) (vs : List bn.V) (hvs : vs.Nodup) (hd : Disjoint E vs.toFinset)
(x : bn.Assignment) :
product (eliminateAll (fs.map (condition E observed)) vs) x =
marg (vs.toFinset ∪ E) (fun y ⇒ product fs y * indicator E observed y) x := by
rw [eliminateAll_correct _ vs hvs, product_condition]
exact (marg_indicator_eq_clamp hd (product fs) observed x).symm

end Factor
end FinBayesNet
end
end BayesianNetworksProofs

```

Checked finite records, positions and repeated input slots

```

import BayesianNetworksProofs.Finite.OrderedParents
import BayesianNetworksProofs.Finite.VariableElimination
import Mathlib.Data.Fintype.EquivFin
import Mathlib.Data.List.OfFn
import Mathlib.Data.Real.Basic

```

Raw rows carry concrete finite IDs, names, references and numeric positions. `decodeId` checks external one-based IDs before constructing those finite IDs. Position validity is only a boundedness/injectivity check on each owner fibre; finiteness derives surjectivity onto every contiguous position. Input variables need not be injective: repeated slots read the same assignment coordinate, i.e. diagonal rather than independent evaluation.

Valid consists of decidable structural facts, not a compiler-correctness certificate. The compiler derives the abstract parent sets, state spaces, closedness and topological order. Raw names/references remain attached to the records; the finite-network `reduct` deliberately erases metadata. This is not a theorem about the Julia compiler or its JSON parser.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.Raw

open FinBayesNet

def decodeId (n external : Nat) : Option (Fin n) :=

```

```

    if h : 0 < external ∧ external ≤ n then some (external - 1, by omega) else none

theorem decodeId_none_iff (n external : Nat) :
  decodeId n external = none ↔ external = 0 ∨ n < external := by
  unfold decodeId
  split_ifs <|> simp_all <|> omega

theorem decodeId_value {n external : Nat} {i : Fin n} (h : decodeId n external = some i) :
  i.val + 1 = external := by
  unfold decodeId at h
  split_ifs at h with hb
  · cases Option.some.inj h
    change external - 1 + 1 = external
    omega

inductive Ref
| named (key : String)
| policy (key : String)
| pointMass (state : String)
| none
deriving DecidableEq

structure VariableRow where
  name : String
  spaceRef : Ref

structure StateRow (nv : Nat) where
  var : Fin nv
  position : Nat
  name : String

structure MechanismRow (nv : Nat) where
  target : Fin nv
  name : String
  kernelRef : Ref

structure InputRow (nv nm : Nat) where
  mechanism : Fin nm
  var : Fin nv
  position : Nat

structure Network where
  nv : Nat
  ns : Nat
  nm : Nat
  ni : Nat
  vars : Fin nv → VariableRow
  states : Fin ns → StateRow nv
  mechanisms : Fin nm → MechanismRow nv
  inputs : Fin ni → InputRow nv nm
  rank : Fin nv → Fin nv

def Positioned {nr no : Nat} (owner : Fin nr → Fin no) (pos : Fin nr → Nat) : Prop :=
  (∀ r, pos r < Fintype.card {s : Fin nr // owner s = owner r}) ∧
  ∀ r s, owner r = owner s → pos r = pos s → r = s

instance {nr no : Nat} (owner : Fin nr → Fin no) (pos : Fin nr → Nat) :
  Decidable (Positioned owner pos) := by unfold Positioned; infer_instance

def positionMap {nr no : Nat} (owner : Fin nr → Fin no) (pos : Fin nr → Nat)
  (h : Positioned owner pos) (o : Fin no) (r : {s : Fin nr // owner s = o}) :
  Fin (Fintype.card {s : Fin nr // owner s = o}) :=
  ⟨pos r.val, by simp only [r.property] using h.1 r.val⟩

/-- Total positional coverage follows from checked bounds, uniqueness, and equal finite
cardinalities; an ordering equivalence is not an input assumption. -/

```

```

noncomputable def positionEquiv {nr no : Nat} (owner : Fin nr → Fin no) (pos : Fin nr → Nat)
  (h : Positioned owner pos) (o : Fin no) :
  {s : Fin nr // owner s = o} ≈ Fin (Fintype.card {s : Fin nr // owner s = o}) :=
  Equiv.ofBijective (positionMap owner pos h o) ((Fintype.bijective_iff_injective_and_card _).2 <
    (by
      intro r s he
      apply Subtype.ext
      exact h.2 r.val s.val (r.property.trans s.property.symm) (congrArg Fin.val he)),
    by simp))

namespace Network

def stateOwner (r : Network) (s : Fin r.ns) := (r.states s).var
def inputOwner (r : Network) (i : Fin r.ni) := (r.inputs i).mechanism
def stateCount (r : Network) (v : Fin r.nv) := Fintype.card {s : Fin r.ns // r.stateOwner s = v}
def inputCount (r : Network) (m : Fin r.nm) := Fintype.card {i : Fin r.ni // r.inputOwner i = m}

structure Valid (r : Network) : Prop where
  state_positions : Positioned r.stateOwner (fun s ⇒ (r.states s).position)
  input_positions : Positioned r.inputOwner (fun i ⇒ (r.inputs i).position)
  nonempty_states : ∀ v, 0 < r.stateCount v
  state_names : ∀ s t, (r.states s).var = (r.states t).var →
    (r.states s).name = (r.states t).name → s = t
  generators : Function.Bijective (fun m ⇒ (r.mechanisms m).target)
  rank_injective : Function.Injective r.rank
  causal_rank : ∀ i, r.rank (r.inputs i).var < r.rank (r.mechanisms (r.inputs i).mechanism).target

theorem valid_iff (r : Network) : r.Valid ↔
  Positioned r.stateOwner (fun s ⇒ (r.states s).position) ∧
  Positioned r.inputOwner (fun i ⇒ (r.inputs i).position) ∧
  (∀ v, 0 < r.stateCount v) ∧
  (∀ s t, (r.states s).var = (r.states t).var → (r.states s).name = (r.states t).name → s = t) ∧
  Function.Bijective (fun m ⇒ (r.mechanisms m).target) ∧ Function.Injective r.rank ∧
  (∀ i, r.rank (r.inputs i).var < r.rank (r.mechanisms (r.inputs i).mechanism).target) :=
  ⟨fun h ⇒ ⟨h.state_positions, h.input_positions, h.nonempty_states, h.state_names,
    h.generators, h.rank_injective, h.causal_rank⟩,
    fun h ⇒ ⟨h.1, h.2.1, h.2.2.1, h.2.2.2.1, h.2.2.2.2.1, h.2.2.2.2.2⟩⟩

instance (r : Network) : Decidable r.Valid := by
  letI : Decidable (Function.Bijective (fun m ⇒ (r.mechanisms m).target)) := by
    unfold Function.Bijective Function.Injective Function.Surjective
    infer_instance
  letI : Decidable (Function.Injective r.rank) := by
    unfold Function.Injective
    infer_instance
  exact decidable_of_iff _ (valid_iff r).symm

def check (r : Network) : Bool := decide r.Valid

theorem check_iff (r : Network) : r.check = true ↔ r.Valid := by simp [check]

noncomputable section

@[reducible] def compile (r : Network) (h : r.Valid) : FinBayesNet where
  V := Fin r.nv
  M := Fin r.nm
  states v := Fin (r.stateCount v)
  nonemptyS v := ⟨0, h.nonempty_states v⟩
  target m := (r.mechanisms m).target
  parents m := (Finset.univ.filter fun i ⇒ (r.inputs i).mechanism = m).image
    (fun i ⇒ (r.inputs i).var)

def stateOrder (r : Network) (h : r.Valid) (v : Fin r.nv) :
  {s : Fin r.ns // (r.states s).var = v} ≈ Fin (r.stateCount v) :=
  positionEquiv r.stateOwner (fun s ⇒ (r.states s).position) h.state_positions v

```

```

def inputOrder (r : Network) (h : r.Valid) (m : Fin r.nm) :
  {i : Fin r.ni // (r.inputs i).mechanism = m} ≈ Fin (r.inputCount m) :=
  positionEquiv r.inputOwner (fun i => (r.inputs i).position) h.input_positions m

def stateLabel (r : Network) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) : String :=
  (r.states ((r.stateOrder h v).symm a).val).name

theorem stateLabel_injective (r : Network) (h : r.Valid) (v : Fin r.nv) :
  Function.Injective (r.stateLabel h v) := by
  intro a b he
  have hs : (r.stateOrder h v).symm a = (r.stateOrder h v).symm b := by
    apply Subtype.ext
  exact h.state_names _ _ (((r.stateOrder h v).symm a).property.trans
    ((r.stateOrder h v).symm b).property.symm) he
  exact (r.stateOrder h v).symm.injective hs

def slotVariable (r : Network) (h : r.Valid) (m : Fin r.nm) (j : Fin (r.inputCount m)) : Fin r.nv :=
  (r.inputs ((r.inputOrder h m).symm j).val).var

theorem slot_position (r : Network) (h : r.Valid) (m : Fin r.nm) (j : Fin (r.inputCount m)) :
  (r.inputs ((r.inputOrder h m).symm j).val).position = j.val :=
  congrArg Fin.val ((r.inputOrder h m).apply_symm_apply j)

theorem state_position (r : Network) (h : r.Valid) (v : Fin r.nv) (j : Fin (r.stateCount v)) :
  (r.states ((r.stateOrder h v).symm j).val).position = j.val :=
  congrArg Fin.val ((r.stateOrder h v).apply_symm_apply j)

theorem slot_image (r : Network) (h : r.Valid) (m : Fin r.nm) :
  Finset.univ.image (r.slotVariable h m) = (r.compile h).parents m := by
  ext v
  constructor
  · rintro hv
    obtain ⟨j, _, rfl⟩ := Finset.mem_image.1 hv
    exact Finset.mem_image.2 ⟨((r.inputOrder h m).symm j).val,
      Finset.mem_filter.2 ⟨Finset.mem_univ _, ((r.inputOrder h m).symm j).property⟩, rfl⟩
  · intro hv
    obtain ⟨i, hi, rfl⟩ := Finset.mem_image.1 hv
    let s : {i : Fin r.ni // (r.inputs i).mechanism = m} := ⟨i, (Finset.mem_filter.1 hi).2⟩
    refine Finset.mem_image.2 ⟨r.inputOrder h m s, Finset.mem_univ _, ?_⟩
    simp [slotVariable, s]

def rankEquiv (r : Network) (h : r.Valid) : Fin r.nv ≈ Fin r.nv :=
  Equiv.ofBijective r.rank ⟨h.rank_injective, Finite.surjective_of_injective h.rank_injective⟩

/-- A checked ranking produces the topological list, rather than assuming a compiler theorem. -/
def topological (r : Network) (h : r.Valid) : (r.compile h).TopoOrder where
  order := List.ofFn (r.rankEquiv h).symm
  nodup := List.nodup_ofFn.2 (r.rankEquiv h).symm.injective
  complete v := List.mem_ofFn.2 ⟨r.rankEquiv h v, (r.rankEquiv h).symm_apply_apply v⟩
  parents_before := by
    apply List.pairwise_ofFn.2
    intro i j hij m hm hv
    obtain ⟨slot, hs, he⟩ := Finset.mem_image.1 hv
    have heOwner := (Finset.mem_filter.1 hs).2
    have hr := h.causal_rank slot
    have hi : r.rank ((r.rankEquiv h).symm i) = i := (r.rankEquiv h).apply_symm_apply i
    have hj : r.rank ((r.rankEquiv h).symm j) = j := (r.rankEquiv h).apply_symm_apply j
    change (r.mechanisms m).target = (r.rankEquiv h).symm i at hm
    rw [heOwner, hm, he, hi, hj] at hr
    exact (not_lt_of_ge hij.le) hr
  no_self m := by
    intro hm
    obtain ⟨i, hi, he⟩ := Finset.mem_image.1 hm
    have hr := h.causal_rank i
    rw [(Finset.mem_filter.1 hi).2, he] at hr
    exact (lt_irrefl _ hr)

```

```

theorem compile_closed (r : Network) (h : r.Valid) : (r.compile h).Closed := h.generators

abbrev SlotValues (r : Network) (h : r.Valid) (m : Fin r.nm) :=
  (j : Fin (r.inputCount m)) → (r.compile h).states (r.slotVariable h m j)

def gather (r : Network) (h : r.Valid) (m : Fin r.nm) (x : (r.compile h).Assignment) :
  r.SlotValues h m := fun j ⇒ x (r.slotVariable h m j)

abbrev CPT (r : Network) (h : r.Valid) (R : Type) :=
  (m : Fin r.nm) → r.SlotValues h m → (r.compile h).states ((r.compile h).target m) → R

def kernel (r : Network) (h : r.Valid) {R : Type} (t : r.CPT h R) : (r.compile h).Kernel R :=
  fun m x ⇒ t m (r.gather h m x)

theorem kernel_local (r : Network) (h : r.Valid) {R : Type} (t : r.CPT h R) :
  ∀ m, Local (r.kernel h t) m := by
  intro m x y hx
  apply congrArg (t m)
  funext j
  apply hx
  rw [← r.slot_image h m]
  exact Finset.mem_image_of_mem _ (Finset.mem_univ j)

/-- Repeated input-variable slots are evaluated on the diagonal, not sampled independently. -/
theorem repeated_slots_diagonal (r : Network) (h : r.Valid) (m : Fin r.nm)
  (i j : Fin (r.inputCount m)) (he : r.slotVariable h m i = r.slotVariable h m j)
  (x : (r.compile h).Assignment) :
  (r.gather h m x i).val = (r.gather h m x j).val :=
  congrArg (fun v ⇒ (x v).val) he

theorem kernel_normalised (r : Network) (h : r.Valid) {R : Type} [CommSemiring R]
  (t : r.CPT h R) (ht : ∀ m z, ∑ a, t m z a = 1) :
  ∀ m, Normalised (r.kernel h t) m := fun m x ⇒ ht m (r.gather h m x)

theorem compiled_joint_normalised (r : Network) (h : r.Valid) {R : Type} [CommSemiring R]
  (t : r.CPT h R) (ht : ∀ m z, ∑ a, t m z a = 1) :
  ∑ x, joint (r.kernel h t) x = 1 :=
  sum_joint_eq_one _ (r.compile_closed h) (r.topological h)
  (r.kernel_local h t) (r.kernel_normalised h t ht)

/-- The existing valuation/factor compiler preserves the actual repeated-slot CPT product. -/
theorem factor_compilation (r : Network) (h : r.Valid) (t : r.CPT h R) :
  Factor.product (Factor.ofKernel (r.kernel h t) (r.kernel_local h t)) =
  fun x ⇒ ∏ m, t m (r.gather h m x) (x ((r.compile h).target m)) := by
  rw [Factor.product_ofKernel]
  rfl

end
end Network
end BayesianNetworksProofs.Raw

```

Finite reference tables and checked resolution

```

import BayesianNetworksProofs.Finite.RawRecords
import Mathlib.Data.List.Forall2
import Mathlib.Data.Rat.Cast.Order
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Algebra.BigOperators.Fin

```

Bindings are finite lists of named/policy references and rational columns. Coordinates are zero-based after checked external-ID conversion. Complete coordinate coverage, unique columns, output lengths and nonnegative entries are checked independently of compilation; exact normalization is a separate check. NoRef and missing bindings

are rejected, not silently replaced by a distribution. Point masses resolve through the checked state labels.

This file specifies a finite data checker/model bridge, not the correctness of Julia or of a JSON parser. Rational values may represent stored Float64 values exactly without pretending their row sums are exactly one.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.Raw

open FinBayesNet

def coordinates : List Nat → List (List Nat)
| [] ⇒ [[]]
| n :: ns ⇒ (List.range n).flatMap fun a ⇒ (coordinates ns).map (a :: ·)

theorem coordinates_ofFn (n : Nat) (sizes : Fin n → Nat) (x : (i : Fin n) → Fin (sizes i)) :
  List.ofFn (fun i ⇒ (x i).val) ∈ coordinates (List.ofFn sizes) := by
  induction n with
  | zero ⇒ simp [coordinates]
  | succ n ih ⇒
    simp only [List.ofFn_succ, coordinates, List.mem_flatMap, List.mem_range, List.mem_map]
    exact ⟨(x 0).val, (x 0).isLt, List.ofFn (fun i ⇒ (x i.succ).val),
      ih (fun i ⇒ sizes i.succ) (fun i ⇒ x i.succ), rfl⟩

structure Column where
  parents : List Nat
  weights : List ℚ
  deriving DecidableEq

structure Table where
  inputSizes : List Nat
  outputSize : Nat
  columns : List Column
  inputLabels : List (List String) := []
  outputLabels : List String := []
  deriving DecidableEq

def Table.column (t : Table) (ctx : List Nat) : Option Column :=
  t.columns.find? fun c ⇒ c.parents == ctx

def Table.read (t : Table) (ctx : List Nat) (a : Nat) : ℚ :=
  ((t.column ctx).getD [], []).weights.getD a 0

def Table.Ready (t : Table) : Prop :=
  (t.columns.map Column.parents).Nodup ∧
  (∀ ctx ∈ coordinates t.inputSizes, ∃ c ∈ t.columns, c.parents = ctx) ∧
  ∀ c ∈ t.columns, c.parents ∈ coordinates t.inputSizes ∧
  c.weights.length = t.outputSize ∧ ∀ q ∈ c.weights, 0 ≤ q

def Table.Normalized (t : Table) : Prop := ∀ c ∈ t.columns, c.weights.sum = 1

instance (t : Table) : Decidable t.Ready := by unfold Table.Ready; infer_instance
instance (t : Table) : Decidable t.Normalized := by unfold Table.Normalized; infer_instance

theorem find_mem {A : Type} (p : A → Bool) (xs : List A) (a : A)
  (h : xs.find? p = some a) : a ∈ xs ∧ p a = true := by
  induction xs with
  | nil ⇒ simp at h
  | cons b xs ih ⇒
    by_cases hb : p b
    • simp [List.find?, hb] at h
      subst a
      exact ⟨List.mem_cons_self .., hb⟩
    • simp [List.find?, hb] at h
      have hi := ih h
      exact ⟨List.mem_cons_of_mem b hi.1, hi.2⟩

```

```

theorem find_exists {A : Type} (p : A → Bool) (xs : List A)
  (h : ∃ a ∈ xs, p a = true) : ∃ a, xs.find? p = some a := by
  induction xs with
  | nil ⇒ simp at h
  | cons b xs ih ⇒
    by_cases hb : p b
    · exact ⟨b, by simp [List.find?, hb]⟩
    · obtain ⟨a, ha, hp⟩ := h
      have ht : a ∈ xs := (List.mem_cons.1 ha).resolve_left (fun he ⇒ hb (he ▸ hp))
      obtain ⟨c, hc⟩ := ih ⟨a, ht, hp⟩
      exact ⟨c, by simp [List.find?, hb, hc]⟩

theorem column_exists (t : Table) (h : t.Ready) (ctx : List Nat)
  (hc : ctx ∈ coordinates t.inputSizes) : ∃ c, t.column ctx = some c := by
  apply find_exists
  obtain ⟨c, hmem, he⟩ := h.2.1 ctx hc
  exact ⟨c, hmem, by simp using he⟩

theorem getD_nonnegative (xs : List ℚ) (h : ∀ q ∈ xs, 0 ≤ q) (n : Nat) :
  0 ≤ xs.getD n 0 := by
  induction xs generalizing n with
  | nil ⇒ simp
  | cons q xs ih ⇒
    cases n with
    | zero ⇒ exact h q (List.mem_cons_self ..)
    | succ n ⇒ simp using ih (fun r hr ⇒ h r (List.mem_cons_of_mem q hr)) n

theorem sum_getD (xs : List ℚ) : (∑ i : Fin xs.length, xs.getD i.val 0) = xs.sum := by
  induction xs with
  | nil ⇒ simp
  | cons q xs ih ⇒
    rw [List.length_cons, Fin.sum_univ_succ]
    simp

theorem Table.read_nonnegative (t : Table) (h : t.Ready) (ctx : List Nat)
  (hc : ctx ∈ coordinates t.inputSizes) (a : Nat) : 0 ≤ t.read ctx a := by
  obtain ⟨c, he⟩ := column_exists t h ctx hc
  have hmem := (find_mem _ t.columns c he).1
  simp [Table.read, he] using getD_nonnegative c.weights (h.2.2 c hmem).2.2 a

theorem Table.read_normalized (t : Table) (h : t.Ready) (hn : t.Normalized)
  (ctx : List Nat) (hc : ctx ∈ coordinates t.inputSizes) :
  (∑ a : Fin t.outputSize, t.read ctx a.val) = 1 := by
  obtain ⟨c, he⟩ := column_exists t h ctx hc
  have hmem := (find_mem _ t.columns c he).1
  have hlen := (h.2.2 c hmem).2.1
  simp only [Table.read, he, Option.getD_some]
  rw [← hlen, sum_getD]
  exact hn c hmem

structure Binding where
  ref : Ref
  table : Table
  deriving DecidableEq

abbrev Bank := List Binding

def resolve (bank : Bank) (ref : Ref) : Option Table :=
  (bank.find? fun b ⇒ b.ref == ref).map Binding.table

theorem resolve_has_key (bank : Bank) (ref : Ref) (t : Table) (h : resolve bank ref = some t) :
  ∃ b ∈ bank, b.ref = ref ∧ b.table = t := by
  unfold resolve at h
  obtain ⟨b, hb, ht⟩ := Option.map_eq_some_iff.1 h
  have hm := find_mem _ bank b hb
  exact ⟨b, hm.1, by simp using hm.2, ht⟩

```

namespace Network

noncomputable section

```
def slotSizes (r : Network) (h : r.Valid) (m : Fin r.nm) : List Nat :=
  List.ofFn fun j => r.stateCount (r.slotVariable h m j)
```

```
def slotIndices (r : Network) (h : r.Valid) (m : Fin r.nm) (x : r.SlotValues h m) : List Nat :=
  List.ofFn fun j => (x j).val
```

```
theorem slotIndices_valid (r : Network) (h : r.Valid) (m : Fin r.nm) (x : r.SlotValues h m) :
  r.slotIndices h m x ∈ coordinates (r.slotSizes h m) :=
  coordinates_ofFn _ _ x
```

```
def Ready (r : Network) (h : r.Valid) (bank : Bank) : Prop :=
  (bank.map Binding.ref).Nodup ∧ ∀ m,
    match (r.mechanisms m).kernelRef with
    | .none => False
    | .pointMass label => ∃ a : Fin (r.stateCount (r.mechanisms m).target),
      r.stateLabel h (r.mechanisms m).target a = label
    | ref => match resolve bank ref with
      | none => False
      | some t => t.inputSizes = r.slotSizes h m ∧
        t.outputSize = r.stateCount (r.mechanisms m).target ∧ t.Ready
```

```
def Normalized (r : Network) (bank : Bank) : Prop :=
  ∀ m t, resolve bank (r.mechanisms m).kernelRef = some t → t.Normalized
```

```
def resolvedCPT (r : Network) (h : r.Valid) (bank : Bank) : r.CPT h ℚ :=
  fun m x a =>
    match (r.mechanisms m).kernelRef with
    | .none => 0
    | .pointMass label => if r.stateLabel h (r.mechanisms m).target a = label then 1 else 0
    | ref => match resolve bank ref with
      | none => 0
      | some t => t.read (r.slotIndices h m x) a.val
```

```
theorem resolvedCPT_nonnegative (r : Network) (h : r.Valid) (bank : Bank) (hb : r.Ready h bank) :
  ∀ m x a, 0 ≤ r.resolvedCPT h bank m x a := by
  intro m x a
  have hm := hb.2 m
  unfold resolvedCPT
  cases hr : (r.mechanisms m).kernelRef with
  | none => simp [hr] at hm
  | pointMass label =>
    dsimp only
    split_ifs <|> norm_num
  | named key | policy key =>
    cases he : resolve bank (r.mechanisms m).kernelRef with
    | none =>
      simp only [hr] at he
      simp [hr, he] at hm
    | some t =>
      simp only [hr] at he
      have ht : t.inputSizes = r.slotSizes h m ∧
        t.outputSize = r.stateCount (r.mechanisms m).target ∧ t.Ready := by simpa [hr, he] using hm
      simpa only [he] using t.read_nonnegative ht.2.2 - (ht.1.symm ▸ r.slotIndices_valid h m x) a.val
```

```
theorem resolvedCPT_normalized (r : Network) (h : r.Valid) (bank : Bank) (hb : r.Ready h bank)
  (hn : r.Normalized bank) : ∀ m x, ∑ a, r.resolvedCPT h bank m x a = 1 := by
  intro m x
  change (∑ a : Fin (r.stateCount (r.mechanisms m).target), r.resolvedCPT h bank m x a) = 1
  have hm := hb.2 m
  cases hr : (r.mechanisms m).kernelRef with
  | none => simp [hr] at hm
```

```

| pointMass label ⇒
  obtain ⟨a₀, ha⟩ : ∃ a, r.stateLabel h (r.mechanisms m).target a = label := by simp [hr] using hm
  have he : ∀ a, r.stateLabel h (r.mechanisms m).target a = label ↔ a = a₀ := by
    intro a
    rw [← ha, (r.stateLabel_injective h _).eq_iff]
  simp [resolvedCPT, hr, he]
| named key | policy key ⇒
  cases ht : resolve bank (r.mechanisms m).kernelRef with
  | none ⇒
    simp only [hr] at ht
    simp [hr, ht] at hm
  | some t ⇒
    simp only [hr] at ht
    have hv : t.inputSizes = r.slotSizes h m ∧
      t.outputSize = r.stateCount (r.mechanisms m).target ∧ t.Ready := by simp [hr, ht] using hm
    have hn' : t.Normalized := hn m t (by simp [hr] using ht)
    have hs := t.read_normalized hv.2.2 hn' _ (hv.1.symm ▸ r.slotIndices_valid h m x)
    rw [hv.2.1] at hs
    simp only [resolvedCPT, hr, ht] using hs

/-- Checked reference resolution plus checked column normalization yields a normalized joint;
the compiler is not assumed correct, and repeated parent slots are evaluated diagonally. -/
theorem resolved_joint_normalized (r : Network) (h : r.Valid) (bank : Bank)
  (hb : r.Ready h bank) (hn : r.Normalized bank) :
  ∑ x, joint (r.kernel h (r.resolvedCPT h bank)) x = 1 :=
  r.compiled_joint_normalised h _ (r.resolvedCPT_normalized h bank hb hn)

end

/-- Executable finite search by owner and position, before any ordering equivalence is built. -/
def rawInputAt (r : Network) (m : Fin r.nm) (position : Nat) : Option (Fin r.ni) :=
  (List.finRange r.ni).find? fun i ⇒
    decide ((r.inputs i).mechanism = m ∧ (r.inputs i).position = position)

def rawSlotSize (r : Network) (m : Fin r.nm) (position : Nat) : Nat :=
  match r.rawInputAt m position with
  | none ⇒ 0
  | some i ⇒ r.stateCount (r.inputs i).var

def rawSlotSizes (r : Network) (m : Fin r.nm) : List Nat :=
  List.ofFn fun i : Fin (r.inputCount m) ⇒ r.rawSlotSize m i.val

theorem rawSlotSize_eq (r : Network) (h : r.Valid) (m : Fin r.nm) (i : Fin (r.inputCount m)) :
  r.rawSlotSize m i.val = r.stateCount (r.slotVariable h m i) := by
  let wanted := (r.inputOrder h m).symm i
  have hex : ∃ j, r.rawInputAt m i.val = some j := by
    apply find_exists
    refine ⟨wanted.val, by simp, ?_⟩
    simp only [decide_eq_true_eq]
    exact ⟨wanted.property, r.slot_position h m i⟩
  obtain ⟨j, hj⟩ := hex
  have hfound := (find_mem _ (List.finRange r.ni) j hj).2
  have hpair : (r.inputs j).mechanism = m ∧ (r.inputs j).position = i.val := by simp using hfound
  have he : j = wanted.val := h.input_positions.2 j wanted.val
  (hpair.1.trans wanted.property.symm) (hpair.2.trans (r.slot_position h m i).symm)
  simp only [rawSlotSize, hj, he]
  rfl

theorem rawSlotSizes_eq (r : Network) (h : r.Valid) (m : Fin r.nm) :
  r.rawSlotSizes m = r.slotSizes h m := by
  apply congrArg List.ofFn
  funext i
  exact r.rawSlotSize_eq h m i

def AxisMatch (r : Network) (m : Fin r.nm) (t : Table) : Prop :=
  t.outputLabels.length = r.stateCount (r.mechanisms m).target ∧

```

```

(∀ s, (r.states s).var = (r.mechanisms m).target →
  t.outputLabels.getD (r.states s).position "" = (r.states s).name) ∧
t.inputLabels.length = r.inputCount m ∧
∀ i, (r.inputs i).mechanism = m →
  (t.inputLabels.getD (r.inputs i).position []).length = r.stateCount (r.inputs i).var ∧
  ∀ s, (r.states s).var = (r.inputs i).var →
    (t.inputLabels.getD (r.inputs i).position []).getD (r.states s).position "" = (r.states
← s).name

instance (r : Network) (m : Fin r.nm) (t : Table) : Decidable (r.AxisMatch m t) := by
  unfold AxisMatch
  infer_instance

theorem AxisMatch.output (r : Network) (h : r.Valid) (m : Fin r.nm) (t : Table)
  (ht : r.AxisMatch m t) (a : Fin (r.stateCount (r.mechanisms m).target)) :
  t.outputLabels.getD a.val "" = r.stateLabel h (r.mechanisms m).target a := by
  have he := ht.2.1 ((r.stateOrder h _).symm a).val ((r.stateOrder h _).symm a).property
  rw [r.state_position h _ a] at he
  exact he

theorem AxisMatch.input (r : Network) (h : r.Valid) (m : Fin r.nm) (t : Table)
  (ht : r.AxisMatch m t) (j : Fin (r.inputCount m))
  (a : Fin (r.stateCount (r.slotVariable h m j))) :
  (t.inputLabels.getD j.val []).getD a.val "" = r.stateLabel h (r.slotVariable h m j) a := by
  let i := (r.inputOrder h m).symm j
  have hi := ht.2.2.2 i.val i.property
  have he := hi.2 ((r.stateOrder h _).symm a).val ((r.stateOrder h _).symm a).property
  change (t.inputLabels.getD (r.inputs i.val).position []).getD
    (r.states ((r.stateOrder h (r.slotVariable h m j)).symm a).val).position "" = _ at he
  rw [r.state_position h _ a, r.slot_position h m j] at he
  exact he

def optionReady (r : Network) (m : Fin r.nm) : Option Table → Prop
| none ⇒ False
| some t ⇒ (t.inputSizes = r.rawSlotSizes m ∧
  t.outputSize = r.stateCount (r.mechanisms m).target ∧ t.Ready) ∧ r.AxisMatch m t

instance (r : Network) (m : Fin r.nm) (o : Option Table) : Decidable (r.optionReady m o) :=
  match o with
| none ⇒ isFalse (fun h ⇒ h)
| some t ⇒ inferInstanceAs (Decidable
  ((t.inputSizes = r.rawSlotSizes m ∧ t.outputSize = r.stateCount (r.mechanisms m).target ∧
← t.Ready) ∧
  r.AxisMatch m t))

def rawReferenceReady (r : Network) (bank : Bank) (m : Fin r.nm) : Prop :=
  match (r.mechanisms m).kernelRef with
| .none ⇒ False
| .pointMass label ⇒ ∃ s : Fin r.ns,
  (r.states s).var = (r.mechanisms m).target ∧ (r.states s).name = label
| .named key ⇒ r.optionReady m (resolve bank (.named key))
| .policy key ⇒ r.optionReady m (resolve bank (.policy key))

instance (r : Network) (bank : Bank) (m : Fin r.nm) : Decidable (r.rawReferenceReady bank m) := by
  unfold rawReferenceReady
  exact match (r.mechanisms m).kernelRef with
| .none ⇒ isFalse (fun h ⇒ h)
| .pointMass label ⇒ inferInstanceAs (Decidable (∃ s : Fin r.ns,
  (r.states s).var = (r.mechanisms m).target ∧ (r.states s).name = label))
| .named key ⇒ inferInstanceAs (Decidable (r.optionReady m (resolve bank (.named key))))
| .policy key ⇒ inferInstanceAs (Decidable (r.optionReady m (resolve bank (.policy key))))

def RawReady (r : Network) (bank : Bank) : Prop :=
  (bank.map Binding.ref).Nodup ∧ ∀ m, r.rawReferenceReady bank m

def optionNormalized : Option Table → Prop

```

```

| none ⇒ True
| some t ⇒ t.Normalized

instance (o : Option Table) : Decidable (optionNormalized o) :=
  match o with
  | none ⇒ isTrue trivial
  | some t ⇒ inferInstanceAs (Decidable t.Normalized)

def rawReferenceNormalized (r : Network) (bank : Bank) (m : Fin r.nm) : Prop :=
  optionNormalized (resolve bank (r.mechanisms m).kernelRef)

instance (r : Network) (bank : Bank) (m : Fin r.nm) : Decidable (r.rawReferenceNormalized bank m) :=
  inferInstanceAs (Decidable (optionNormalized (resolve bank (r.mechanisms m).kernelRef)))

def RawNormalized (r : Network) (bank : Bank) : Prop :=
  ∀ m, r.rawReferenceNormalized bank m

instance (r : Network) (bank : Bank) : Decidable (r.RawReady bank) := by
  unfold RawReady
  infer_instance

instance (r : Network) (bank : Bank) : Decidable (r.RawNormalized bank) := by
  unfold RawNormalized
  infer_instance

def checkReady (r : Network) (bank : Bank) : Bool := decide (r.RawReady bank)
def checkNormalized (r : Network) (bank : Bank) : Bool := decide (r.RawNormalized bank)

theorem rawReady_sound (r : Network) (h : r.Valid) (bank : Bank) (hb : r.RawReady bank) :
  r.Ready h bank := by
  refine ⟨hb.1, ?_⟩
  intro m
  have hm := hb.2 m
  unfold rawReferenceReady at hm
  cases hr : (r.mechanisms m).kernelRef with
  | none ⇒ simp [hr] at hm
  | pointMass label ⇒
    obtain ⟨s, hs, hn⟩ : ∃ s, (r.states s).var = (r.mechanisms m).target ∧
      (r.states s).name = label := by simpa [hr] using hm
    refine ⟨r.stateOrder h _ ⟨s, hs⟩, ?_⟩
    simpa [stateLabel] using hn
  | named key | policy key ⇒
    cases ht : resolve bank (r.mechanisms m).kernelRef with
    | none ⇒
      simp only [hr] at ht
      simp [hr, ht, optionReady] at hm
    | some t ⇒
      simp only [hr] at ht
      have hv' : (t.inputSizes = r.rawSlotSizes m ∧
        t.outputSize = r.stateCount (r.mechanisms m).target ∧ t.Ready) ∧ r.AxisMatch m t := by
        simpa [hr, ht, optionReady] using hm
      have hv := hv'.1
      simpa only [ht] using And.intro (hv.1.trans (r.rawSlotSizes_eq h m)) hv.2

theorem rawNormalized_sound (r : Network) (bank : Bank) (hn : r.RawNormalized bank) :
  r.Normalized bank := by
  intro m t ht
  have hm := hn m
  simpa [rawReferenceNormalized, optionNormalized, ht] using hm

/-- A finite Boolean certificate check, not an assumed compiler theorem, establishes normalized
semantics for the raw reference-resolved, repeated-slot network. -/
theorem checked_reference_normalization (r : Network) (bank : Bank)
  (hs : r.check = true) (hb : r.checkReady bank = true) (hn : r.checkNormalized bank = true) :
  let h := (r.check_iff).1 hs
  ∑ x, joint (r.kernel h (r.resolvedCPT h bank)) x = 1 := by

```

```

let h := (r.check_iff).1 hs
exact r.resolved_joint_normalized h bank (r.rawReady_sound h bank (of_decide_eq_true hb))
(r.rawNormalized_sound bank (of_decide_eq_true hn))

noncomputable def realKernel (r : Network) (h : r.Valid) (bank : Bank) : (r.compile h).Kernel ℝ :=
r.kernel h (fun m x a => (r.resolvedCPT h bank m x a : ℝ))

theorem realKernel_local (r : Network) (h : r.Valid) (bank : Bank) :
  ∀ m, Local (r.realKernel h bank) m := r.kernel_local h _

theorem realKernel_nonnegative (r : Network) (h : r.Valid) (bank : Bank) (hb : r.RawReady bank) :
  ∀ m x a, 0 ≤ r.realKernel h bank m x a := by
  intro m x a
  change (0 : ℝ) ≤ (r.resolvedCPT h bank m (r.gather h m x) a : ℝ)
  exact_mod_cast r.resolvedCPT_nonnegative h bank (r.rawReady_sound h bank hb) m (r.gather h m x) a

theorem realKernel_normalized (r : Network) (h : r.Valid) (bank : Bank)
  (hb : r.RawReady bank) (hn : r.RawNormalized bank) :
  ∀ m, Normalised (r.realKernel h bank) m := by
  intro m x
  have hs := r.resolvedCPT_normalized h bank (r.rawReady_sound h bank hb)
  (r.rawNormalized_sound bank hn) m (r.gather h m x)
  change (∑ a : Fin (r.stateCount (r.mechanisms m).target),
    (r.resolvedCPT h bank m (r.gather h m x) a : ℝ)) = 1
  have hr := congrArg (Rat.castHom ℝ) hs
  rw [map_sum, map_one] at hr
  exact hr

end Network
end BayesianNetworksProofs.Raw

```

Scoped factor marginalization

```

import BayesianNetworksProofs.Finite.Assignments
import BayesianNetworksProofs.Finite.VariableElimination
import Mathlib.Data.List.Forall2
import Mathlib.Algebra.Order.BigOperators.Group.Finset

```

project sums only the factor coordinates being removed, not the full joint state space. The private-variable replacement law is the algebra used in junction-tree message passing. It permits arbitrary real entries; nonnegativity is needed later only for posterior semantics.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet.Factor

noncomputable section
variable {bn : FinBayesNet}

def unit : Factor bn ℝ where
  scope := ∅
  value _ := 1
  dependsOn := fun _ _ => rfl

def multiply (f g : Factor bn ℝ) : Factor bn ℝ where
  scope := f.scope ∪ g.scope
  value x := f.value x * g.value x
  dependsOn x y h := by
    rw [f.dependsOn x y (fun v hv => h v (Finset.mem_union_left _ hv)),
      g.dependsOn x y (fun v hv => h v (Finset.mem_union_right _ hv))]

def combine : List (Factor bn ℝ) → Factor bn ℝ
| [] => unit
| f :: fs => multiply f (combine fs)

```

```

@[ext] theorem ext {f g : Factor bn R} (hs : f.scope = g.scope) (hv : f.value = g.value) :
  f = g := by
  cases f
  cases g
  cases hs
  cases hv
  rfl

theorem multiply_comm (f g : Factor bn R) : multiply f g = multiply g f := by
  apply ext
  · exact Finset.union_comm _ _
  · funext x
    exact mul_comm _ _

theorem multiply_assoc (f g h : Factor bn R) :
  multiply (multiply f g) h = multiply f (multiply g h) := by
  apply ext
  · exact Finset.union_assoc _ _ _
  · funext x
    exact mul_assoc _ _ _

theorem unit_multiply (f : Factor bn R) : multiply unit f = f := by
  apply ext
  · simp [multiply, unit]
  · funext x
    simp [multiply, unit]

theorem multiply_unit (f : Factor bn R) : multiply f unit = f := by
  rw [multiply_comm, unit_multiply]

theorem multiply_left_comm (f g h : Factor bn R) :
  multiply f (multiply g h) = multiply g (multiply f h) := by
  rw [← multiply_assoc, multiply_comm f g, multiply_assoc]

instance : Std.Associative (multiply (bn := bn)) := ⟨multiply_assoc⟩
instance : Std.Commutative (multiply (bn := bn)) := ⟨multiply_comm⟩

theorem combine_value (fs : List (Factor bn R)) : (combine fs).value = product fs := by
  induction fs with
  | nil => rfl
  | cons f fs ih =>
    funext x
    change f.value x * (combine fs).value x = f.value x * product fs x
    rw [ih]

theorem combine_scope (fs : List (Factor bn R)) : (combine fs).scope = jointScope fs := by
  induction fs with
  | nil => rfl
  | cons f fs ih => exact congrArg (fun S => f.scope ∪ S) ih

def project (Q : Finset bn.V) (f : Factor bn R) : Factor bn R where
  scope := f.scope ∩ Q
  value x := ∑ a : PartialAssignment bn (f.scope \ Q), f.value (patch (f.scope \ Q) x a)
  dependsOn x y h := by
    apply Finset.sum_congr rfl
    intro a _
    apply f.dependsOn
    intro v hv
    by_cases he : v ∈ f.scope \ Q
    · simp [patch, he]
    · have hq : v ∈ Q := by
        by_contra hq
        exact he (Finset.mem_sdiff.2 ⟨hv, hq⟩)
    simp [patch, he] using h v (Finset.mem_inter.2 ⟨hv, hq⟩)

```

```

theorem project_value (Q : Finset bn.V) (f : Factor bn R) (x : bn.Assignment) :
  (project Q f).value x = marg (f.scope \ Q) f.value x :=
  sum_partial_eq_marg _ _

theorem project_of_scope {Q : Finset bn.V} (f : Factor bn R) (h : f.scope ⊆ Q) :
  project Q f = f := by
  apply ext
  · exact Finset.inter_eq_left.2 h
  · funext x
    rw [project_value, Finset.sdiff_eq_empty_iff_subset.2 h, marg_empty]

theorem project_nonnegative (Q : Finset bn.V) (f : Factor bn R)
  (h : ∀ x, 0 ≤ f.value x) : ∀ x, 0 ≤ (project Q f).value x := by
  intro x
  exact Finset.sum_nonneg fun a _ => h _

theorem private_factor_constant (S : Finset bn.V) (g : Factor bn R)
  (h : Disjoint S g.scope) (x y : bn.Assignment) (hy : y ∈ fibre S x) :
  g.value y = g.value x :=
  g.dependsOn y x (fun v hv => mem_fibre.1 hy v
    (fun hs => Finset.disjoint_left.1 h hs hv))

def marginal (S : Finset bn.V) (f : Factor bn R) : Factor bn R where
  scope := f.scope \ S
  value x := ∑ a : PartialAssignment bn S, f.value (patch S x a)
  dependsOn x y h := by
    apply Finset.sum_congr rfl
    intro a _
    apply f.dependsOn
    intro v hv
    by_cases hs : v ∈ S
    · simp [patch, hs]
    · simp [patch, hs] using h v (Finset.mem_sdiff.2 ⟨hv, hs⟩)

theorem marginal_value (S : Finset bn.V) (f : Factor bn R) (x : bn.Assignment) :
  (marginal S f).value x = marg S f.value x := sum_partial_eq_marg _ _

theorem marg_product_disjoint (S T : Finset bn.V) (f g : Factor bn R)
  (hST : Disjoint S T) (hTf : Disjoint T f.scope) (hSg : Disjoint S g.scope)
  (x : bn.Assignment) :
  marg (S ∪ T) (multiply f g).value x = marg S f.value x * marg T g.value x := by
  rw [marg_union_disjoint hST]
  have hinner : ∀ y, marg T (multiply f g).value y = f.value y * (marginal T g).value y := by
    intro y
    rw [marginal_value]
    exact marg_mul_left (fun z hz => private_factor_constant T f hTf y z hz)
  simp_rw [hinner]
  have hg : Disjoint S (marginal T g).scope := hSg.mono_right Finset.sdiff_subset
  unfold marg
  rw [Finset.sum_mul]
  apply Finset.sum_congr rfl
  intro y hy
  dsimp only
  rw [private_factor_constant S (marginal T g) hg x y hy, marginal_value]
  rfl

/-- Sum private variables inside one factor before combining it with the remaining factors.
This is the actual message-passing replacement law, with only structural scope conditions. -/
theorem project_private (Q C : Finset bn.V) (f g : Factor bn R)
  (h : Disjoint (f.scope \ C) (g.scope ∪ Q)) :
  project Q (multiply f g) = project Q (multiply (project C f) g) := by
  let E := f.scope \ C
  let T := ((f.scope ∩ C) ∪ g.scope) \ Q
  have he : (f.scope ∪ g.scope) \ Q = T ∪ E := by
    ext v
    have hd := Finset.disjoint_left.1 h

```

```

by_cases hf : v ∈ f.scope <=> by_cases hg : v ∈ g.scope <=>
  by_cases hc : v ∈ C <=> by_cases hq : v ∈ Q <=> simp_all [T, E]
have hdis : Disjoint T E := by
  rw [Finset.disjoint_left]
  intro v hv he
  have hf := Finset.mem_sdiff.1 he
  have ht := Finset.mem_union.1 (Finset.mem_sdiff.1 hv).1
  rcases ht with ht | ht
  · exact hf.2 (Finset.mem_inter.1 ht).2
  · exact Finset.disjoint_left.1 h he (Finset.mem_union_left _ ht)
have hg : Disjoint E g.scope := h.mono_right Finset.subset_union_left
apply ext
· ext v
  have hd := Finset.disjoint_left.1 h
  by_cases hf : v ∈ f.scope <=> by_cases hg : v ∈ g.scope <=>
    by_cases hc : v ∈ C <=> by_cases hq : v ∈ Q <=>
      simp_all [project, multiply]
· funext x
  rw [project_value, project_value]
  change marg ((f.scope ∪ g.scope) \ Q) (fun y => f.value y * g.value y) x =
    marg T (fun y => (project C f).value y * g.value y) x
  rw [he, marg_union_disjoint hdis]
  apply congrArg (fun F => marg T F x)
  funext y
  rw [project_value]
  unfold marg
  rw [Finset.sum_mul]
  apply Finset.sum_congr rfl
  intro z hz
  dsimp only
  rw [private_factor_constant E g hg y z hz]

theorem project_project (P Q : Finset bn.V) (f : Factor bn R)
  (h : f.scope ∩ P ⊆ Q) : project P (project Q f) = project P f := by
have hd : Disjoint (f.scope \ Q) (unit.scope ∪ P) := by
  apply Finset.disjoint_left.2
  intro v hv hp
  have hP : v ∈ P := by simpa [unit] using hp
  exact (Finset.mem_sdiff.1 hv).2 (h (Finset.mem_inter.2 <(Finset.mem_sdiff.1 hv).1, hP>))
have he := project_private P Q f unit hd
simpa only [multiply_unit] using he.symm

theorem project_inter (P Q : Finset bn.V) (f : Factor bn R) (h : f.scope ∩ P ⊆ Q) :
  project P f = project (P ∩ Q) f := by
have hscope : f.scope ∩ P = f.scope ∩ (P ∩ Q) := by
  ext v
  constructor
  · intro hv
    exact Finset.mem_inter.2 <(Finset.mem_inter.1 hv).1,
      Finset.mem_inter.2 <(Finset.mem_inter.1 hv).2, h hv>
  · intro hv
    exact Finset.mem_inter.2 <(Finset.mem_inter.1 hv).1,
      (Finset.mem_inter.1 (Finset.mem_inter.1 hv).2).1>
have he : f.scope \ P = f.scope \ (P ∩ Q) := by
  ext v
  constructor
  · intro hv
    exact Finset.mem_sdiff.2 <(Finset.mem_sdiff.1 hv).1,
      fun hi => (Finset.mem_sdiff.1 hv).2 (Finset.mem_inter.1 hi).1>
  · intro hv
    refine Finset.mem_sdiff.2 <(Finset.mem_sdiff.1 hv).1, ?_>
    intro hp
    exact (Finset.mem_sdiff.1 hv).2
      (Finset.mem_inter.2 <hp, h (Finset.mem_inter.2 <(Finset.mem_sdiff.1 hv).1, hp>)>))
apply ext hscope
funext x

```

```

rw [project_value, project_value, he]

/-- Independent private coordinates can be summed in separate factors. Shared coordinates
must be retained, which is exactly the structural separator condition. -/
theorem project_multiply (Q : Finset bn.V) (f g : Factor bn R)
  (h : f.scope ∩ g.scope ⊆ Q) :
  project Q (multiply f g) = multiply (project Q f) (project Q g) := by
have hf : Disjoint (f.scope \ Q) (g.scope ∪ Q) := by
  apply Finset.disjoint_left.2
  intro v hv hg
  rcases Finset.mem_union.1 hg with hg | hq
  · exact (Finset.mem_sdiff.1 hv).2 (h (Finset.mem_inter.2 ((Finset.mem_sdiff.1 hv).1, hg)))
  · exact (Finset.mem_sdiff.1 hv).2 hq
have hg : Disjoint (g.scope \ Q) ((project Q f).scope ∪ Q) := by
  apply Finset.disjoint_left.2
  intro v hv hf
  have hq : v ∈ Q := (Finset.mem_union.1 hf).elim (fun h => (Finset.mem_inter.1 h).2) id
  exact (Finset.mem_sdiff.1 hv).2 hq
rw [project_private Q Q f g hf, multiply_comm (project Q f) g,
  project_private Q Q g (project Q f) hg,
  project_of_scope _ (Finset.union_subset Finset.inter_subset_right Finset.inter_subset_right),
  multiply_comm]

end
end BayesianNetworksProofs.FinBayesNet.Factor

```

Collect/distribute on a finite junction tree

```

import BayesianNetworksProofs.Finite.FactorMarginal
import BayesianNetworksProofs.Finite.Posterior
import Mathlib.Algebra.Order.BigOperators.GroupWithZero.List

```

The working representation is a binary junction tree. Arbitrary branching is represented by copying a parent bag along a binary chain with empty local factor lists; no factors or weights are duplicated. The recursive running-intersection conditions are structural: occurrences shared by two branches meet in the parent bag, and a branch's intersection with its parent is present in the branch root.

prepare computes only local products and upward separator projections. distribute uses the cached upward messages and an incoming message to send the downward messages and produce beliefs. No whole-joint enumeration occurs in either computation. full is a proof oracle. An empty virtual-root bag connects disconnected components by scalar messages.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.Junction

open FinBayesNet FinBayesNet.Factor

noncomputable section
variable {bn : FinBayesNet} {I : Type}

inductive Tree (V I : Type)
| leaf (bag : Finset V) (assigned : List I)
| branch (bag : Finset V) (assigned : List I) (left right : Tree V I)

def Tree.bag : Tree bn.V I → Finset bn.V
| .leaf B _ => B
| .branch B _ _ => B

def Tree.vars : Tree bn.V I → Finset bn.V
| .leaf B _ => B
| .branch B _ l r => B ∪ (l.vars ∪ r.vars)

```

```

def Tree.indices : Tree bn.V I → List I
| .leaf _ fs ⇒ fs
| .branch _ fs l r ⇒ fs ++ l.indices ++ r.indices

def localFactor (F : I → Factor bn R) (fs : List I) := combine (fs.map F)

def full (F : I → Factor bn R) : Tree bn.V I → Factor bn R
| .leaf _ fs ⇒ localFactor F fs
| .branch _ fs l r ⇒ multiply (localFactor F fs) (multiply (full F l) (full F r))

def Good (F : I → Factor bn R) : Tree bn.V I → Prop
| .leaf B fs ⇒ (localFactor F fs).scope ⊆ B
| .branch B fs l r ⇒ (localFactor F fs).scope ⊆ B ∧ Good F l ∧ Good F r ∧
  l.vars ∩ B ⊆ l.bag ∧ r.vars ∩ B ⊆ r.bag ∧ l.vars ∩ r.vars ⊆ B

def decideGood (F : I → Factor bn R) : (t : Tree bn.V I) → Decidable (Good F t)
| .leaf B fs ⇒ inferInstanceAs (Decidable ((localFactor F fs).scope ⊆ B))
| .branch B fs l r ⇒
  letI := decideGood F l
  letI := decideGood F r
  inferInstanceAs (Decidable ((localFactor F fs).scope ⊆ B ∧ Good F l ∧ Good F r ∧
    l.vars ∩ B ⊆ l.bag ∧ r.vars ∩ B ⊆ r.bag ∧ l.vars ∩ r.vars ⊆ B))

def checkGood (F : I → Factor bn R) (t : Tree bn.V I) : Bool :=
  @decide (Good F t) (decideGood F t)

theorem checkGood_iff (F : I → Factor bn R) (t : Tree bn.V I) :
  checkGood F t = true ↔ Good F t := by
  letI := decideGood F t
  unfold checkGood
  exact decide_eq_true_iff

def checkAssignment [Fintype I] [DecidableEq I] (t : Tree bn.V I) : Bool :=
  decide (t.indices.Nodup ∧ ∀ i : I, i ∈ t.indices)

theorem checkAssignment_sound [Fintype I] [DecidableEq I] (t : Tree bn.V I)
  (h : checkAssignment t = true) : t.indices.Perm Finset.univ.toList := by
  have hv : t.indices.Nodup ∧ ∀ i : I, i ∈ t.indices := of_decide_eq_true h
  apply (List.perm_ext_iff_of_nodup hv.1 (Finset.nodup_toList _)).2
  intro i
  simp [hv.2 i]

theorem bag_subset_vars (t : Tree bn.V I) : t.bag ⊆ t.vars := by
  cases t
  · exact Finset.Subset.refl _
  · exact Finset.subset_union_left

theorem full_scope (F : I → Factor bn R) (t : Tree bn.V I) (h : Good F t) :
  (full F t).scope ⊆ t.vars := by
  induction t with
  | leaf B fs ⇒ exact h
  | branch B fs l r ihl ihr ⇒
    exact Finset.union_subset_union h.1 (Finset.union_subset_union (ihl h.2.1) (ihr h.2.2.1))

theorem full_value (F : I → Factor bn R) (t : Tree bn.V I) :
  (full F t).value = Factor.product (t.indices.map F) := by
  induction t with
  | leaf B fs ⇒ exact combine_value _
  | branch B fs l r ihl ihr ⇒
    funext x
    simp [full, localFactor, multiply, combine_value, Tree.indices, Factor.product,
      List.map_append, List.map_map, ihl, ihr]

inductive Prepared (bn : FinBayesNet)
| leaf (bag : Finset bn.V) (pot : Factor bn R) (up : Factor bn R)
| branch (bag : Finset bn.V) (pot : Factor bn R) (up : Factor bn R)

```

```

    (left right : Prepared bn)

def Prepared.bag : Prepared bn → Finset bn.V
| .leaf B _ _ ⇒ B
| .branch B _ _ _ ⇒ B

def Prepared.up : Prepared bn → Factor bn R
| .leaf _ _ u ⇒ u
| .branch _ _ u _ _ ⇒ u

/-- The collect pass; projections enumerate only removed factor coordinates. -/
def prepare (F : I → Factor bn R) (parent : Finset bn.V) : Tree bn.V I → Prepared bn
| .leaf B fs ⇒
  let f := localFactor F fs
  .leaf B f (project parent f)
| .branch B fs l r ⇒
  let pl := prepare F B l
  let pr := prepare F B r
  let f := localFactor F fs
  .branch B f (project parent (multiply f (multiply pl.up pr.up))) pl pr

theorem prepare_bag (F : I → Factor bn R) (P : Finset bn.V) (t : Tree bn.V I) :
  (prepare F P t).bag = t.bag := by cases t <|> rfl

theorem project_three (B : Finset bn.V) (f l r : Factor bn R)
  (hf : f.scope ⊆ B) (hlr : l.scope ∩ r.scope ⊆ B) :
  project B (multiply f (multiply l r)) =
    multiply f (multiply (project B l) (project B r)) := by
  rw [project_multiply B f _ (Finset.inter_subset_left.trans hf),
    project_of_scope f hf, project_multiply B l r hlr]

/-- Every upward message is the side-of-cut marginal of its assigned factors. -/
theorem prepare_up (F : I → Factor bn R) (t : Tree bn.V I) (h : Good F t)
  (P : Finset bn.V) (hb : t.vars ∩ P ⊆ t.bag) :
  (prepare F P t).up = project P (full F t) := by
  induction t generalizing P with
  | leaf B fs ⇒ rfl
  | branch B fs l r ihl ihr ⇒
    change project P (multiply (localFactor F fs)
      (multiply (prepare F B l).up (prepare F B r).up)) = _
  rw [ihl h.2.1 B h.2.2.2.1, ihr h.2.2.1 B h.2.2.2.2.1]
  have hc : (full F l).scope ∩ (full F r).scope ⊆ B :=
    (Finset.inter_subset_inter (full_scope F l h.2.1) (full_scope F r h.2.2.1)).trans h.2.2.2.2
  rw [← project_three B _ _ h.1 hc]
  exact project_project P B (full F (.branch B fs l r))
    ((Finset.inter_subset_inter (full_scope F _ h) (Finset.Subset.refl P)).trans hb)

theorem prepare_up_separator (F : I → Factor bn R) (t : Tree bn.V I) (h : Good F t)
  (P : Finset bn.V) (hb : t.vars ∩ P ⊆ t.bag) :
  (prepare F P t).up = project (t.bag ∩ P) (full F t) := by
  rw [prepare_up F t h P hb, project_inter P t.bag (full F t)
    ((Finset.inter_subset_inter (full_scope F t h) (Finset.Subset.refl P)).trans hb),
    Finset.inter_comm P t.bag]

/-- The distribute pass, using the collect pass's cached upward messages. -/
def distribute (incoming : Factor bn R) : Prepared bn → List (Finset bn.V × Factor bn R)
| .leaf B f _ ⇒ [(B, multiply f incoming)]
| .branch B f _ l r ⇒
  (B, multiply f (multiply incoming (multiply l.up r.up))) ::
    (distribute (project l.bag (multiply f (multiply incoming r.up))) l ++
     distribute (project r.bag (multiply f (multiply incoming l.up))) r)

theorem frame_scope (B V : Finset bn.V) (f o g : Factor bn R)
  (hf : f.scope ⊆ B) (ho : o.scope ∩ V ⊆ B) (hg : g.scope ∩ V ⊆ B) :
  (multiply o (multiply f g)).scope ∩ V ⊆ B := by
  intro v hv

```

```

have hvV := (Finset.mem_inter.1 hv).2
rcases Finset.mem_union.1 (Finset.mem_inter.1 hv).1 with hv | hv
• exact ho (Finset.mem_inter.2 ⟨hv, hvV⟩)
• rcases Finset.mem_union.1 hv with hv | hv
  • exact hf hv
  • exact hg (Finset.mem_inter.2 ⟨hv, hvV⟩)

theorem outgoing (B C : Finset bn.V) (f o g : Factor bn R)
  (hf : f.scope ⊆ B) (hog : o.scope ∩ g.scope ⊆ B)
  (hC : (multiply o (multiply f g)).scope ∩ C ⊆ B) :
  project C (multiply f (multiply (project B o) (project B g))) =
  project C (multiply o (multiply f g)) := by
calc
  _ = project C (project B (multiply o (multiply f g))) := by
    rw [multiply_left_comm o f g, project_three B f o g hf hog]
  _ = _ := project_project C B _ hC

/-- The collect/distribute beliefs are the global factor product projected to each bag.
The outside factor is an induction frame; the public calibration theorem supplies unit. -/
theorem distribute_correct (F : I → Factor bn R) (t : Tree bn.V I) (h : Good F t)
  (outside : Factor bn R) (hb : outside.scope ∩ t.vars ⊆ t.bag)
  (P : Finset bn.V) :
  ∀ b ∈ distribute (project t.bag outside) (prepare F P t),
    b.2 = project b.1 (multiply outside (full F t)) := by
induction t generalizing outside P with
| leaf B fs ⇒
  intro b hmem
  have he : b = (B, multiply (localFactor F fs) (project B outside)) := by simpa [distribute,
    ← prepare] using hmem
  subst b
  change multiply (localFactor F fs) (project B outside) = project B (multiply outside (localFactor
    F fs))
  rw [project_multiply B outside _ (Finset.inter_subset_right.trans h), project_of_scope _ h,
    multiply_comm]
| branch B fs l r ihl ihr ⇒
  let f := localFactor F fs
  let fl := full F l
  let fr := full F r
  have hl : fl.scope ⊆ l.vars := full_scope F l h.2.1
  have hr : fr.scope ⊆ r.vars := full_scope F r h.2.2.1
  have hvl : l.vars ⊆ (Tree.branch B fs l r).vars :=
    (Finset.subset_union_left : l.vars ⊆ l.vars ∪ r.vars).trans Finset.subset_union_right
  have hvr : r.vars ⊆ (Tree.branch B fs l r).vars :=
    (Finset.subset_union_right : r.vars ⊆ l.vars ∪ r.vars).trans Finset.subset_union_right
  have hoL : outside.scope ∩ l.vars ⊆ B :=
    (Finset.inter_subset_inter (Finset.Subset.refl _) hvl).trans hb
  have hoR : outside.scope ∩ r.vars ⊆ B :=
    (Finset.inter_subset_inter (Finset.Subset.refl _) hvr).trans hb
  have hlr : fl.scope ∩ fr.scope ⊆ B :=
    (Finset.inter_subset_inter hl hr).trans h.2.2.2.2
  have hrl : fr.scope ∩ l.vars ⊆ B := by
    intro v hv
    exact h.2.2.2.2 (Finset.mem_inter.2 ⟨(Finset.mem_inter.1 hv).2, hr (Finset.mem_inter.1
      hv).1⟩)
  have hLR : fl.scope ∩ r.vars ⊆ B :=
    (Finset.inter_subset_inter hl (Finset.Subset.refl _)).trans h.2.2.2.2
  let ol := multiply outside (multiply f fr)
  let or := multiply outside (multiply f fl)
  have hoLB : ol.scope ∩ l.vars ⊆ B := frame_scope B l.vars f outside fr h.1 hoL hrl
  have horB : or.scope ∩ r.vars ⊆ B := frame_scope B r.vars f outside fl h.1 hoR hLR
  have hoL : ol.scope ∩ l.vars ⊆ l.bag := by
    intro v hv
    exact h.2.2.2.1 (Finset.mem_inter.2 ⟨(Finset.mem_inter.1 hv).2, hoLB hv⟩)
  have hor : or.scope ∩ r.vars ⊆ r.bag := by
    intro v hv
    exact h.2.2.2.1 (Finset.mem_inter.2 ⟨(Finset.mem_inter.1 hv).2, horB hv⟩)

```

```

have houtl : project l.bag (multiply f (multiply (project B outside) (project B fr))) = project
  ↪ l.bag ol :=
    outgoing B l.bag f outside fr h.1
    ((Finset.inter_subset_inter (Finset.Subset.refl _) hr).trans hoR)
    ((Finset.inter_subset_inter (Finset.Subset.refl _) (bag_subset_vars l)).trans holB)
have houtr : project r.bag (multiply f (multiply (project B outside) (project B fl))) = project
  ↪ r.bag or :=
    outgoing B r.bag f outside fl h.1
    ((Finset.inter_subset_inter (Finset.Subset.refl _) hl).trans hol)
    ((Finset.inter_subset_inter (Finset.Subset.refl _) (bag_subset_vars r)).trans horB)
intro b hm
simp only [prepare, distribute, prepare_bag] at hm
rw [prepare_up F l h.2.1 B h.2.2.2.1, prepare_up F r h.2.2.1 B h.2.2.2.1] at hm
change b ∈ (B, multiply f (multiply (project B outside) (multiply (project B fl) (project B
  ↪ fr)))) ::
  (distribute (project l.bag (multiply f (multiply (project B outside) (project B fr)))) (prepare
  ↪ F B l) ++
    distribute (project r.bag (multiply f (multiply (project B outside) (project B fl)))) (prepare
  ↪ F B r)) at hm
rw [houtl, houtr] at hm
rcases List.mem_cons.1 hm with rfl | hm
• change multiply f (multiply (project B outside) (multiply (project B fl) (project B fr))) =
  project B (multiply outside (multiply f (multiply fl fr)))
  have hf := full_scope F (.branch B fs l r) h
  change (multiply f (multiply fl fr)).scope ⊆ (Tree.branch B fs l r).vars at hf
  have hcross : outside.scope ∩ (multiply f (multiply fl fr)).scope ⊆ B :=
    (Finset.inter_subset_inter (Finset.Subset.refl _) hf).trans hb
  rw [project_multiply B outside (multiply f (multiply fl fr)) hcross,
    project_three B f fl fr h.1 hlr]
  ac_rfl
• rcases List.mem_append.1 hm with hm | hm
  • rw [ihl h.2.1 ol hol B b hm]
    congr 1
    change multiply (multiply outside (multiply f fr)) fl = multiply outside (multiply f
  ↪ (multiply fl fr))
    ac_rfl
    • rw [ihr h.2.2.1 or hor B b hm]
      congr 1
      change multiply (multiply outside (multiply f fl)) fr = multiply outside (multiply f
  ↪ (multiply fl fr))
      ac_rfl

def calibrate (F : I → Factor bn R) (t : Tree bn.V I) :=
  distribute unit (prepare F ∅ t)

theorem calibrate_correct (F : I → Factor bn R) (t : Tree bn.V I) (h : Good F t) :
  ∀ b ∈ calibrate F t, b.2 = project b.1 (full F t) := by
  have hu : project t.bag (unit (bn := bn)) = unit := project_of_scope _ (Finset.empty_subset _)
  have hh := distribute_correct F t h unit (by intro v hv; exact False.elim (Finset.notMem_empty v
  ↪ (Finset.mem_inter.1 hv).1)) ∅
  simp only [hu, unit_multiply] using hh

/-- Arbitrary branching is encoded by a chain of copies of the same bag with empty local
factor lists. This is a representation transformation, not a restriction to degree three. -/
def graft (B : Finset bn.V) (fs : List I) : List (Tree bn.V I) → Tree bn.V I
| [] => .leaf B fs
| t :: ts => .branch B fs t (graft B [] ts)

def forestVars : List (Tree bn.V I) → Finset bn.V
| [] => ∅
| t :: ts => t.vars ∪ forestVars ts

theorem graft_bag (B : Finset bn.V) (fs : List I) (ts : List (Tree bn.V I)) :
  (graft B fs ts).bag = B := by cases ts <|> rfl

theorem graft_vars (B : Finset bn.V) (fs : List I) (ts : List (Tree bn.V I)) :

```

```

    (graft B fs ts).vars = B U forestVars ts := by
induction ts generalizing fs with
| nil => simp [graft, Tree.vars, forestVars]
| cons t ts ih =>
    simp only [graft, Tree.vars, forestVars, ih]
    ac_rfl

theorem graft_indices (B : Finset bn.V) (fs : List I) (ts : List (Tree bn.V I)) :
    (graft B fs ts).indices = fs ++ ts.flatMap Tree.indices := by
induction ts generalizing fs with
| nil => simp [graft, Tree.indices]
| cons t ts ih => simp [graft, Tree.indices, ih, List.append_assoc]

theorem mem_forestVars (ts : List (Tree bn.V I)) (v : bn.V) :
    v ∈ forestVars ts ↔ ∃ t ∈ ts, v ∈ t.vars := by
induction ts with
| nil => simp [forestVars]
| cons t ts ih => simp [forestVars, ih, List.mem_cons, or_and_right, exists_or]

/-- Ordinary multi-child running intersection and factor-cover conditions generate a valid
binary working representation. No probability or message invariant occurs in the premises. -/
theorem graft_good (F : I → Factor bn R) (B : Finset bn.V) (fs : List I)
    (ts : List (Tree bn.V I)) (hf : (localFactor F fs).scope ⊆ B)
    (hg : ∀ t ∈ ts, Good F t) (hb : ∀ t ∈ ts, t.vars ∩ B ⊆ t.bag)
    (hpair : ts.Pairwise (fun t s => t.vars ∩ s.vars ⊆ B)) :
    Good F (graft B fs ts) := by
induction ts generalizing fs with
| nil => exact hf
| cons t ts ih =>
    have ht := (List.pairwise_cons.1 hpair)
    refine ⟨hf, hg t (List.mem_cons_self ..),
        ih [] (Finset.empty_subset _) (fun s hs => hg s (List.mem_cons_of_mem t hs))
        (fun s hs => hb s (List.mem_cons_of_mem t hs)) ht.2,
        hb t (List.mem_cons_self ..), ?_, ?_⟩
    • rw [graft_bag]
      exact Finset.inter_subset_right
    • intro v hv
      have hvT := (Finset.mem_inter.1 hv).1
      rw [graft_vars] at hv
      rcases Finset.mem_union.1 (Finset.mem_inter.1 hv).2 with hB | hrest
      • exact hB
      • obtain ⟨s, hs, hvS⟩ := (mem_forestVars ts v).1 hrest
        exact ht.1 s hs (Finset.mem_inter.2 ⟨hvT, hvS⟩)

/-- Disconnected components are joined only by empty-separator scalar messages. -/
theorem forest_good (F : I → Factor bn R) (ts : List (Tree bn.V I))
    (hg : ∀ t ∈ ts, Good F t)
    (hd : ts.Pairwise (fun t s => Disjoint t.vars s.vars)) :
    Good F (graft ∅ [] ts) := by
apply graft_good F ∅ [] ts (Finset.empty_subset _) hg
• intro t _
  simp
• exact hd.imp (by
    intro t s h
    rw [Finset.disjoint_iff_inter_eq_empty] at h
    simp [h])

theorem mem_local_scope (F : I → Factor bn R) (fs : List I) (v : bn.V) :
    v ∈ (localFactor F fs).scope ↔ ∃ i ∈ fs, v ∈ (F i).scope := by
induction fs with
| nil => simp [localFactor, combine, unit]
| cons i fs ih =>
    simp only [localFactor, List.map_cons, combine, multiply, Finset.mem_union] at *
    simp [ih, List.mem_cons, or_and_right, exists_or]

theorem mem_full_scope (F : I → Factor bn R) (t : Tree bn.V I) (v : bn.V) :

```

```

    v ∈ (full F t).scope ↔ ∃ i ∈ t.indices, v ∈ (F i).scope := by
induction t with
| leaf B fs ⇒ exact mem_local_scope F fs v
| branch B fs l r ihl ihr ⇒
    simp only [full, multiply, Finset.mem_union, mem_local_scope, Tree.indices,
        List.mem_append, or_and_right, exists_or, ihl, ihr, or_assoc]

theorem full_scope_univ (F : I → Factor bn R) (t : Tree bn.V I)
    (hcover : ∀ v, ∃ i ∈ t.indices, v ∈ (F i).scope) : (full F t).scope = Finset.univ := by
    apply Finset.eq_univ_of_forall
    intro v
    exact (mem_full_scope F t v).2 (hcover v)

theorem full_product_of_perm [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
    (hassign : t.indices.Perm Finset.univ.toList) :
    (full F t).value = Factor.product (Finset.univ.toList.map F) := by
    rw [full_value]
    funext x
    simp only [Factor.product, List.map_map, Function.comp_def] using
    (hassign.map (fun i ⇒ (F i).value x)).prod_eq

/-- Calibrated clique beliefs agree with the independently defined variable-elimination
algorithm for any duplicate-free elimination order of the other variables. -/
theorem calibrate_eq_ve [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
    (ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
    (hvars : ∀ v, ∃ i, v ∈ (F i).scope) (b : Finset bn.V × Factor bn R)
    (hb : b ∈ calibrate F t) (vs : List bn.V) (hvs : vs.Nodup) (hset : vs.toFinset = b.1c)
    (x : bn.Assignment) :
    b.2.value x = Factor.product (Factor.eliminateAll (Finset.univ.toList.map F) vs) x := by
    have hs : (full F t).scope = Finset.univ := full_scope_univ F t (by
        intro v
        obtain ⟨i, hi⟩ := hvars v
        exact ⟨i, hassign.mem_iff.mpr (Finset.mem_toList.2 (Finset.mem_univ _)), hi⟩)
    rw [calibrate_correct F t ht b hb, project_value, hs, ← Finset.compl_eq_univ_sdiff,
        full_product_of_perm F t hassign, Factor.eliminateAll_correct _ vs hvs, hset]

def beliefWeight (b : Finset bn.V × Factor bn R) (base : bn.Assignment)
    (q : PartialAssignment bn b.1) : ℝ :=
    b.2.value (patch b.1 base q)

/-- The sum of a calibrated clique belief is the GLOBAL mass, including all disconnected
components joined through the empty virtual root. Zero entries require no division. -/
theorem belief_mass [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
    (ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
    (hvars : ∀ v, ∃ i, v ∈ (F i).scope) (b : Finset bn.V × Factor bn R)
    (hb : b ∈ calibrate F t) (base : bn.Assignment) :
    FiniteDistribution.mass (beliefWeight b base) =
    FiniteDistribution.mass (Factor.product (Finset.univ.toList.map F)) := by
    have hs : (full F t).scope = Finset.univ := full_scope_univ F t (by
        intro v
        obtain ⟨i, hi⟩ := hvars v
        exact ⟨i, hassign.mem_iff.mpr (Finset.mem_toList.2 (Finset.mem_univ _)), hi⟩)
    have hw : beliefWeight b base =
        FiniteDistribution.pushWeight (restrictTo b.1) (Factor.product (Finset.univ.toList.map F)) :=
    by
        funext q
        unfold beliefWeight
        rw [calibrate_correct F t ht b hb, project_value, hs, ← Finset.compl_eq_univ_sdiff,
            full_product_of_perm F t hassign, marginal_eq_query_weight, restrict_patch]
        rw [hw, FiniteDistribution.mass_pushWeight]

def queryWeight (Q : Finset bn.V) (b : Finset bn.V × Factor bn R) (base : bn.Assignment)
    (q : PartialAssignment bn Q) : ℝ :=
    (project Q b.2).value (patch Q base q)

theorem queryWeight_correct [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)

```

```

(ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
(hvars : ∀ v, ∃ i, v ∈ (F i).scope) (b : Finset bn.V × Factor bn R)
(hb : b ∈ calibrate F t) (Q : Finset bn.V) (hQ : Q ⊆ b.1) (base : bn.Assignment) :
queryWeight Q b base =
  FiniteDistribution.pushWeight (restrictTo Q) (Factor.product (Finset.univ.toList.map F)) := by
have hs : (full F t).scope = Finset.univ := full_scope_univ F t (by
  intro v
  obtain ⟨i, hi⟩ := hvars v
  exact ⟨i, hassign.mem_iff.mpr (Finset.mem_toList.2 (Finset.mem_univ _)), hi⟩)
funext q
unfold queryWeight
rw [calibrate_correct F t ht b hb,
  project_project Q b.1 _ (Finset.inter_subset_right.trans hQ),
  project_value, hs, ← Finset.compl_eq_univ_sdiff, full_product_of_perm F t hassign,
  marginal_eq_query_weight, restrict_patch]

theorem factor_product_nonnegative [Fintype I] (F : I → Factor bn R)
(hF : ∀ i x, 0 ≤ (F i).value x) : ∀ x, 0 ≤ Factor.product (Finset.univ.toList.map F) x := by
intro x
unfold Factor.product
apply List.prod_nonneg
intro a ha
obtain ⟨f, hf, rfl⟩ := List.mem_map.1 ha
obtain ⟨i, _, rfl⟩ := List.mem_map.1 hf
exact hF i x

def queryPosterior [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
(ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
(hvars : ∀ v, ∃ i, v ∈ (F i).scope) (hF : ∀ i x, 0 ≤ (F i).value x)
(b : Finset bn.V × Factor bn R) (hb : b ∈ calibrate F t)
(Q : Finset bn.V) (hQ : Q ⊆ b.1) (base : bn.Assignment) :
Option (FiniteDistribution.Distribution (PartialAssignment bn Q)) :=
FiniteDistribution.normalize (queryWeight Q b base) (by
  rw [queryWeight_correct F t ht hassign hvars b hb Q hQ base]
  exact FiniteDistribution.pushWeight_nonneg _ _ (factor_product_nonnegative F hF))

/-- Exact global feasibility, including a zero-mass component disconnected from the queried
clique. The normalized result cannot silently ignore that component. -/
theorem queryPosterior_none_iff [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
(ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
(hvars : ∀ v, ∃ i, v ∈ (F i).scope) (hF : ∀ i x, 0 ≤ (F i).value x)
(b : Finset bn.V × Factor bn R) (hb : b ∈ calibrate F t)
(Q : Finset bn.V) (hQ : Q ⊆ b.1) (base : bn.Assignment) :
queryPosterior F t ht hassign hvars hF b hb Q hQ base = none ↔
  ¬ ∃ x, 0 < Factor.product (Finset.univ.toList.map F) x := by
rw [queryPosterior, FiniteDistribution.normalize_none_iff,
  queryWeight_correct F t ht hassign hvars b hb Q hQ base, FiniteDistribution.mass_pushWeight]
have hn := FiniteDistribution.mass_nonneg _ (factor_product_nonnegative F hF)
have hz : FiniteDistribution.mass (Factor.product (Finset.univ.toList.map F)) = 0 ↔
  ¬ 0 < FiniteDistribution.mass (Factor.product (Finset.univ.toList.map F)) :=
  {fun he => by rw [he]; simp, fun he => le_antisymm (le_of_not_gt he) hn}
rw [hz, FiniteDistribution.mass_pos_iff _ (factor_product_nonnegative F hF)]

/-- A returned normalized clique/query posterior is exactly the exhaustive joint posterior. -/
theorem queryPosterior_value [Fintype I] (F : I → Factor bn R) (t : Tree bn.V I)
(ht : Good F t) (hassign : t.indices.Perm Finset.univ.toList)
(hvars : ∀ v, ∃ i, v ∈ (F i).scope) (hF : ∀ i x, 0 ≤ (F i).value x)
(b : Finset bn.V × Factor bn R) (hb : b ∈ calibrate F t)
(Q : Finset bn.V) (hQ : Q ⊆ b.1) (base : bn.Assignment)
(d : FiniteDistribution.Distribution (PartialAssignment bn Q))
(hd : queryPosterior F t ht hassign hvars hF b hb Q hQ base = some d)
(q : PartialAssignment bn Q) :
d.pmf q =
  FiniteDistribution.pushWeight (restrictTo Q) (Factor.product (Finset.univ.toList.map F)) q /
  FiniteDistribution.mass (Factor.product (Finset.univ.toList.map F)) := by
have he := FiniteDistribution.normalize_value _ _ d hd q

```

```

rw [queryWeight_correct F t ht hassign hvars b hb Q hQ base, FiniteDistribution.mass_pushWeight]
  <| at he
exact he

end

end BayesianNetworksProofs.Junction

```

Conditional independence from separator factorization

```

import BayesianNetworksProofs.Finite.FactorMarginal
import BayesianNetworksProofs.Finite.Posterior
import Mathlib.Tactic.Ring
import Mathlib.Tactic.FieldSimp

```

Conditional independence is stated as the finite event cross-product identity. At positive conditioning mass it is the usual product of conditional probabilities. Zero conditioning mass is not divided by. This module proves the probability algebra; graphical separation is defined independently in `DSeparation.lean`.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet

open Factor
noncomputable section
variable {bn : FinBayesNet}

def eventMass (p : bn.Assignment → ℝ) (A B C : Finset bn.V)
  (a b c : bn.Assignment) : ℝ :=
  ∑ x, p x * indicator A a x * indicator B b x * indicator C c x

def ConditionalIndependent (p : bn.Assignment → ℝ) (A B C : Finset bn.V) : Prop :=
  ∀ a b c, eventMass p A B C a b c * eventMass p ∅ ∅ C a b c =
    eventMass p A ∅ C a b c * eventMass p ∅ B C a b c

def relativeEventMass (U : Finset bn.V) (f : bn.Assignment → ℝ) (A B C : Finset bn.V)
  (a b c : bn.Assignment) : ℝ :=
  marg U (fun x ⇒ f x * indicator A a x * indicator B b x * indicator C c x) c

theorem indicator_empty (a x : bn.Assignment) : indicator ∅ a x = 1 := by
  simp [indicator, agrees]

theorem separator_event_mass (L R C A B : Finset bn.V) (f g : Factor bn ℝ)
  (hLR : Disjoint L R) (hLC : Disjoint L C) (hRC : Disjoint R C)
  (hf : f.scope ⊆ L ∪ U C) (hg : g.scope ⊆ R ∪ U C)
  (hA : A ⊆ L) (hB : B ⊆ R) (a b c : bn.Assignment) :
  relativeEventMass ((L ∪ R) ∪ C) (multiply f g).value A B C a b c =
    marg L (fun x ⇒ f.value x * indicator A a x) c *
    marg R (fun x ⇒ g.value x * indicator B b x) c := by
  let fl := multiply f (evidenceFactor A a)
  let fr := multiply g (evidenceFactor B b)
  have hfl : fl.scope ⊆ L ∪ C :=
    Finset.union_subset hf (hA.trans Finset.subset_union_left)
  have hfr : fr.scope ⊆ R ∪ C :=
    Finset.union_subset hg (hB.trans Finset.subset_union_left)
  have hd : Disjoint C (L ∪ R) := (Finset.disjoint_union_right.2 ⟨hLC.symm, hRC.symm⟩)
  have he : (fun x ⇒ (multiply f g).value x * indicator A a x *
    indicator B b x * indicator C c x) =
    (fun x ⇒ (multiply fl fr).value x * indicator C c x) := by
    funext x
    dsimp [multiply, evidenceFactor, fl, fr]
    ring
  unfold relativeEventMass
  rw [he, marg_indicator_eq_clamp hd, marg_clamp hd]

```

```

have hc : clamp C c c = c := by funext v; simp [clamp]
rw [hc]
apply marg_product_disjoint L R fl fr hLR
· exact (Finset.disjoint_union_right.2 ⟨hLR.symm, hRC⟩).mono_right hfl
· exact (Finset.disjoint_union_right.2 ⟨hLR, hLC⟩).mono_right hfr

theorem separator_cross_product (L R C A B : Finset bn.V) (f g : Factor bn.R)
(hLR : Disjoint L R) (hLC : Disjoint L C) (hRC : Disjoint R C)
(hf : f.scope ⊆ L ∪ U C) (hg : g.scope ⊆ R ∪ U C)
(hA : A ⊆ L) (hB : B ⊆ R) (a b c : bn.Assignment) :
relativeEventMass ((L ∪ R) ∪ C) (multiply f g).value A B C a b c *
relativeEventMass ((L ∪ R) ∪ C) (multiply f g).value ∅ ∅ C a b c =
relativeEventMass ((L ∪ R) ∪ C) (multiply f g).value A ∅ C a b c *
relativeEventMass ((L ∪ R) ∪ C) (multiply f g).value ∅ B C a b c := by
rw [separator_event_mass L R C A B f g hLR hLC hRC hf hg hA hB,
separator_event_mass L R C ∅ ∅ f g hLR hLC hRC hf hg (Finset.empty_subset _) (Finset.empty_subset
_),
separator_event_mass L R C A ∅ f g hLR hLC hRC hf hg hA (Finset.empty_subset _),
separator_event_mass L R C ∅ B f g hLR hLC hRC hf hg (Finset.empty_subset _) hB]
ring

/-- Marginalizing away variables outside an ancestral factorization commutes with an event
that depends only on retained variables. No numerical separation condition is assumed. -/
theorem eventMass_of_marginal (p q : bn.Assignment → ℝ) (U A B C : Finset bn.V)
(hm : ∀ x, marg Uc p x = q x) (hA : A ⊆ U) (hB : B ⊆ U) (hC : C ⊆ U)
(a b c : bn.Assignment) :
eventMass p A B C a b c = relativeEventMass U q A B C a b c := by
let mask := multiply (evidenceFactor A a) (multiply (evidenceFactor B b) (evidenceFactor C c))
have hs : mask.scope ⊆ U := Finset.union_subset hA (Finset.union_subset hB hC)
have hinner : ∀ x, marg Uc (fun y ⇒ p y * mask.value y) x = q x * mask.value x := by
intro x
have he : (fun y ⇒ p y * mask.value y) = (fun y ⇒ mask.value y * p y) := by
funext y
ring
rw [he, marg_mul_left (fun y hy ⇒ mask.dependsOn y x fun v hv ⇒
mem_fibre.1 hy v (fun hc ⇒ Finset.mem_compl.1 hc (hs hv))), hm, mul_comm]
have he : (fun x ⇒ p x * indicator A a x * indicator B b x * indicator C c x) =
(fun x ⇒ p x * mask.value x) := by
funext x
dsimp [mask, multiply, evidenceFactor]
ring
unfold eventMass relativeEventMass
rw [he, ← marg_univ _ c]
have hU : U ∪ Uc = Finset.univ := Finset.union_compl U
rw [← hU, marg_union_disjoint (Finset.disjoint_left.2 fun _ hu hc ⇒ Finset.mem_compl.1 hc hu)]
simp_rw [hinner]
apply congrArg (fun f ⇒ marg U f c)
funext x
dsimp [mask, multiply, evidenceFactor]
ring

/-- Positive conditioning mass turns the cross-product identity into the usual conditional
probability factorization. The positivity boundary is explicit. -/
theorem ConditionalIndependent.conditional (p : bn.Assignment → ℝ) (A B C : Finset bn.V)
(h : ConditionalIndependent p A B C) (a b c : bn.Assignment)
(hz : 0 < eventMass p ∅ ∅ C a b c) :
eventMass p A B C a b c / eventMass p ∅ ∅ C a b c =
(eventMass p A ∅ C a b c / eventMass p ∅ ∅ C a b c) *
(eventMass p ∅ B C a b c / eventMass p ∅ ∅ C a b c) := by
rw [div_mul_div_comm, ← h a b c]
field_simp [ne_of_gt hz]

end
end BayesianNetworksProofs.FinBayesNet

```

D-separation soundness on a finite DAG

```
import BayesianNetworksProofs.Finite.ConditionalIndependence
import Mathlib.Combinatorics.SimpleGraph.Connectivity.Connected
import Mathlib.Logic.Relation
```

Separation is defined solely from the directed parent graph: take the ancestors of the query and conditioning variables, moralize every retained child/parent family, delete conditioning vertices, and require no graph path between the queries. It is not defined numerically.

Reachability generates a component partition. Every mechanism scope is a clique in the moral graph, so no retained factor crosses the partition except through conditioning variables. The ancestor-normalization theorem and the separator probability algebra prove conditional independence. This is the moralized-ancestral criterion, not a verification of Julia Bayes-ball.

```
set_option autoImplicit false

namespace BayesianNetworksProofs.FinBayesNet

open Factor
noncomputable section

variable {bn : FinBayesNet}

def ParentEdge (bn : FinBayesNet) (v w : bn.V) : Prop :=
  ∃ m, bn.target m = w ∧ v ∈ bn.parents m

def ancestors (S : Finset bn.V) : Finset bn.V := by
  classical
  exact Finset.univ.filter fun v => ∃ w ∈ S, Relation.ReflTransGen (ParentEdge bn) v w

theorem subset_ancestors (S : Finset bn.V) : S ⊆ ancestors S := by
  classical
  intro v hv
  exact Finset.mem_filter.2 ⟨Finset.mem_univ _, v, hv, Relation.ReflTransGen.refl⟩

theorem ancestors_upstream (S : Finset bn.V) : UpstreamClosed (ancestors S) := by
  classical
  intro m hm v hv
  obtain ⟨w, hw, path⟩ := (Finset.mem_filter.1 hm).2
  exact Finset.mem_filter.2 ⟨Finset.mem_univ _, w, hw,
    (Relation.ReflTransGen.single ⟨m, rfl, hv⟩).trans path⟩

def family (bn : FinBayesNet) (m : bn.M) : Finset bn.V :=
  insert (bn.target m) (bn.parents m)

def moral (U : Finset bn.V) : SimpleGraph bn.V where
  Adj v w := v ≠ w ∧ ∃ m, bn.target m ∈ U ∧ v ∈ family bn m ∧ w ∈ family bn m
  symm := by
    rintro v w ⟨hne, m, hm, hv, hw⟩
    exact ⟨Ne.symm hne, m, hm, hw, hv⟩
  loopless := ⟨fun v h => h.1 rfl⟩

def cutMoral (U C : Finset bn.V) : SimpleGraph bn.V where
  Adj v w := (moral U).Adj v w ∧ v ∉ C ∧ w ∉ C
  symm := by
    rintro v w ⟨h, hv, hw⟩
    exact ⟨h.symm, hw, hv⟩
  loopless := ⟨fun v h => h.1.1 rfl⟩

def GraphSeparated (U A B C : Finset bn.V) : Prop :=
  ∀ a ∈ A, ∀ b ∈ B, ¬(cutMoral U C).Reachable a b

def DSeparated (A B C : Finset bn.V) : Prop :=
  GraphSeparated (ancestors ((A ∪ B) ∪ C)) A B C
```

```

def leftSide (U A C : Finset bn.V) : Finset bn.V := by
  classical
  exact (U \ C).filter fun v => ∃ a ∈ A, (cutMoral U C).Reachable a v

def rightSide (U A C : Finset bn.V) : Finset bn.V := U \ (leftSide U A C U C)

theorem left_mem {U A C : Finset bn.V} {v : bn.V} :
  v ∈ leftSide U A C ↔ v ∈ U ∧ v ∉ C ∧ ∃ a ∈ A, (cutMoral U C).Reachable a v := by
  simp [leftSide, and_assoc]

theorem left_subset (U A C : Finset bn.V) : leftSide U A C ⊆ U :=
  fun _ hv => (left_mem.1 hv).1

theorem left_avoids (U A C : Finset bn.V) : Disjoint (leftSide U A C) C :=
  Finset.disjoint_left.2 fun _ hv hc => (left_mem.1 hv).2.1 hc

theorem family_subset {U : Finset bn.V} {hU : UpstreamClosed U} (m : bn.M) (hm : bn.target m ∈ U) :
  family bn m ⊆ U := Finset.insert_subset_iff.2 ⟨hm, hU m hm⟩

theorem left_closed_edge {U A C : Finset bn.V} {v w : bn.V}
  (hv : v ∈ leftSide U A C) (hw : w ∈ U) (he : (cutMoral U C).Adj v w) :
  w ∈ leftSide U A C := by
  obtain ⟨_, _, a, ha, path⟩ := left_mem.1 hv
  exact left_mem.2 ⟨hw, he.2.2, a, ha, path.trans he.reachable⟩

theorem queries_in_sides {U A B C : Finset bn.V}
  (hA : A ⊆ U) (hB : B ⊆ U) (hAC : Disjoint A C) (hBC : Disjoint B C)
  (hsep : GraphSeparated U A B C) :
  A ⊆ leftSide U A C ∧ B ⊆ rightSide U A C := by
  constructor
  · intro v hv
    exact left_mem.2 ⟨hA hv, fun hc => Finset.disjoint_left.1 hAC hv hc,
      v, hv, SimpleGraph.Reachable.refl v⟩
  · intro v hv
    refine Finset.mem_sdiff.2 ⟨hB hv, ?_⟩
    intro hc
    rcases Finset.mem_union.1 hc with hl | hc
    · obtain ⟨_, _, a, ha, path⟩ := left_mem.1 hl
      exact hsep a ha v hv path
    · exact Finset.disjoint_left.1 hBC hv hc

/-- A moral clique touching the reachable side cannot also touch the other side. -/
theorem family_left {U A C : Finset bn.V} (hU : UpstreamClosed U)
  (m : bn.M) (hm : bn.target m ∈ U) (ht : (family bn m ∩ leftSide U A C).Nonempty) :
  family bn m ⊆ leftSide U A C U C := by
  obtain ⟨v, hv⟩ := ht
  have hvF := (Finset.mem_inter.1 hv).1
  have hvL := (Finset.mem_inter.1 hv).2
  intro w hw
  by_cases hc : w ∈ C
  · exact Finset.mem_union_right _ hc
  · apply Finset.mem_union_left
    by_cases he : v = w
    · simpa [he] using hvL
    · exact left_closed_edge hvL (family_subset hU m hm hw)
      ⟨⟨he, m, hm, hvF, hw⟩, (left_mem.1 hvL).2.1, hc⟩

theorem family_right {U A C : Finset bn.V} (hU : UpstreamClosed U)
  (m : bn.M) (hm : bn.target m ∈ U) (ht : ¬(family bn m ∩ leftSide U A C).Nonempty) :
  family bn m ⊆ rightSide U A C U C := by
  intro v hv
  by_cases hc : v ∈ C
  · exact Finset.mem_union_right _ hc
  · apply Finset.mem_union_left
    refine Finset.mem_sdiff.2 ⟨family_subset hU m hm hv, ?_⟩
    intro hbad

```

```

    rcases Finset.mem_union.1 hbad with hl | hc'
    • exact ht ⟨v, Finset.mem_inter.2 ⟨hv, hl⟩⟩
    • exact hc hc'

def groupedFactor (κ : bn.Kernel R) (hloc : ∀ m, Local κ m) (S : Finset bn.V)
  (ms : Finset bn.M) (hS : ∀ m ∈ ms, family bn m ⊆ S) : Factor bn R where
scope := S
value x := ∏ m ∈ ms, κ m x (x (bn.target m))
dependsOn x y h := by
  apply Finset.prod_congr rfl
  intro m hm
  rw [hloc m x y (fun v hv => h v (hS m hm (Finset.mem_insert_of_mem hv))),
    h (bn.target m) (hS m hm (Finset.mem_insert_self _ _))]

/-- **D-separation soundness.** The premise is graph path separation in the moralized ancestral
graph. The conclusion is the finite conditional-independence event identity. No numerical
independence or precomputed correct factor partition is assumed. -/
theorem d_separation_sound (κ : bn.Kernel R) (hclosed : bn.Closed) (ord : bn.TopoOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (A B C : Finset bn.V) (hAC : Disjoint A C) (hBC : Disjoint B C)
  (hsep : DSeparated A B C) : ConditionalIndependent (joint κ) A B C := by
classical
let U := ancestors ((A ∪ B) ∪ C)
have hU : UpstreamClosed U := ancestors_upstream _
have hAU : A ⊆ U := (Finset.subset_union_left.trans Finset.subset_union_left).trans
  <| (subset_ancestors _)
have hBU : B ⊆ U := (Finset.subset_union_right.trans Finset.subset_union_left).trans
  <| (subset_ancestors _)
have hCU : C ⊆ U := Finset.subset_union_right.trans (subset_ancestors _)
let L := leftSide U A C
let R := rightSide U A C
have hSides := queries_in_sides hAU hBU hAC hBC hsep
have hLC : Disjoint L C := left_avoids U A C
have hLR : Disjoint L R := Finset.disjoint_left.2 fun v hv hr =>
  (Finset.mem_sdiff.1 hr).2 (Finset.mem_union_left _ hv)
have hRC : Disjoint R C := Finset.disjoint_left.2 fun v hr hv =>
  (Finset.mem_sdiff.1 hr).2 (Finset.mem_union_right _ hv)
have hsplit : (L ∪ R) ∪ C = U := by
  have hLU : L ⊆ U := left_subset U A C
  apply Finset.Subset.antisymm
  • intro v hv
    rcases Finset.mem_union.1 hv with hv | hc
    • rcases Finset.mem_union.1 hv with hl | hr
      • exact hLU hl
      • exact (Finset.mem_sdiff.1 hr).1
    • exact hCU hc
  • intro v hv
    by_cases hc : v ∈ C
    • exact Finset.mem_union_right _ hc
    • apply Finset.mem_union_left
      by_cases hl : v ∈ L
      • exact Finset.mem_union_left _ hl
      • exact Finset.mem_union_right _ (Finset.mem_sdiff.2 ⟨hv, by simp using And.intro hl hc⟩)
let ms := Finset.univ.filter (fun m => bn.target m ∈ U)
let touches := fun m => (family bn m ∩ L).Nonempty
let ml := ms.filter touches
let mr := ms.filter (fun m => ¬touches m)
have hls : ∀ m ∈ ml, family bn m ⊆ L ∪ C := by
  intro m hm
  exact family_left hU m (Finset.mem_filter.1 (Finset.mem_filter.1 hm).1).2
    (Finset.mem_filter.1 hm).2
have hrs : ∀ m ∈ mr, family bn m ⊆ R ∪ C := by
  intro m hm
  exact family_right hU m (Finset.mem_filter.1 (Finset.mem_filter.1 hm).1).2
    (Finset.mem_filter.1 hm).2
let f := groupedFactor κ hloc (L ∪ C) ml hls

```

```

let g := groupedFactor κ hloc (R U C) mr hrs
have hfac : (multiply f g).value = fun x => ∏ m ∈ ms, κ m x (x (bn.target m)) := by
  funext x
  exact Finset.prod_filter_mul_prod_filter_not ms touches _
have hmarg : ∀ x, marg Uc (joint κ) x = (multiply f g).value x := by
  intro x
  rw [hfac]
  exact marg_joint_upstream κ hclosed ord hloc hnorm U hU x
intro a b c
rw [eventMass_of_marginal (joint κ) (multiply f g).value U A B C hmarg hAU hBU hCU,
  eventMass_of_marginal (joint κ) (multiply f g).value U ∅ ∅ C hmarg
    (Finset.empty_subset _) (Finset.empty_subset _) hCU,
  eventMass_of_marginal (joint κ) (multiply f g).value U A ∅ C hmarg hAU (Finset.empty_subset _)]
← hCU,
eventMass_of_marginal (joint κ) (multiply f g).value U ∅ B C hmarg (Finset.empty_subset _) hBU
← hCU,
← hsplit]
exact separator_cross_product L R C A B f g hLR hLC hRC (Finset.Subset.refl _)
(Finset.Subset.refl _) hSides.1 hSides.2 a b c

/-- Probability interpretation, with nonnegativity and normalization explicitly supplied. -/
theorem d_separation_sound_probability (κ : bn.Kernel ℝ) (hclosed : bn.Closed) (ord : bn.TopoOrder)
(hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
(A B C : Finset bn.V) (hAC : Disjoint A C) (hBC : Disjoint B C) (hsep : DSeparated A B C) :
(∀ x, 0 ≤ joint κ x) ∧ (∑ x, joint κ x = 1) ∧ ConditionalIndependent (joint κ) A B C :=
⟨joint_nonneg κ hnonneg, sum_joint_eq_one κ hclosed ord hloc hnorm,
  d_separation_sound κ hclosed ord hloc hnorm A B C hAC hBC hsep⟩

end
end BayesianNetworksProofs.FinBayesNet

```

Conditional numerical error contracts

```

import BayesianNetworksProofs.Finite.Posterior
import Mathlib.Algebra.BigOperators.Field
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Ring

```

These are quantitative hypotheses and conclusions, not a universal Float64/oracle identity. Finite product perturbations are bounded from entry errors. Normalization has an explicit positive-mass lower bound; small evidence mass amplifies error. A separate rounded-product model assumes local multiplication-error and range-preservation contracts and derives a whole-product bound. No IEEE implementation of those local contracts is assumed proved here.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.Numerical

open FiniteDistribution
noncomputable section

variable {A : Type} [Fintype A]

def l1 (p q : A → ℝ) : ℝ := ∑ a, |p a - q a|

theorem l1_nonneg (p q : A → ℝ) : 0 ≤ l1 p q := Finset.sum_nonneg fun _ => abs_nonneg _

theorem mass_error (p q : A → ℝ) : |mass p - mass q| ≤ l1 p q := by
  rw [mass, mass, ← Finset.sum_sub_distrib]
  exact Finset.abs_sum_le_sum_abs _ _

theorem point_normalization_error (p q Z W : ℝ) (hq : 0 ≤ q) (hZ : 0 < Z) (hW : 0 < W) :

```

```

|p / Z - q / W| ≤ |p - q| / Z + (q / W) * |W - Z| / Z := by
have he : p / Z - q / W = (p - q) / Z + (q / W) * (W - Z) / Z := by
  field_simp
  ring
rw [he]
calc
  ≤ |(p - q) / Z| + |(q / W) * (W - Z) / Z| := abs_add_le _ _
  = _ := by
    rw [abs_div, abs_of_pos hZ, abs_div, abs_of_pos hZ, abs_mul,
        abs_of_nonneg (div_nonneg hq hW.le)]

/-- Exact normalization is L1-stable only relative to its mass. The comparison weight is
nonnegative and both masses are positive; no entrywise lower bound is required. -/
theorem normalization_l1 (p q : A → ℝ) (hq : ∀ a, 0 ≤ q a)
  (hZ : 0 < mass p) (hW : 0 < mass q) :
  l1 (fun a => p a / mass p) (fun a => q a / mass q) ≤ 2 * l1 p q / mass p := by
have hm : |mass q - mass p| ≤ l1 p q := by simp only [abs_sub_comm] using mass_error p q
have hsum : ∑ a, q a / mass q = 1 := by
  rw [← Finset.sum_div]
  exact div_self (ne_of_gt hW)
calc
  ≤ ∑ a, (|p a - q a| / mass p + (q a / mass q) * |mass q - mass p| / mass p) :=
    Finset.sum_le_sum fun a _ => point_normalization_error _ _ _ (hq a) hZ hW
  = (l1 p q + |mass q - mass p|) / mass p := by
    rw [Finset.sum_add_distrib, ← Finset.sum_div, ← Finset.sum_div, ← Finset.sum_mul, hsum]
    simp [l1, add_div]
  ≤ (l1 p q + l1 p q) / mass p :=
    div_le_div_of_nonneg_right (add_le_add (le_refl _)) hm hZ.le
  = _ := by ring

/-- A supplied absolute input error below a genuine evidence-mass floor guarantees positive
approximate mass and bounds posterior L1 error. This is not meaningful with η=0. -/
theorem posterior_error (p q : A → ℝ) (hq : ∀ a, 0 ≤ q a)
  (η ε : ℝ) (hη : 0 < η) (hfloor : η ≤ mass p)
  (herr : l1 p q ≤ ε) (hsmall : ε < η) :
  0 < mass q ∧ l1 (fun a => p a / mass p) (fun a => q a / mass q) ≤ 2 * ε / η := by
have hZ : 0 < mass p := hη.trans_le hfloor
have hm := (le_abs_self (mass p - mass q)).trans ((mass_error p q).trans herr)
have hW : 0 < mass q := by linarith
have hε : 0 ≤ ε := (l1_nonneg p q).trans herr
refine ⟨hW, (normalization_l1 p q hq hZ hW).trans ?_⟩
calc
  ≤ 2 * ε / mass p := div_le_div_of_nonneg_right
    (mul_le_mul_of_nonneg_left herr (by norm_num)) hZ.le
  ≤ _ := div_le_div_of_nonneg_left (mul_nonneg (by norm_num) hε) hη hfloor

theorem prod_unit_interval {I : Type} (S : Finset I) (p : I → ℝ)
  (hp : ∀ i ∈ S, 0 ≤ p i ∧ p i ≤ 1) : 0 ≤ ∏ i ∈ S, p i ∧ (∏ i ∈ S, p i) ≤ 1 := by
classical
induction S using Finset.induction_on with
| empty => simp
| @insert i S hi ih =>
  have hpI := hp i (Finset.mem_insert_self _ _)
  have hpS := ih (fun j hj => hp j (Finset.mem_insert_of_mem hj))
  rw [Finset.prod_insert hi]
  exact ⟨mul_nonneg hpI.1 hpS.1,
    (mul_le_mul hpI.2 hpS.2 hpS.1 zero_le_one).trans (by simp)⟩

theorem product_error {I : Type} (S : Finset I) (p q : I → ℝ)
  (hp : ∀ i ∈ S, 0 ≤ p i ∧ p i ≤ 1) (hq : ∀ i ∈ S, 0 ≤ q i ∧ q i ≤ 1) :
  |(∏ i ∈ S, p i) - ∏ i ∈ S, q i| ≤ ∑ i ∈ S, |p i - q i| := by
classical
induction S using Finset.induction_on with
| empty => simp
| @insert i S hi ih =>
  have pI := hp i (Finset.mem_insert_self _ _)

```

```

have qI := hq i (Finset.mem_insert_self _ _)
have hpS := fun j hj => hp j (Finset.mem_insert_of_mem hj)
have hqS := fun j hj => hq j (Finset.mem_insert_of_mem hj)
have pS := prod_unit_interval S p hpS
rw [Finset.prod_insert hi, Finset.prod_insert hi, Finset.sum_insert hi]
have he : p i * (∏ j ∈ S, p j) - q i * (∏ j ∈ S, q j) =
  (p i - q i) * (∏ j ∈ S, p j) + q i * ((∏ j ∈ S, p j) - ∏ j ∈ S, q j) := by ring
rw [he]
calc
  _ ≤ |(p i - q i) * (∏ j ∈ S, p j)| + |q i * ((∏ j ∈ S, p j) - ∏ j ∈ S, q j)| :=
    abs_add_le _ _
  _ = |p i - q i| * (∏ j ∈ S, p j) + q i * |(∏ j ∈ S, p j) - ∏ j ∈ S, q j| := by
    rw [abs_mul, abs_of_nonneg pS.1, abs_of_nonneg qI.1]
  _ ≤ |p i - q i| + |(∏ j ∈ S, p j) - ∏ j ∈ S, q j| := by
    apply add_le_add
    · simp using mul_le_mul_of_nonneg_left pS.2 (abs_nonneg (p i - q i))
    · simp using mul_le_mul_of_nonneg_right qI.2
      (abs_nonneg ((∏ j ∈ S, p j) - ∏ j ∈ S, q j))
  _ ≤ _ := add_le_add (le_refl _) (ih hpS hqS)

def roundedProduct (mulR : ℝ → ℝ → ℝ) : List ℝ → ℝ
| [] => 1
| x :: xs => mulR x (roundedProduct mulR xs)

def UnitInterval (x : ℝ) : Prop := 0 ≤ x ∧ x ≤ 1

theorem roundedProduct_range (mulR : ℝ → ℝ → ℝ)
  (hclosed : ∀ x y, UnitInterval x → UnitInterval y → UnitInterval (mulR x y))
  (xs : List ℝ) (hxs : ∀ x ∈ xs, UnitInterval x) : UnitInterval (roundedProduct mulR xs) := by
  induction xs with
  | nil => exact ⟨zero_le_one, le_refl⟩
  | cons x xs ih =>
    exact hclosed x _ (hxs x (List.mem_cons_self ..))
    (ih (fun y hy => hxs y (List.mem_cons_of_mem x hy)))

/-- An actual recursively rounded product has a linear absolute error bound, derived from
local multiplication contracts. IEEE arithmetic is not identified with mulR without proof. -/
theorem roundedProduct_error (mulR : ℝ → ℝ → ℝ) (δ : ℝ)
  (hclosed : ∀ x y, UnitInterval x → UnitInterval y → UnitInterval (mulR x y))
  (hround : ∀ x y, UnitInterval x → UnitInterval y → |mulR x y - x * y| ≤ δ)
  (xs : List ℝ) (hxs : ∀ x ∈ xs, UnitInterval x) :
  |roundedProduct mulR xs - xs.prod| ≤ xs.length * δ := by
  induction xs with
  | nil => simp [roundedProduct]
  | cons x xs ih =>
    have hx := hxs x (List.mem_cons_self ..)
    have ht := fun y hy => hxs y (List.mem_cons_of_mem x hy)
    have hrangle := roundedProduct_range mulR hclosed xs ht
    change |mulR x (roundedProduct mulR xs) - x * xs.prod| ≤ (xs.length + 1 : Nat) * δ
    calc
      _ ≤ |mulR x (roundedProduct mulR xs) - x * roundedProduct mulR xs| +
        |x * roundedProduct mulR xs - x * xs.prod| := abs_sub_le _ _ _
      _ ≤ δ + |roundedProduct mulR xs - xs.prod| := by
        apply add_le_add (hround x _ hx hrangle)
        rw [← mul_sub, abs_mul, abs_of_nonneg hx.1]
        simp using mul_le_mul_of_nonneg_right hx.2 (abs_nonneg (roundedProduct mulR xs - xs.prod))
      _ ≤ δ + xs.length * δ := add_le_add (le_refl _) (ih ht)
      _ = _ := by push_cast; ring

theorem push_l1 {B : Type} [Fintype B] [DecidableEq B] (f : A → B) (p q : A → ℝ) :
  l1 (pushWeight f p) (pushWeight f q) ≤ l1 p q := by
  have he : ∀ b, pushWeight f p b - pushWeight f q b =
    ∑ a, if f a = b then p a - q a else 0 := by
    intro b
    unfold pushWeight
    rw [← Finset.sum_sub_distrib]

```

```

    apply Finset.sum_congr rfl
    intro a _
    split_ifs <=> simp
  unfold l1
  simp_rw [he]
  calc
    - ≤ ∑ b, ∑ a, |if f a = b then p a - q a else 0| :=
      Finset.sum_le_sum fun b _ => Finset.abs_sum_le_sum_abs _ _
    - = _ := by
      rw [Finset.sum_comm]
      apply Finset.sum_congr rfl
      intro a _
      change (∑ b : B, |if f a = b then p a - q a else 0|) = |p a - q a|
      simp [apply_ite]

open FinBayesNet

theorem evidence_l1 {bn : FinBayesNet} (p q : bn.Assignment → ℝ)
  (E : Finset bn.V) (observed : bn.Assignment) :
  l1 (likelihoodWeight p E observed) (likelihoodWeight q E observed) ≤ l1 p q := by
  apply Finset.sum_le_sum
  intro x _
  by_cases he : agrees E observed x
  · have hi : indicator E observed x = 1 := if_pos he
    simp only [likelihoodWeight, hi, mul_one, le_refl]
  · have hi : indicator E observed x = 0 := if_neg he
    simp only [likelihoodWeight, hi, mul_zero, sub_self, abs_zero]
    exact abs_nonneg _

/-- Entrywise CPT perturbations imply a joint-weight L1 budget. This is an explicit
worst-case finite bound, not an assertion about machine arithmetic. -/
theorem joint_l1_error {bn : FinBayesNet} (κ κ' : bn.Kernel ℝ)
  (hκ : ∀ m x a, 0 ≤ κ m x a ∧ κ m x a ≤ 1)
  (hκ' : ∀ m x a, 0 ≤ κ' m x a ∧ κ' m x a ≤ 1)
  (ε : bn.M → ℝ) (herr : ∀ m x a, |κ m x a - κ' m x a| ≤ ε m) :
  l1 (joint κ) (joint κ') ≤ (Fintype.card bn.Assignment : ℝ) * ∑ m, ε m := by
  calc
    - ≤ ∑ _x : bn.Assignment, ∑ m, ε m := by
      apply Finset.sum_le_sum
      intro x _
      exact (product_error Finset.univ (fun m => κ m x (x (bn.target m)))
        (fun m => κ' m x (x (bn.target m))) (fun m _ => hκ m x _) (fun m _ => hκ' m x _)).trans
        (Finset.sum_le_sum fun m _ => herr m x _))
    - = _ := by simp

/-- A complete query-posterior input-perturbation contract. The exact VE/JT results inherit
this bound through their separately proved equality to the joint/query semantics. -/
theorem kernel_query_posterior_error {bn : FinBayesNet} (κ κ' : bn.Kernel ℝ)
  (hκ : ∀ m x a, 0 ≤ κ m x a ∧ κ m x a ≤ 1)
  (hκ' : ∀ m x a, 0 ≤ κ' m x a ∧ κ' m x a ≤ 1)
  (ε : bn.M → ℝ) (herr : ∀ m x a, |κ m x a - κ' m x a| ≤ ε m)
  (Q E : Finset bn.V) (observed : bn.Assignment) (η : ℝ) (hη : 0 < η)
  (hfloor : η ≤ mass (likelihoodWeight (joint κ) E observed))
  (hsmall : (Fintype.card bn.Assignment : ℝ) * (∑ m, ε m) < η) :
  0 < mass (likelihoodWeight (joint κ') E observed) ∧
  l1
    (fun q => pushWeight (restrictTo Q) (likelihoodWeight (joint κ) E observed) q /
      mass (likelihoodWeight (joint κ) E observed))
    (fun q => pushWeight (restrictTo Q) (likelihoodWeight (joint κ') E observed) q /
      mass (likelihoodWeight (joint κ') E observed)) ≤
  2 * ((Fintype.card bn.Assignment : ℝ) * ∑ m, ε m) / η := by
  have hnonneg := likelihoodWeight_nonneg (joint κ') (joint_nonneg κ' (fun m x a => (hκ' m x a).1)) E
  < observed
  have herror := (push_l1 (restrictTo Q) (likelihoodWeight (joint κ) E observed)
    (likelihoodWeight (joint κ') E observed)).trans
    ((evidence_l1 (joint κ) (joint κ') E observed).trans (joint_l1_error κ κ' hκ hκ' ε herr))

```

```

have h := posterior_error - (pushWeight_nonneg - hnonneg) η - hn
  (by simp only [mass_pushWeight] using hfloor) herror hsmall
simp only [mass_pushWeight] using h

end
end BayesianNetworksProofs.Numerical

```

Nonvacuity and boundary checks

```

import BayesianNetworksProofs.Finite.ReferenceTables
import BayesianNetworksProofs.Finite.JunctionTree
import BayesianNetworksProofs.Finite.DSeparation
import BayesianNetworksProofs.Finite.NumericalContracts
import Mathlib.Tactic.FinCases

```

The raw certificate has shuffled state/input row IDs and two repeated parent slots. The junction example has three disconnected components and impossible evidence in a component other than the query. The graphical example is a three-variable fork: its children are d-separated given their parent, by an actual edgeless cut-moral graph argument.

```

set_option autoImplicit false

namespace BayesianNetworksProofs.RefinementExamples

open FinBayesNet FinBayesNet.Factor Junction

@[reducible] def raw : Raw.Network where
  nv := 2
  ns := 4
  nm := 2
  ni := 2
  vars v := (if v = 0 then "A" else "B", .none)
  states s := if s = 0 then ⟨0, 1, "a1"⟩ else if s = 1 then ⟨0, 0, "a0"⟩ else
    if s = 2 then ⟨1, 0, "b0"⟩ else ⟨1, 1, "b1"⟩
  mechanisms m := ⟨m, if m = 0 then "prior" else "child",
    .named (if m = 0 then "prior" else "child")⟩
  inputs i := ⟨1, 0, if i = 0 then 1 else 0⟩
  rank := id

def bank : Raw.Bank := [
  ⟨.named "prior", ⟨[], 2, [⟨[], [1/2, 1/2]⟩], [], ["a0", "a1"]⟩⟩,
  ⟨.named "child", ⟨[2,2], 2,
    [⟨[0,0], [1,0]⟩, ⟨[0,1], [0,1]⟩, ⟨[1,0], [1/2,1/2]⟩, ⟨[1,1], [0,1]⟩],
    [["a0","a1"], ["a0","a1"], ["b0","b1"]⟩]⟩⟩

theorem raw_structure_checked : raw.check = true := by decide
theorem raw_references_checked : raw.checkReady bank = true := by decide +kernel
theorem raw_normalization_checked : raw.checkNormalized bank = true := by decide +kernel
theorem raw_two_slots : raw.inputCount 1 = 2 := by decide

def badPositions : Raw.Network := {raw with inputs := fun _ => ⟨1, 0, 0⟩}
theorem bad_positions_rejected : badPositions.check = false := by decide +kernel

def badLabels : Raw.Bank := bank.map fun b =>
  if b.ref = .named "prior" then {b with table := {b.table with outputLabels := ["wrong", "a1"]}}
  else b
theorem bad_labels_rejected : raw.checkReady badLabels = false := by decide +kernel

def unnormalized : Raw.Bank := bank.map fun b =>
  if b.ref = .named "prior" then {b with table := {b.table with columns := [⟨[], [2/3, 2/3]⟩]}}
  else b
theorem unnormalized_data_interpretable : raw.checkReady unnormalized = true := by decide +kernel
theorem unnormalized_probability_rejected : raw.checkNormalized unnormalized = false := by decide
  +kernel

```

```

theorem unresolved_reference_rejected : raw.checkReady [] = false := by decide +kernel

noncomputable section

def rawValid : raw.Valid := (raw.check_iff).1 raw_structure_checked

theorem raw_diagonal (i j : Fin (raw.inputCount 1)) (x : (raw.compile rawValid).Assignment) :
  (raw.gather rawValid 1 x i).val = (raw.gather rawValid 1 x j).val := by
  apply raw.repeated_slots_diagonal rawValid
  simp [Raw.Network.slotVariable, raw]

theorem raw_compiled_probability :
   $\sum x$ , joint (raw.kernel rawValid (raw.resolvedCPT rawValid bank)) x = 1 :=
  raw.checked_reference_normalization bank raw_structure_checked raw_references_checked
  ↪ raw_normalization_checked

@[reducible] def fork : FinBayesNet where
  V := Fin 3
  M := Fin 3
  states _ := Bool
  target := id
  parents m := if m = 0 then  $\emptyset$  else {0}

theorem fork_closed : fork.Closed := ⟨Function.injective_id, Function.surjective_id⟩

def fork_order : fork.TopoOrder where
  order := [0,1,2]
  nodup := by decide
  complete := by decide
  parents_before := by decide
  no_self := by decide

def forkKernel : fork.Kernel R := fun (m : Fin 3) (x : Fin 3 → Bool) (a : Bool) ⇒
  if m = 0 then 1/2 else if a = x 0 then 1 else 0

theorem fork_local :  $\forall m$ , Local forkKernel m := by
  intro m x y h
  funext a
  by_cases hm : m = 0
  · simp [forkKernel, hm]
  · have h0 := h 0 (by simp [fork, hm])
    simp [forkKernel, hm, h0]

theorem fork_normalized :  $\forall m$ , Normalised forkKernel m := by
  intro m x
  change ( $\sum a : \text{Bool}$ , forkKernel m x a) = 1
  by_cases hm : m = 0 <=> simp [forkKernel, hm]

theorem fork_nonnegative :  $\forall m \times a$ , 0 ≤ forkKernel m x a := by
  intro m x a
  unfold forkKernel
  split_ifs <=> norm_num

theorem fork_cut_edgeless (U : Finset fork.V) : cutMoral U {0} = 1 := by
  ext v w
  constructor
  · intro he
    obtain ⟨hne, m, _, hv, hw⟩ := he.1
    have hs :  $\forall m : \text{fork.M}$ ,  $\forall v \ w$ ,  $v \in \text{family fork } m \setminus \{0\} \rightarrow w \in \text{family fork } m \setminus \{0\} \rightarrow v = w$  := by
      decide
    exact hne (hs m v w (Finset.mem_sdiff.2 ⟨hv, he.2.1⟩) (Finset.mem_sdiff.2 ⟨hw, he.2.2⟩))
  · intro he
    exact False.elim he

theorem fork_graph_separated : DSeparated (bn := fork) {1} {2} {0} := by
  intro a ha b hb hpath

```

```

have ha' : a = 1 := by simp using ha
have hb' : b = 2 := by simp using hb
subst a
subst b
rw [fork_cut_edgeless, SimpleGraph.reachable_bot] at hpath
exact (by decide : (1 : Fin 3) ≠ 2) hpath

theorem fork_conditional_independence :
  ConditionalIndependent (joint forkKernel) {1} {2} {0} :=
  d_separation_sound forkKernel fork_closed fork_order fork_local fork_normalized
  {1} {2} {0} (by decide) (by decide) fork_graph_separated

def forkFactor (m : fork.M) : Factor fork R := Factor.mechanism forkKernel fork_local m

def connectedTree : Tree fork.V fork.M :=
  .branch {0,1} [0,1] (.leaf {0,2} [2]) (.leaf {0,1} [])

theorem connected_tree_valid : Good forkFactor connectedTree := by
  simp [Good, connectedTree, forkFactor, localFactor, combine, multiply, unit,
    Factor.mechanism, fork, Tree.vars, Tree.bag]
  decide

theorem connected_assignment : connectedTree.indices.Perm (Finset.univ : Finset fork.M).toList := by
  apply (List.perm_ext_iff_of_nodup (by decide) (Finset.nodup_toList _)).2
  intro i
  fin_cases i <|> simp [connectedTree, Tree.indices]

theorem connected_tree_check : checkGood forkFactor connectedTree = true := by decide +kernel

theorem connected_assignment_check : checkAssignment connectedTree = true := by decide +kernel

theorem connected_variables : ∀ v : fork.V, ∃ m, v ∈ (forkFactor m).scope :=
  fun v => ⟨v, Finset.mem_insert_self _ _⟩

theorem nonempty_separator_belief_exists :
  ∃ b ∈ calibrate forkFactor connectedTree, b.1 = ({0,2} : Finset fork.V) := by
  simp [calibrate, connectedTree, prepare, distribute]

theorem connected_beliefs_match_ve (b : Finset fork.V × Factor fork R)
  (hb : b ∈ calibrate forkFactor connectedTree) (vs : List fork.V)
  (hv : vs.Nodup) (hs : vs.toFinset = b.1c) (x : fork.Assignment) :
  b.2.value x = Factor.product (Factor.eliminateAll (Finset.univ.toList.map forkFactor) vs) x :=
  calibrate_eq_ve forkFactor connectedTree connected_tree_valid connected_assignment
  connected_variables b hb vs hv hs x

@[reducible] def roots : FinBayesNet := {fork with parents := fun _ => ∅}

def rootKernel : roots.Kernel R := fun (m : Fin 3) (_ : Fin 3 → Bool) (a : Bool) =>
  if m = 1 then (if a = false then 1 else 0) else 1/2

theorem root_local : ∀ m, Local rootKernel m := fun _ _ _ => rfl

def observed : roots.Assignment := fun _ => true

def forestFactor (i : Fin 4) : Factor roots R :=
  if h : i.val < 3 then Factor.mechanism rootKernel root_local ⟨i.val, h⟩
  else evidenceFactor {1} observed

def forestTree : Tree roots.V (Fin 4) :=
  graft ∅ [] [.leaf {0} [0], .leaf {1} [1,3], .leaf {2} [2]]

theorem forest_valid : Good forestFactor forestTree := by
  apply forest_good
  · intro t ht
    simp only [List.mem_cons, List.not_mem_nil, or_false] at ht
    rcases ht with rfl | rfl | rfl <|>

```

```

    simp [Good, localFactor, combine, multiply, unit, forestFactor, Factor.mechanism,
    evidenceFactor, roots, fork]
  · decide

theorem forest_assignment : forestTree.indices.Perm (Finset.univ : Finset (Fin 4)).toList := by
  apply (List.perm_ext_iff_of_nodup (by decide) (Finset.nodup_toList _)).2
  intro i
  fin_cases i <|> simp [forestTree, graft_indices, Tree.indices]

theorem forest_variables : ∀ v : roots.V, ∃ i, v ∈ (forestFactor i).scope := by
  intro v
  fin_cases v
  · exact ⟨0, by decide⟩
  · exact ⟨1, by decide⟩
  · exact ⟨2, by decide⟩

theorem forest_nonnegative : ∀ i x, 0 ≤ (forestFactor i).value x := by
  intro i x
  fin_cases i
  all_goals simp [forestFactor, Factor.mechanism, rootKernel, evidenceFactor, indicator, agrees]
  all_goals split_ifs <|> norm_num

theorem product_zero_of_mem {fs : List (Factor roots R)} {f : Factor roots R}
  (hf : f ∈ fs) (x : roots.Assignment) (hz : f.value x = 0) : Factor.product fs x = 0 := by
  induction fs with
  | nil => simp at hf
  | cons g fs ih =>
    rw [Factor.product_cons]
    rcases List.mem_cons.1 hf with rfl | hf
    · rw [hz, zero_mul]
    · rw [ih hf, mul_zero]

theorem forest_zero_mass (x : roots.Assignment) :
  Factor.product ((Finset.univ : Finset (Fin 4)).toList.map forestFactor) x = 0 := by
  cases hx : x 1
  · apply product_zero_of_mem (f := forestFactor 3)
    · exact List.mem_map.2 ⟨3, Finset.mem_toList.2 (Finset.mem_univ _), rfl⟩
    · simp [forestFactor, evidenceFactor, indicator, agrees, observed, hx]
  · apply product_zero_of_mem (f := forestFactor 1)
    · exact List.mem_map.2 ⟨1, Finset.mem_toList.2 (Finset.mem_univ _), rfl⟩
    · simp [forestFactor, Factor.mechanism, rootKernel, roots, fork, hx]

/-- Even a query in the other disconnected component must reject the globally impossible event. -/
theorem disconnected_query_rejects (b : Finset roots.V × Factor roots R)
  (hb : b ∈ calibrate forestFactor forestTree) (Q : Finset roots.V) (hQ : Q ⊆ b.1)
  (base : roots.Assignment) :
  queryPosterior forestFactor forestTree forest_valid forest_assignment forest_variables
    forest_nonnegative b hb Q hQ base = none := by
  rw [queryPosterior_none_iff]
  simp [forest_zero_mass]

end
end BayesianNetworksProofs.RefinementExamples

```

Roadmap

```

import BayesianNetworksProofs.Finite.OpenSemantics
import BayesianNetworksProofs.Finite.RefinementExamples

```

The former `osem_compose` hole is now proved in `Finite/OpenSemantics.lean`, in the default target and axiom audit. Its original premises are retained by a compatibility theorem. The stronger `osem_compose_glue` needs only disjoint input/output sets on both sides: locality, normalisation and total interface matching are unnecessary for this finite-sum identity.

Concrete finite records/references and repeated slots, genuine conditioned distributions, the actual Shafer-Shenoy collect/distribute computation, moralized-ancestral d-separation soundness, and conditional numerical error contracts now have checked developments. The remaining boundary is implementation translation: Julia/JSON decoding and reference binding, concrete CliqueTrees/Bayes-ball code, ordered-array layout and IEEE arithmetic adapters are not established merely by those finite-model theorems.

The general structural open-network category, including non-injective output legs, pass-through and coherent copy/discard, is now constructed separately in `CategoricalBayesianNetworks.jl/proofs/` (ADR 0010), together with its numerical semantics: a strong braided monoidal functor into `FinStoch` preserving copy and discard (`OpenNet.Interpretation.functor`), whose composition formula `Interpretation.kernel_comp` covers copied and pass-through outputs. This project's own `OpenFinBayesNet` now has its pass-through formula too: `osem_compose_passthrough` in `Finite/OpenSemantics.lean` sums only the glued variables the composite hides and needs no hypotheses; `osem_compose_glue` is its disjoint-interface special case. The category of stochastic kernels is a different result, provided by `FiniteKernels.jl/proofs/FiniteKernelsProofs/Theory/FinStoch.lean`.